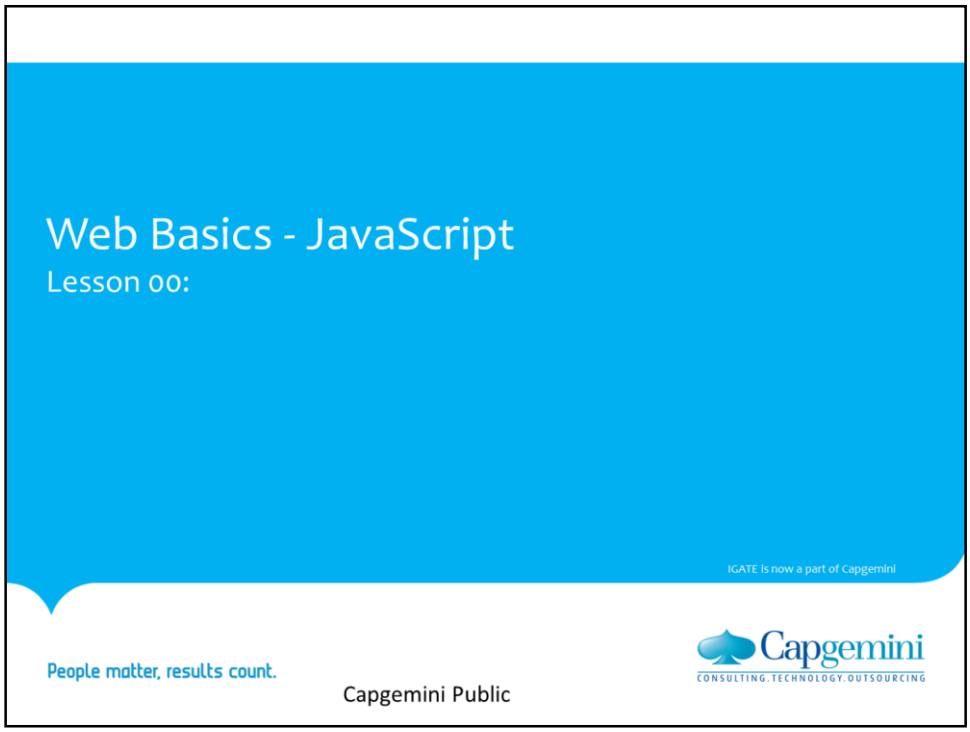
Instructor Notes:



Instructor Notes:

Document History

Date	Course Version No.	Software Version No.	Developer / SME	Change Record Remarks
19-Sep-2009	2.0	NA	Pradnya Jagtap	Updated to new template. And incorporated new examples.
Apr-2011	3.0	NA	Anu Mitra	Integration updates
Apr-2015	3.1	NA	Rathnajothi P	Revamped according to revised curriculum
May-2016	3.2	NA	Kavita Arora	Revamped according to revised curriculum

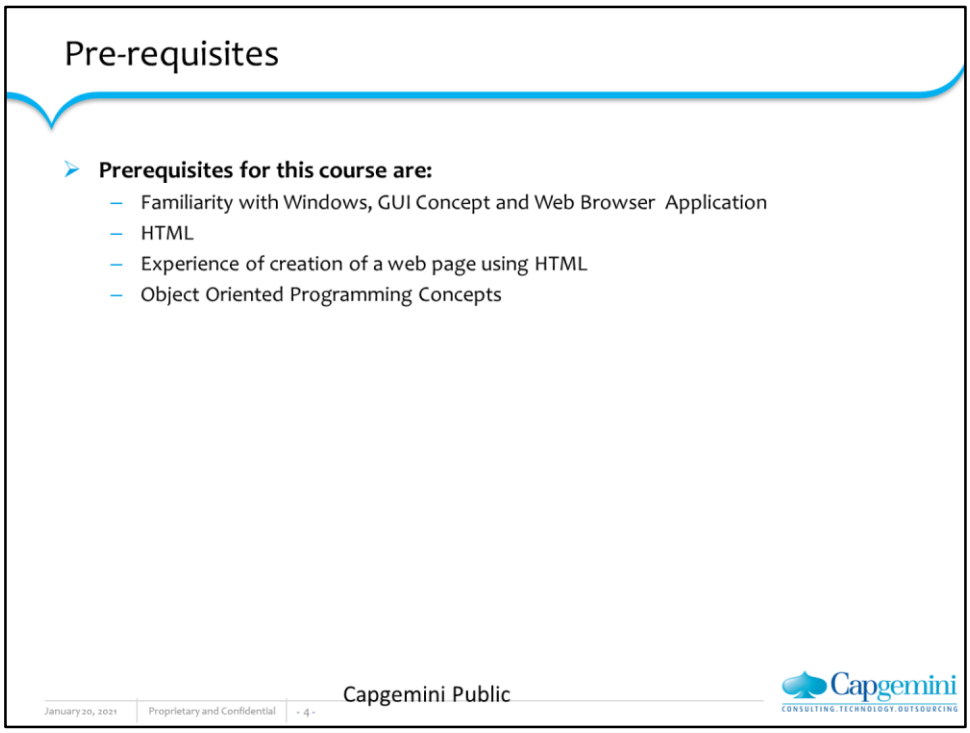
Instructor Notes:

Course Goals and Non Goals

- **Course Goals:**
  - At the end of this course you will be able to:
    - Add interactivity to the static HTML pages
    - Validate the input data provided by users, on the client side
    - Manipulate style sheets on the fly to give a sophisticated look to any website

- **Course Non Goals:**
  - Server-side scripting not covered.

Instructor Notes:

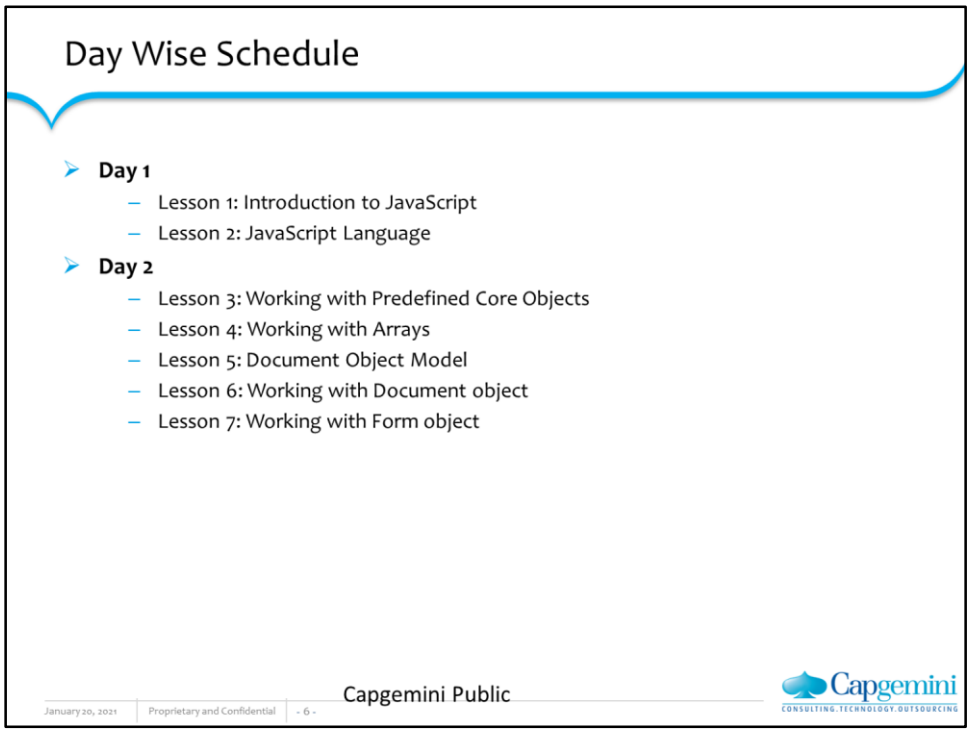


Instructor Notes:

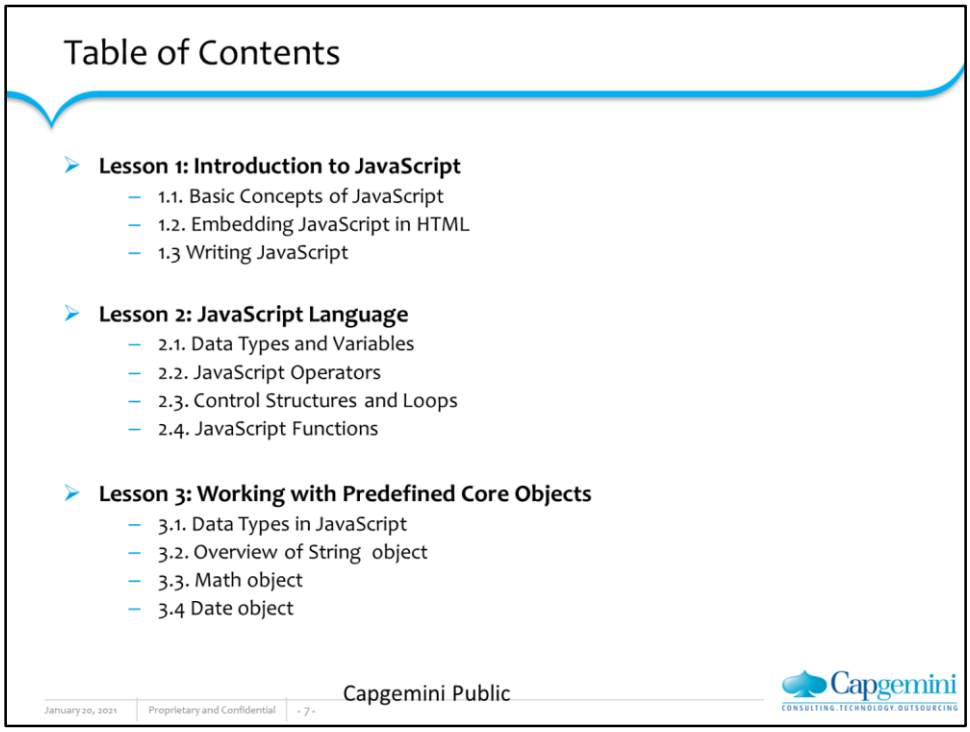
Intended Audience

- **This course is designed for:**
  - Trainee Programmers
  - Software Professionals

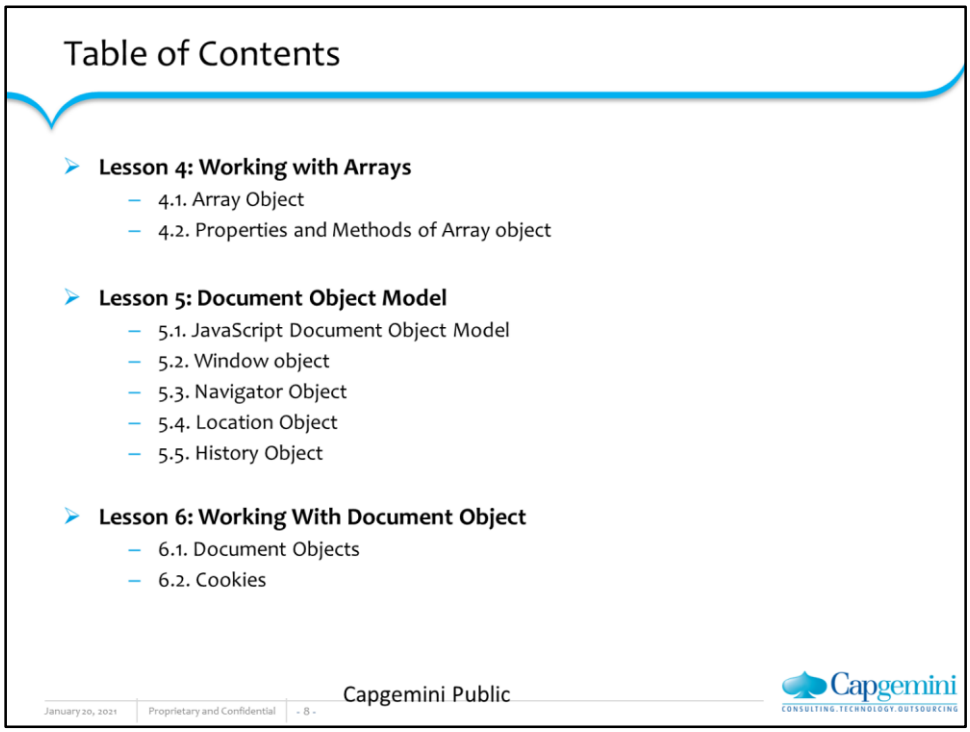
Instructor Notes:



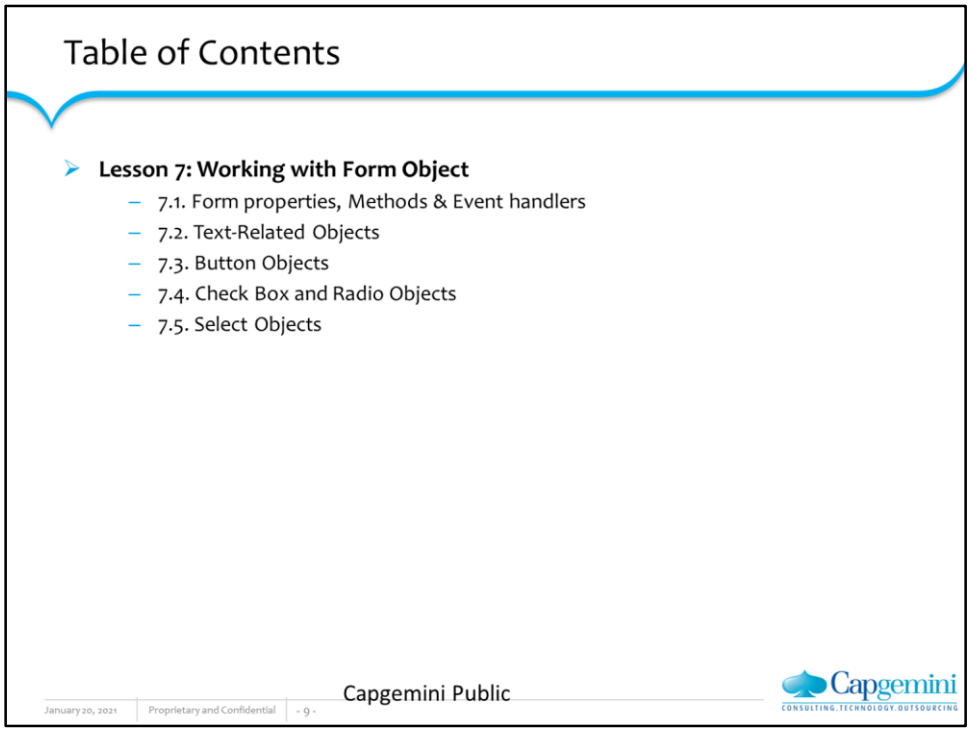
Instructor Notes:



Instructor Notes:



Instructor Notes:



Instructor Notes:

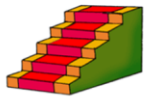
References

- JavaScript: A Beginner's Guide - by John Pollock
- http://www.w3schools.com/

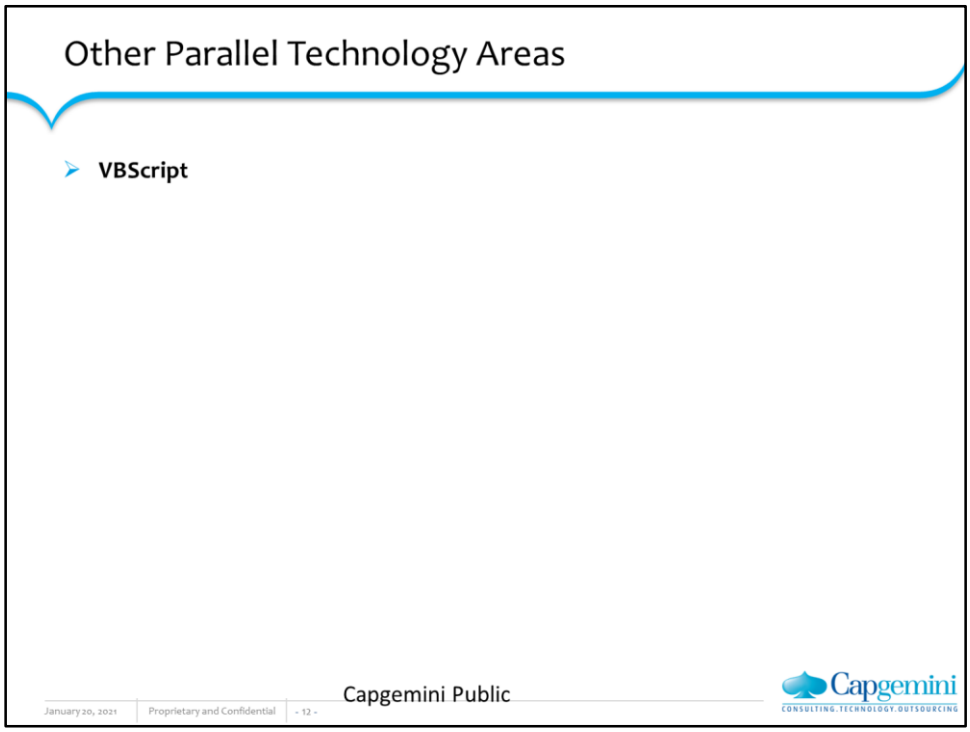
Instructor Notes:

Next Step Courses (if applicable)

- Servlet
- JSP



Instructor Notes:



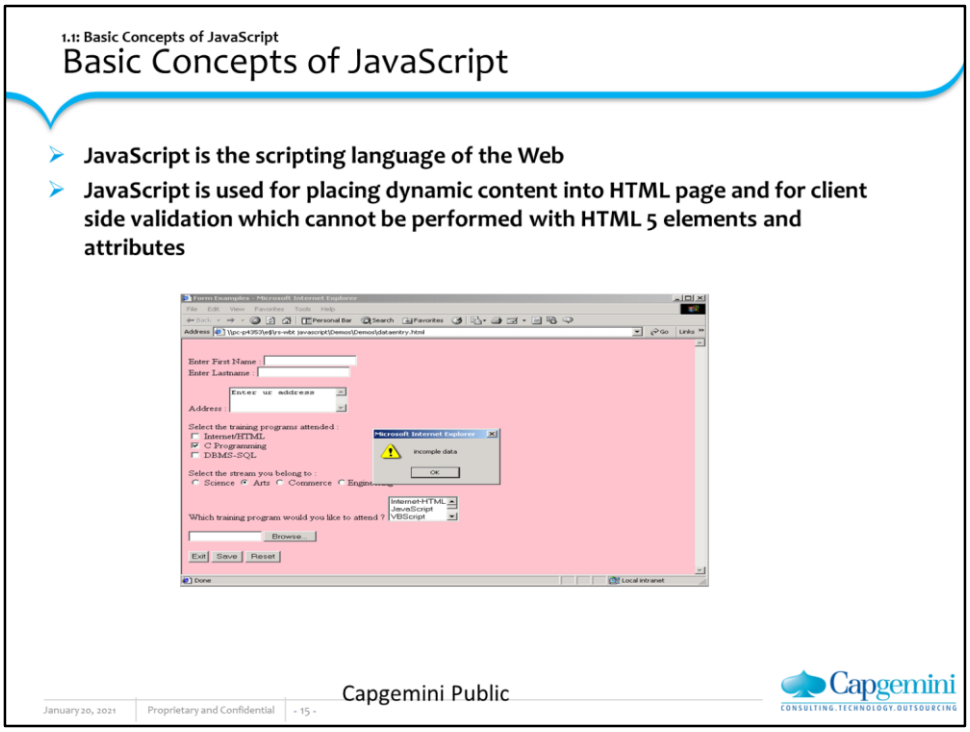
Instructor Notes:

Instructor Notes:

Lesson Objectives

- **To understand the following topics:**
  - Basic Concepts of JavaScript
  - Embedding JavaScript in HTML
  - Writing JavaScript

Instructor Notes:



Basic Concepts of JavaScript

JavaScript History :

Web pages made using only HTML are somewhat static with no interactivity and negligible user involvement.

HTML tags are just instructions on document and the display of the document is dependent on the browser.

Interactive pages cannot be built with only HTML, we need a programming language.

So Netscape came out with a client-side language called as JavaScript.

What is JavaScript?

JavaScript is THE scripting language of the Web.

JavaScript is used in Web pages to add functionality, validate forms, detect browsers, and much more.

JavaScript is the most popular scripting language on the internet and works on all major browsers available like IE, Firefox, Chrome etc..

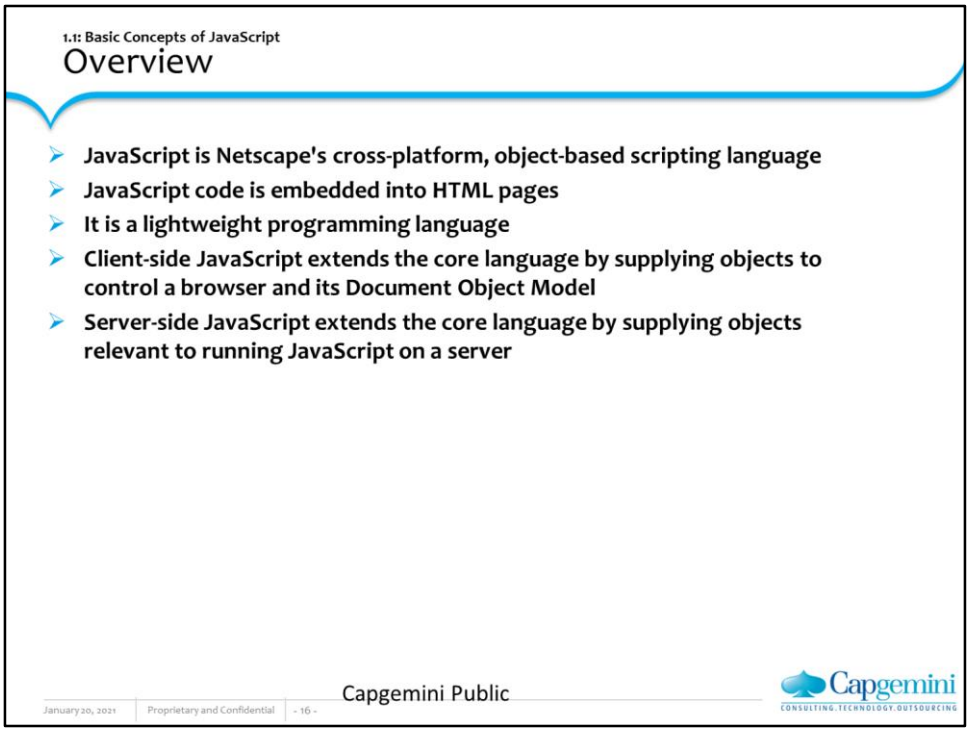
Need of JavaScript:

For placing dynamic content into an HTML Page.

For client side Validation.

For storing and retrieving client's information in the form of Cookies.

Instructor Notes:



What is JavaScript?

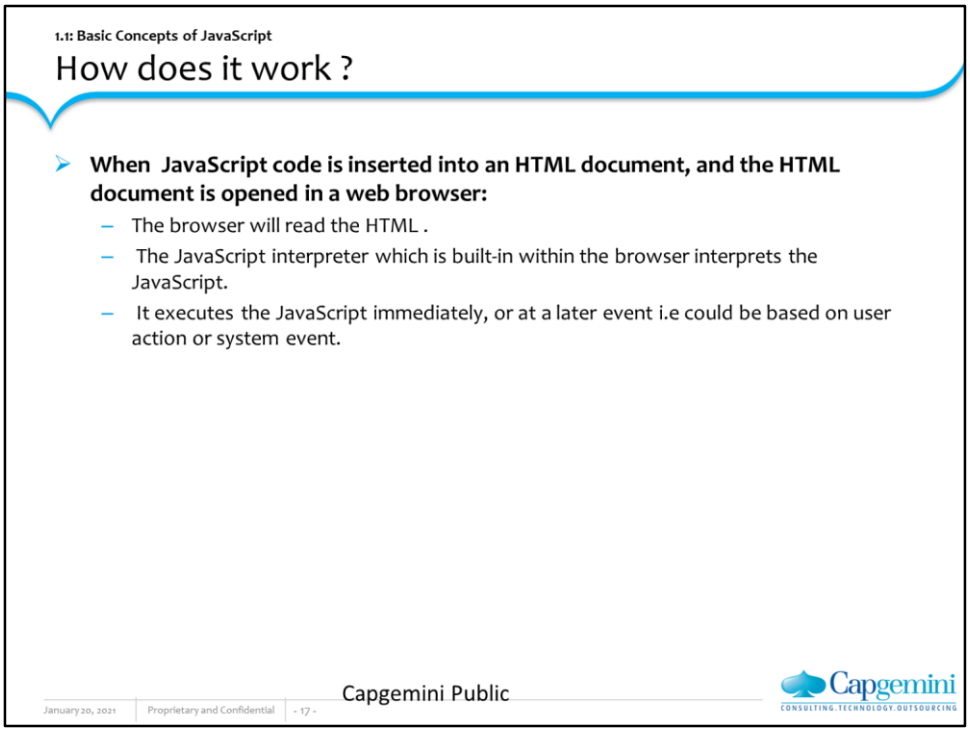
JavaScript is Netscape's cross-platform, object-based scripting language. Core JavaScript contains a core set of objects, such as Array, Date, and Math, and a core set of language elements such as operators, control structures, and statements. Core JavaScript can be extended for a variety of purposes by supplementing it with additional objects; for example:

Client-side JavaScript extends the core language by supplying objects to control a browser (Navigator or another web browser) and its Document Object Model (DOM). For example, client-side extensions allow an application to place elements on an HTML form and respond to user events such as mouse clicks, form input, and page navigation.

Server-side JavaScript extends the core language by supplying objects relevant to running JavaScript on a server. For example, server-side extensions allow an application to communicate with a relational database, provide continuity of information from one invocation to another of the application, or perform file manipulations on a server.

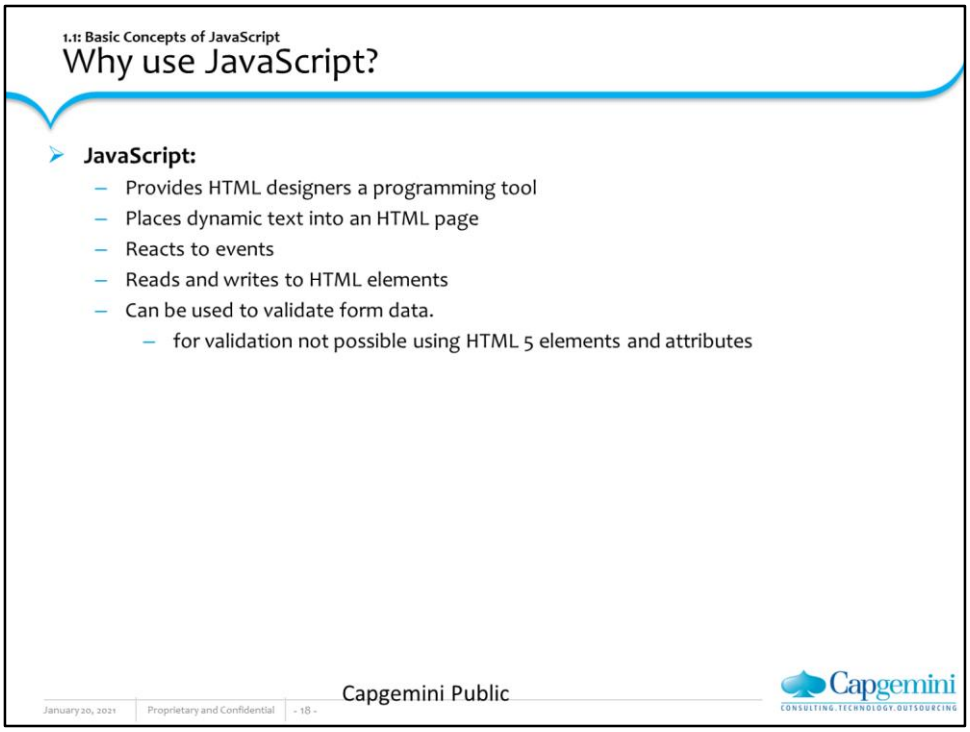
JavaScript lets you create applications that run over the Internet. Client applications run in a browser, such as Internet Explorer/Firefox, and server applications run on a server, such as Netscape Enterprise Server. Using JavaScript, you can create dynamic HTML pages that process user input and maintain persistent data using special objects, files, and relational databases.

Instructor Notes:



How does JavaScript work?
When a JavaScript is inserted into an HTML document, the JavaScript interpreter which is built-in within the Internet browser will read the HTML and interpret the JavaScript. The JavaScript can be executed immediately, or at a later event.

Instructor Notes:



Why use JavaScript?

JavaScript gives HTML designers a programming tool :

JavaScript is a simple scripting language which can be used by HTML authors who essentially are not familiar with programming. Hence the HTML authors can easily put small JavaScript code snippets into HTML pages.

JavaScript can place dynamic text into an HTML page : A simple HTML text which displays static content such as <h1>Welcome</h1> can be written in JavaScript to display dynamic content using document.write("<h1>" + orgname + "</h1>")

JavaScript can react to events : A JavaScript can be set to execute when some action takes place, like when a page has finished loading or when a user clicks on an HTML element.

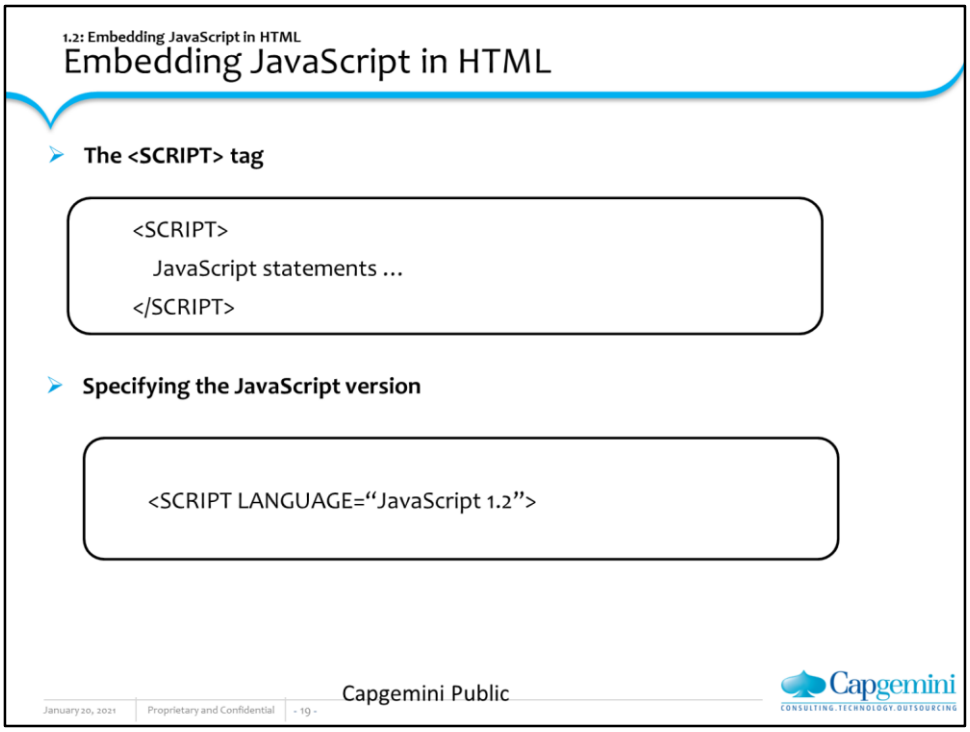
JavaScript can reads and writes HTML elements: A JavaScript can read values of HTML elements and also write/change the content of an HTML element.

JavaScript can be used to validate data : Client side validation can easily taken care of by JavaScript. This reduces the burden on the server.

With the advent of HTML 5, most of the form data validation can be easily performed using HTML 5 elements like input type=email and attributes like required, pattern, maxlength etc

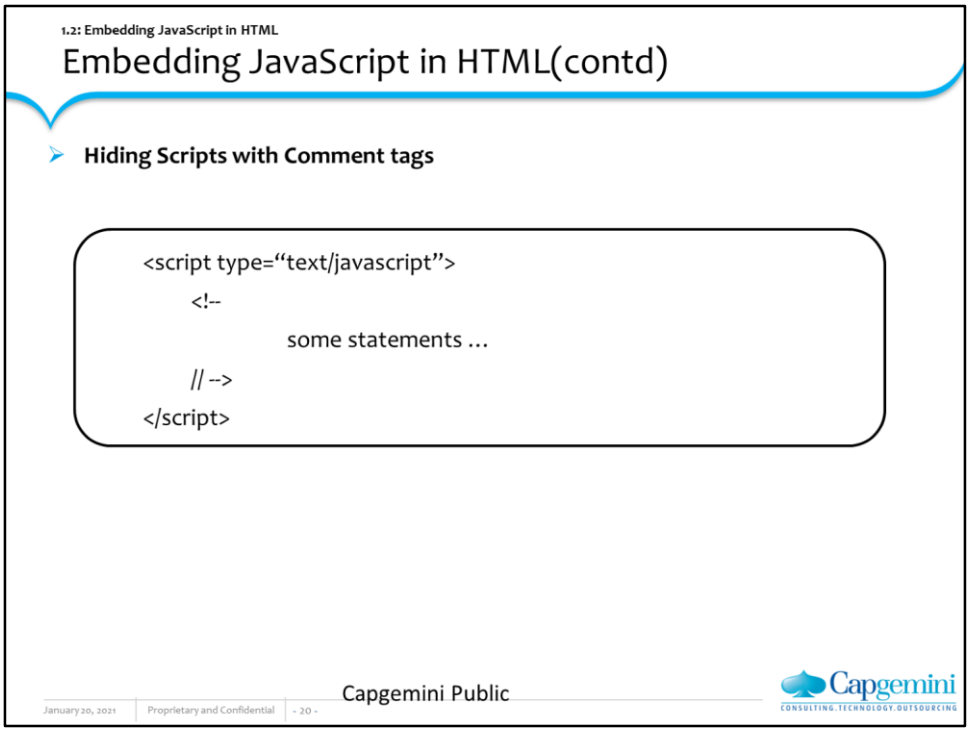
Apart from this JavaScript can also be used to create cookies which is stores and retrieves information about the user preferences. JavaScript can also be used to detect browser which helps in loading a page specifically designed for the browser.

Instructor Notes:

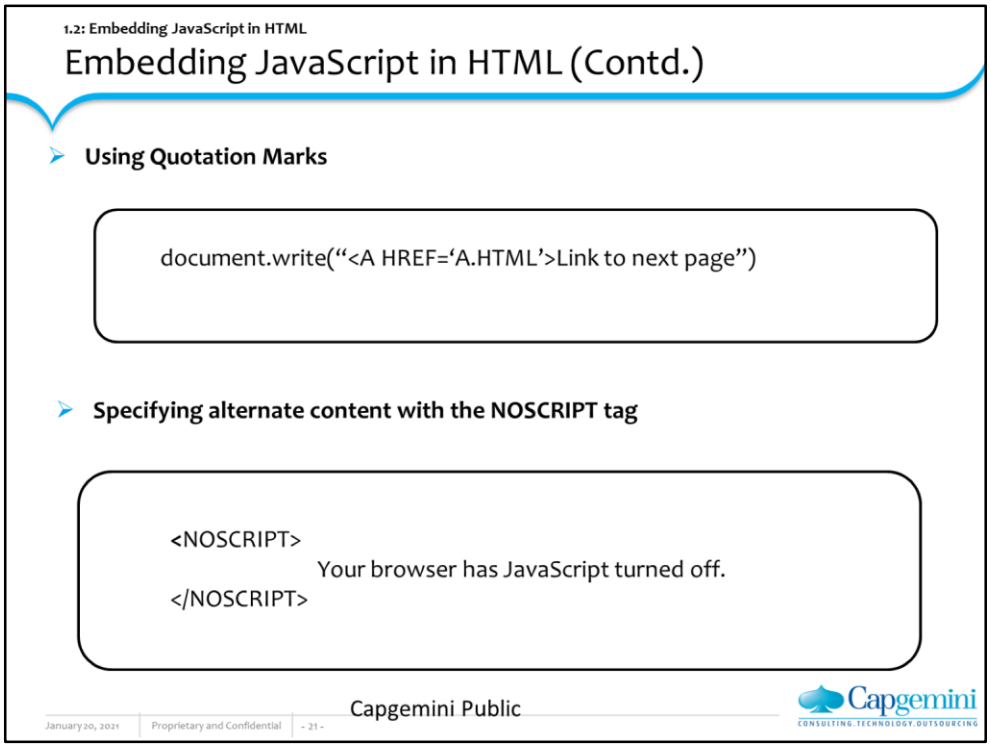


Embedding JavaScript in HTML:
The <SCRIPT> tag is an extension to HTML that can enclose any number of JavaScript statements as shown on the slide.
A document can have multiple <SCRIPT> tags, and each can enclose any number of JavaScript statements.
The Script tag has the following attributes:
Language – This attribute specifies the scripting language. It can have values like VBScript, JavaScript. Optionally you can also specify the scripting language version as mentioned on the slide.
Type – Specifies the MIME type of the scripting language
Src – This attribute is for specifying the path and filename of an external .js file which contains the script code.
Some browsers may not support JavaScript and the JavaScript code is displayed as page content. To prevent this, the HTML comment should be used which will hide the JavaScript as shown on the slide. Note the two forward slashes at the end of the comment line. This is JavaScript comment symbol which prevents JavaScript from executing the -- > tag.
Note that Javascript is a case-sensitive scripting language.

Instructor Notes:



Instructor Notes:



Embedding JavaScript in HTML (contd..):

Whenever you want to indicate a quoted string inside a string literal, use single quotation marks (') to delimit the string literal. This allows the script to distinguish the literal inside the string. In the following example,

The attribute values are in double quotes, but in the call to the function myfunc the argument passed is a string which is enclosed in a single quotes.

```
<INPUT TYPE="button" VALUE="Press Me" onClick="myfunc('astring')">.
```

Use the <NOSCRIPT> tag to specify alternate content for browsers that do not support JavaScript. HTML enclosed within a <NOSCRIPT> tag is displayed by browsers that do not support JavaScript; code within the tag is ignored by browser. In case, if the user has disabled JavaScript from the Advanced tab of the Preferences dialog, the browser displays the code within the <NOSCRIPT> tag.

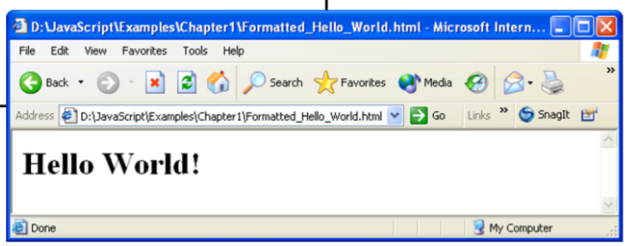
Instructor Notes:

1.2: Embedding JavaScript in HTML

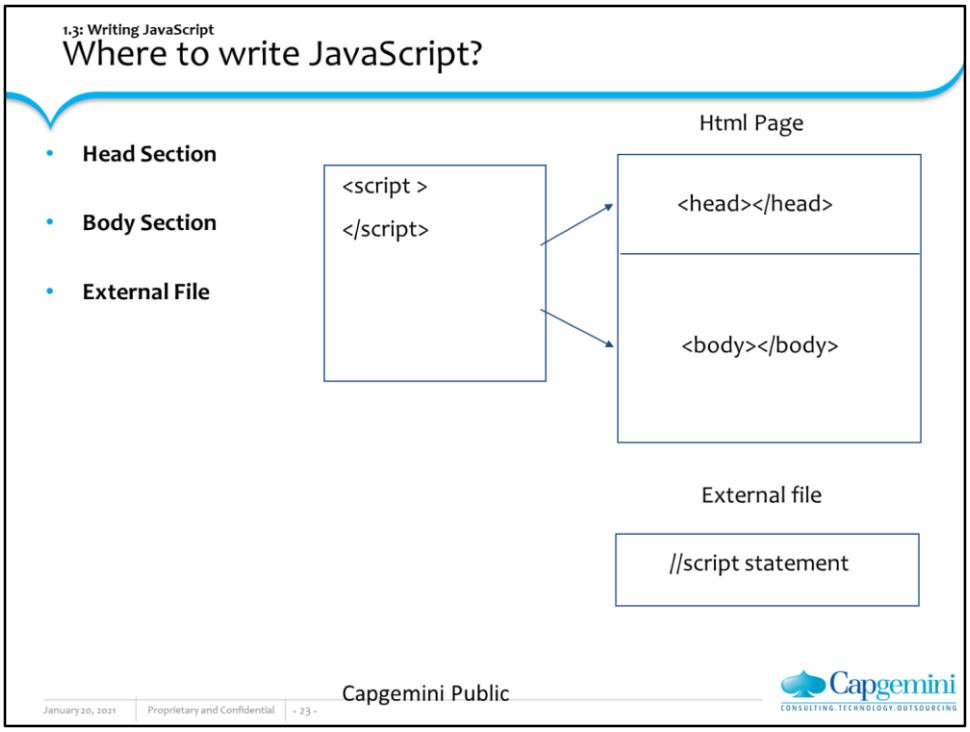
Embedding JavaScript in HTML (Contd.)

- Including text-formatting features

```
<!DOCTYPE html>
<html>
<head> </head>
<body>
<script>
document.write("<H1>Hello World!</H1>")
</script>
</body>
</html>
```



Instructor Notes:



Embedding JavaScript in HTML: Where to Write JavaScript?
Physically the script code can be placed at three different places in the html file.
Head Section: If you want the Scripts to be executed when they are called, or whenever a user action happens, put script tag in the head section.
Body Section : If you want the Scripts to be executed when the page loads then put script tag in the body section
External File : If you want to run the same JavaScript on several pages, without having to write the same script on every page, you can write a JavaScript in an external file.

Instructor Notes:

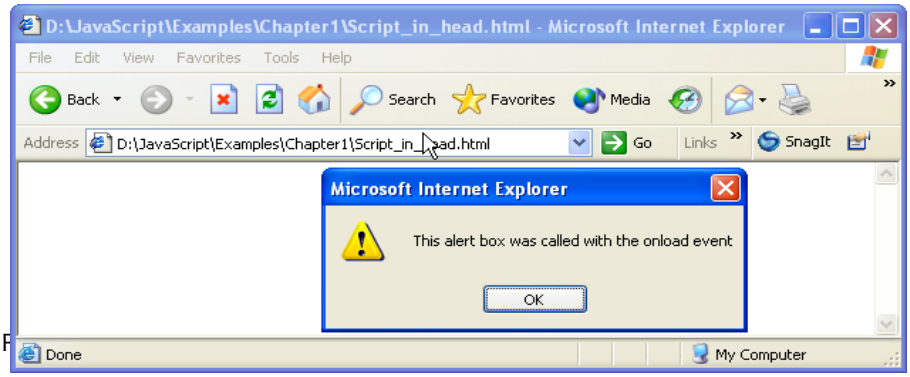
1.3: Writing JavaScript

JavaScript in Head Section

- Syntax

```
<!DOCTYPE html>
<html>
<head>
<script type="text/javascript">
function message()
    {
      alert("This alert box was called with the
            onload event")
    }
</script>
</head>
<body onload="message()">
</body>
</html>
```


Where to Write JavaScript? In the Head Section:
Head Section : JavaScripts in an HTML page will be executed when the page loads. This might always not be the case. Sometimes we want to execute a JavaScript when an event occurs, such as when a user clicks a button. In such scenarios, we can put the script inside a function. Scripts that contain functions go in the head section of the document. Then we can be sure that the script is loaded before the function is called.
The example shown on the slide shows the following output:



Instructor Notes:

1.3: Writing JavaScript

JavaScript in Body Section

- Syntax

```
<!DOCTYPE html>
<html>
<head>
<title>script tag in body</title>
</head>
<body >
<script language="javascript">
        document.write("this message is written when
the page loads")
</script>
</body>
</html>
```

Capgemini Public

January 20, 2021 | Proprietary and Confidential | - 25 -

Capgemini CONSULTING.TECHNOLOGY.OUTSOURCING

Where to Write JavaScript? In the Body Section:
Body Section : Execute a script that is placed in the body section. The example on the slide shows script written in the body section.
And it produces this output:

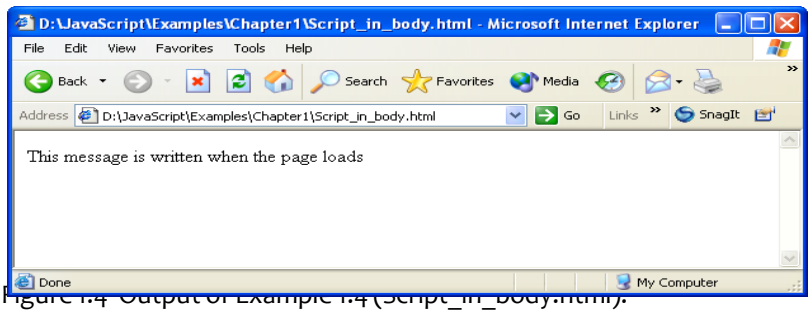
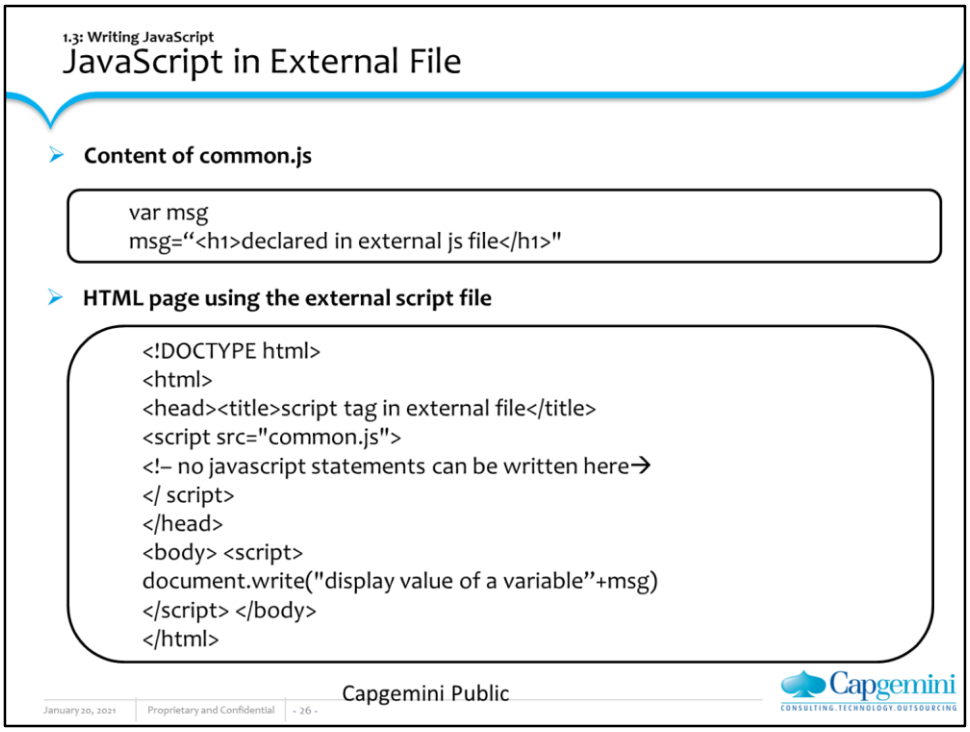


Figure 1.4 Output of Example 1.4 (Script_in_body.html).

Instructor Notes:



Where to Write JavaScript? In External File:

The example on the slide demonstrates how to write JavaScript code in an external file. The extension of the external file has to be .js and the it does not contain the script tag.

Instructor Notes:

Demo

- Hello.html
- Head_section.html
- Extern_file.html
- Comm.js
- Var_ex.html

Instructor Notes:

Lab

- **Lab 1:**
- **Basic concepts of JavaScript**

Instructor Notes:

Summary

- **JavaScript is the client side scripting language**
- **When JavaScript code is placed inside a web page, the browser loads the page & the built-in interpreter reads the script & executes the same**
- **JavaScript is used in Web pages for**
  - Validating client side data.
  - Placing dynamic content into an HTML page.
  - Storing client's information in the form of Cookies.

Instructor Notes:

Answers

1. Option 2
2. False
3. noscript

Review Question

- **Question 1: JavaScript, a scripting language, is:**
  - Option 1: Cross-platform, object-oriented
  - Option 2: Cross-platform, object-based
  - Option 3: Non cross-platform, object-oriented

- **Question 2:The <SCR> tag is an extension to HTML that can enclose any number of JavaScript statements.**
  - True/False

- **Question 3:HTML enclosed within a _____ tag is displayed by browsers that do not support JavaScript.**

Instructor Notes:

Answers

1. 3
2. 1
3. 4
4. 2

Review Question: Match the Following

1. Embed JavaScript in an HTML document as statements and functions within this tag	1. <NOSCRIPT>
2. To specify alternate content for browsers that do not support JavaScript	2. Head, Body
3. Feature of JavaScript	3. SCRIPT
4. JavaScript can be written in these sections	4. To validate and process user data

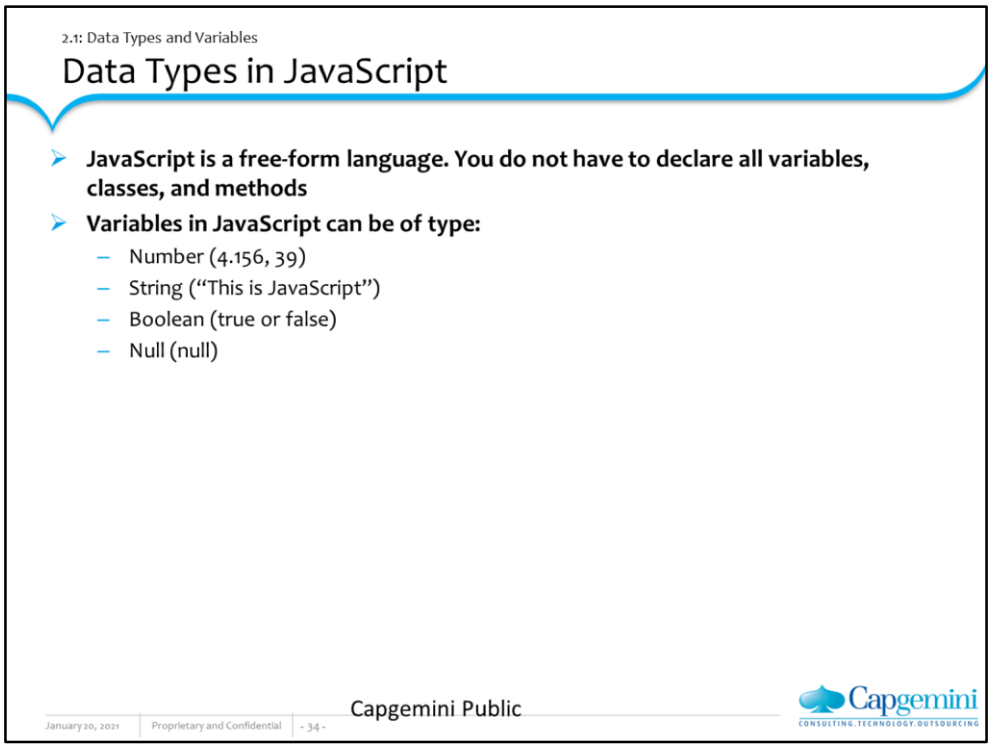
Instructor Notes:

Instructor Notes:

Lesson Objectives

- Data Types and Variables
- JavaScript Operators
- Control Structures and Loops
- JavaScript Functions

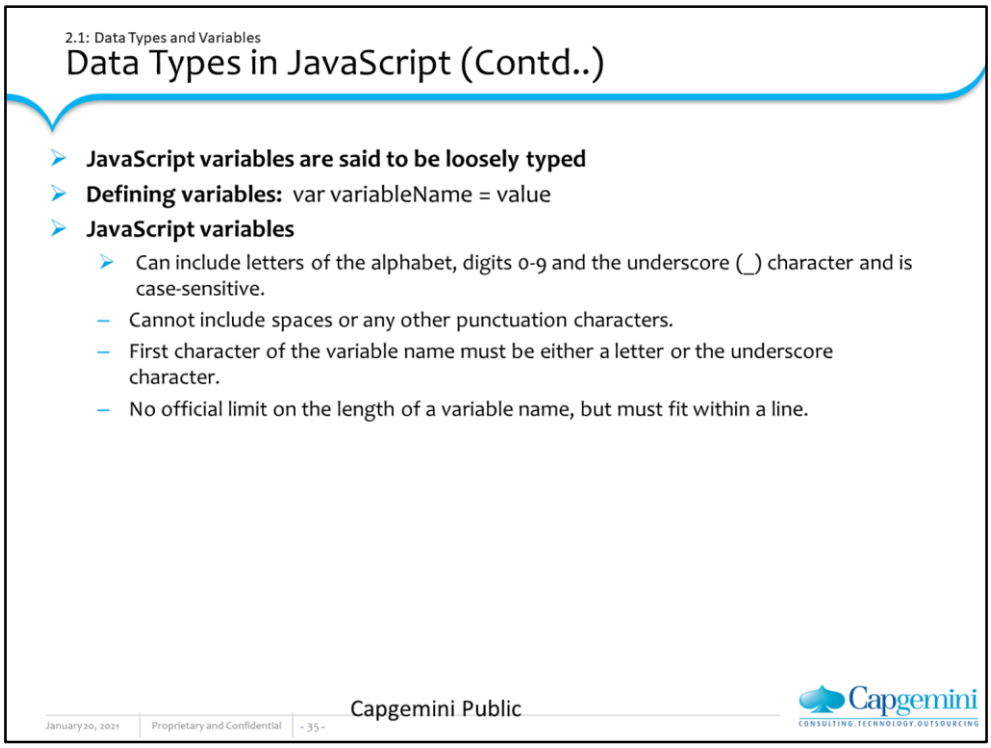
Instructor Notes:



Data Types in JavaScript:

Although the number of data types is small, they are sufficient for the tasks that JavaScript performs. Notice that there is no distinction between integers and real numbers; both types are just numbers. JavaScript does not provide an explicit data type for a date. However, there are related functions and a built-in date object that enable the Web page designer to manage dates.

Instructor Notes:



Defining Variables:

There are specific rules you must follow when choosing variable names.

Variable names can include letters of the alphabet, both upper and lowercase. They can also include the digits 0-9 and the underscore (_) character.

Variable names cannot include spaces or any other punctuation characters.

The first character of the variable name must be either a letter or the underscore character.

Variable names are case-sensitive; totalnum, Totalnum, and TotalNum are separate variable names.

There is no official limit on the length of a variable name, but it must fit within one line.

JavaScript variables are said to be loosely typed. To declare a variable for a JavaScript program, you would write this:

```
var variablename = value ;
```

Instructor Notes:

2.2: JavaScript Operators

Arithmetic Operator

Operator	Description	Example	Result
+	Addition	2 + 2	4
-	Subtraction	5 – 2	3
*	Multiplication	4 * 5	20
/	Division	5 / 2	2.5
%	Modulus	10 % 8	2
++	Increment	x = 5; x++	x = 6
--	Decrement	x = 5; x--	x = 4

Instructor Notes:

2.2: JavaScript Operators

Comparison Operator

Operator	Description	Example	Result
==	is equal to	5 == 8	false
!=	is not equal	5 != 8	true
>	is greater than	5 > 8	false
<	is less than	5 <= 8	true
>=	is greater or equal	5 >= 8	false
<=	is less or equal	5 <= 8	true

Instructor Notes:

2.2: JavaScript Operators

Assignment Operator

Operator	Example	Is same as
+=	x += y	x = x + y
-=	x -= y	x = x – y
*=	x *= y	x = x * y
/=	x /= y	x = x / y
%=	x %= y	x = x % y

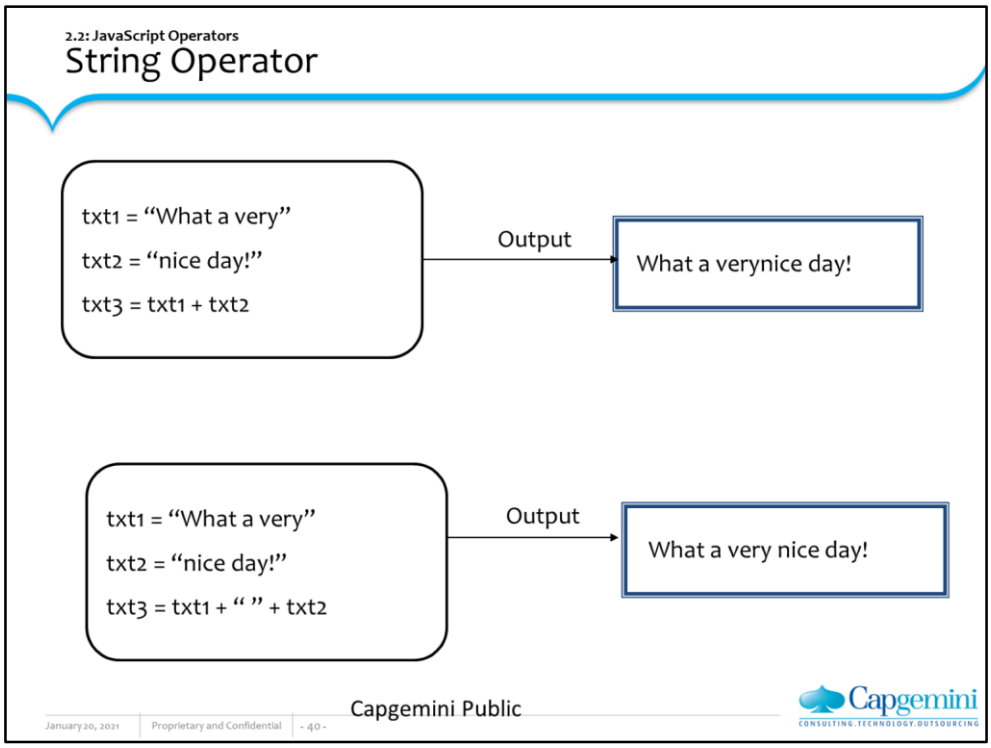
Instructor Notes:

2.2: JavaScript Operators

Logical Operator

Operator	Description	Example
&&	and	x = 6; y = 3 x < 10 && y > 1 returns true
||	or	x = 6; y = 3 x < 10 || y > 5 returns true
!	not	x = false !x returns true

Instructor Notes:



String Operator:

```
txt1="What a very"
txt2="nice day!"
txt3=txt1+txt2
```

A string is most often a text, for example "Hello World!". To stick two or more string variables together, use the + operator.
The variable txt3 now contains "What a verynice day!". To add a space between two string variables, insert a space into the expression, OR in one of the strings.

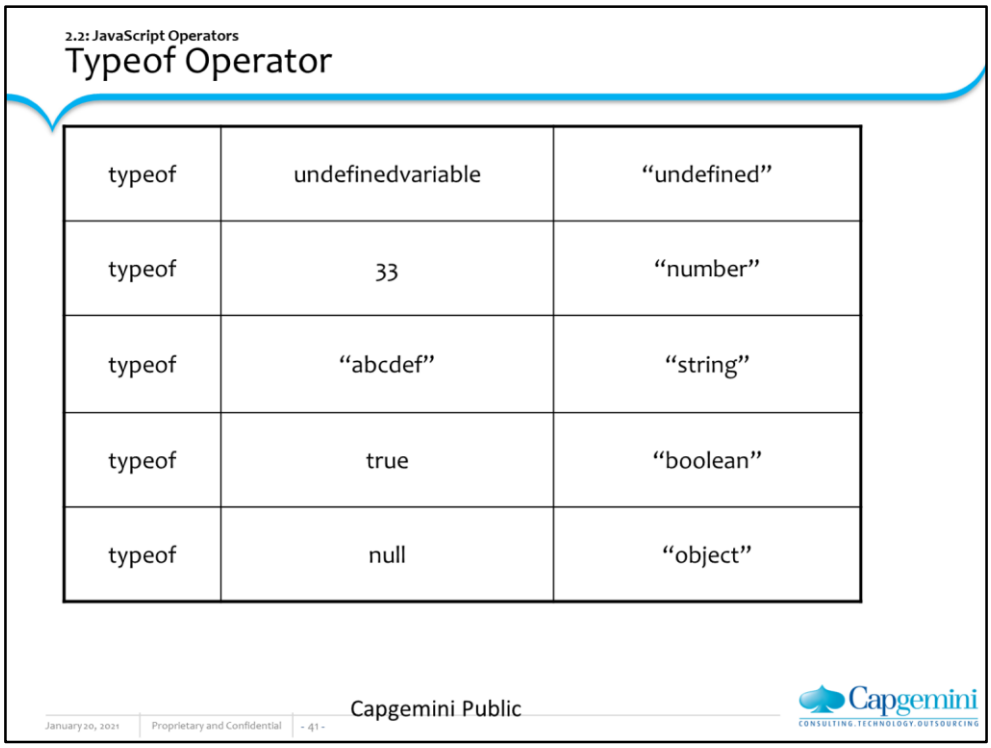
```
txt1="What a very"
txt2="nice day!"
txt3=txt1+" "+txt2
```

Or

```
txt1="What a very "
txt2="nice day!"
txt3=txt1+txt2
```

The variable txt3 now contains "What a very nice day!".

Instructor Notes:



Typeof Operator:

The typeof operator returns the type of data that its operand currently holds.

```
<HTML>
<HEAD>
<TITLE>Using typeof</TITLE>
</HEAD>
<BODY>
<SCRIPT LANGUAGE="JavaScript">
<!-- var num1=20
 var str1="abc"
 var bool1=true
 var num2=null
 var var1;
 document.write("type of str1 : "+typeof(str1)+"<BR>")
 document.write("type of num1 : "+typeof(num1)+"<BR>")
document.write("type of bool1 : "+typeof(bool1)+"<BR>")
document.write("type of num2 : "+typeof(num2)+"<BR>") -->
</SCRIPT>
</BODY>
</HTML>
```

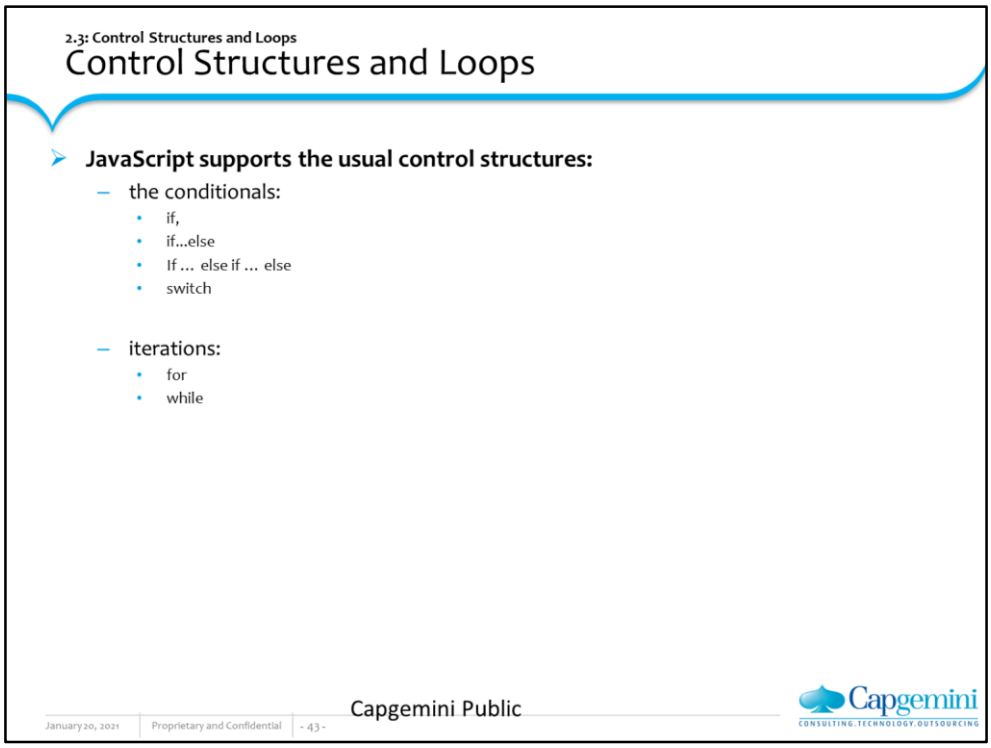
Example 2.1 typeof operator (typeof_ex.html)

Instructor Notes:

Demo

- Typeof_ex.html

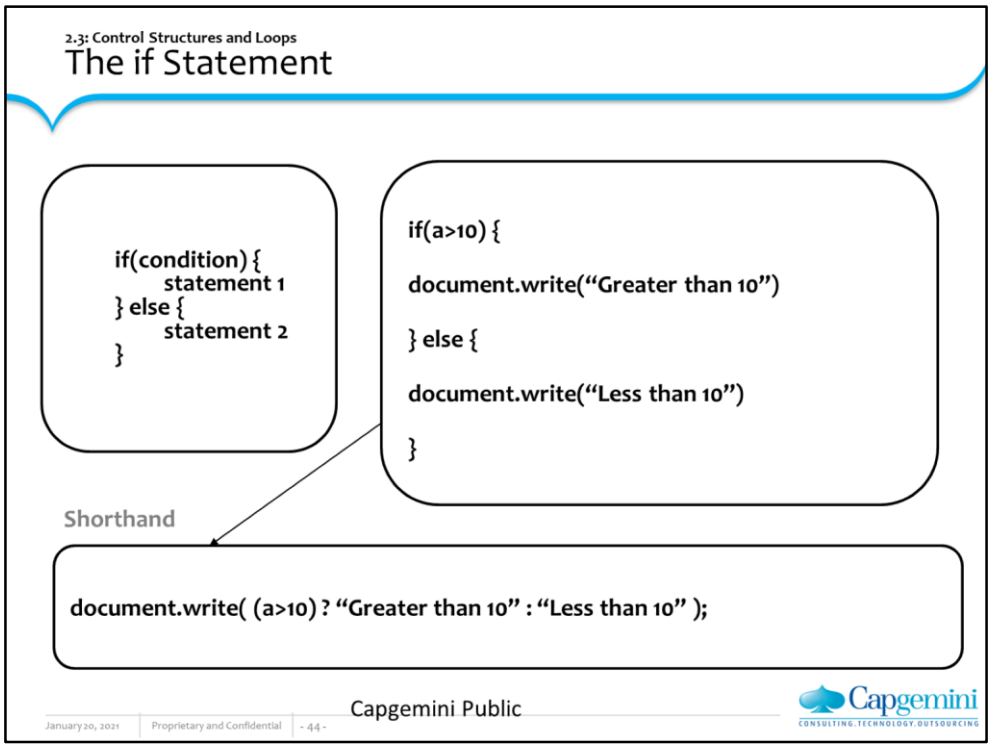
Instructor Notes:



Control Structures and Loops

Conditional statements are used to perform different actions based on different conditions. Loops execute a block of code for specified number of times or while a specified condition is true.

Instructor Notes:



The if Statement:

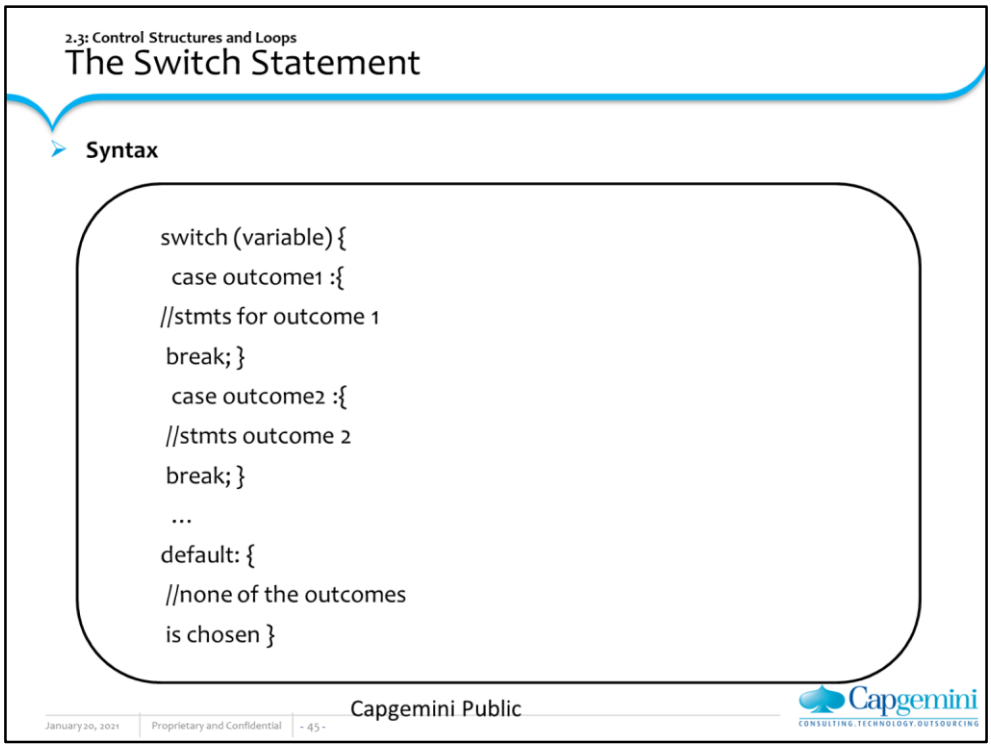
The condition is any JavaScript expression that evaluates to the Boolean type, either true or false.

The example as shown on the slide.

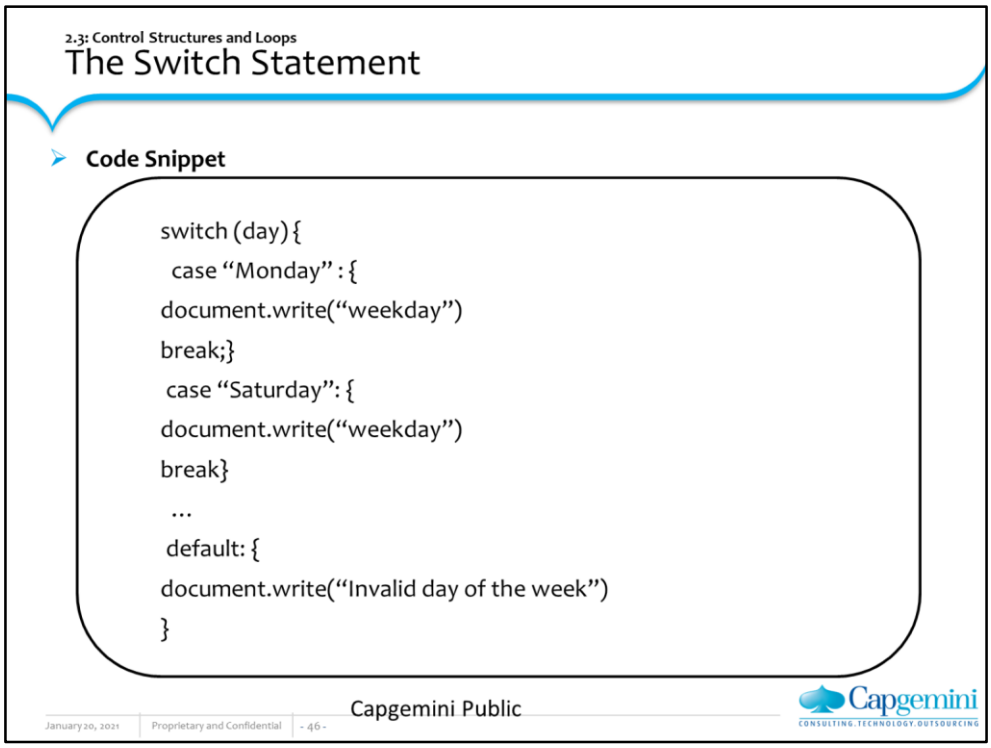
A shorthand method can also be used for these types of statements, where ? indicates the 'if' portion and : indicates the 'else' portion. This statement is equivalent to the previous example:

The equivalent shorthand method is also seen on the slide.

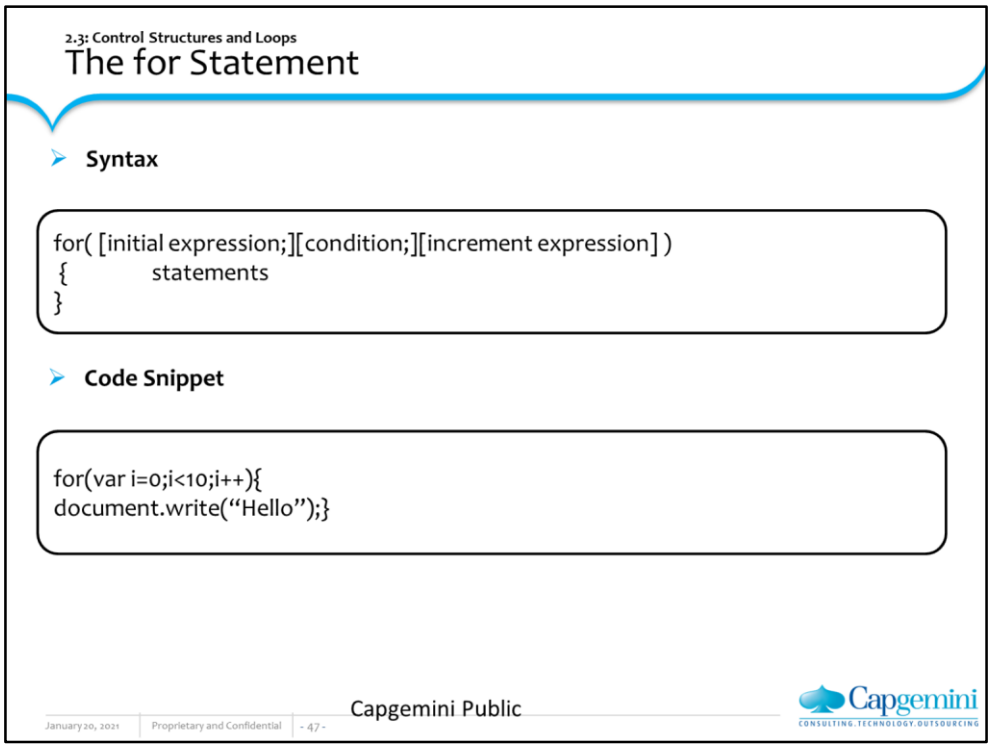
Instructor Notes:



Instructor Notes:



Instructor Notes:



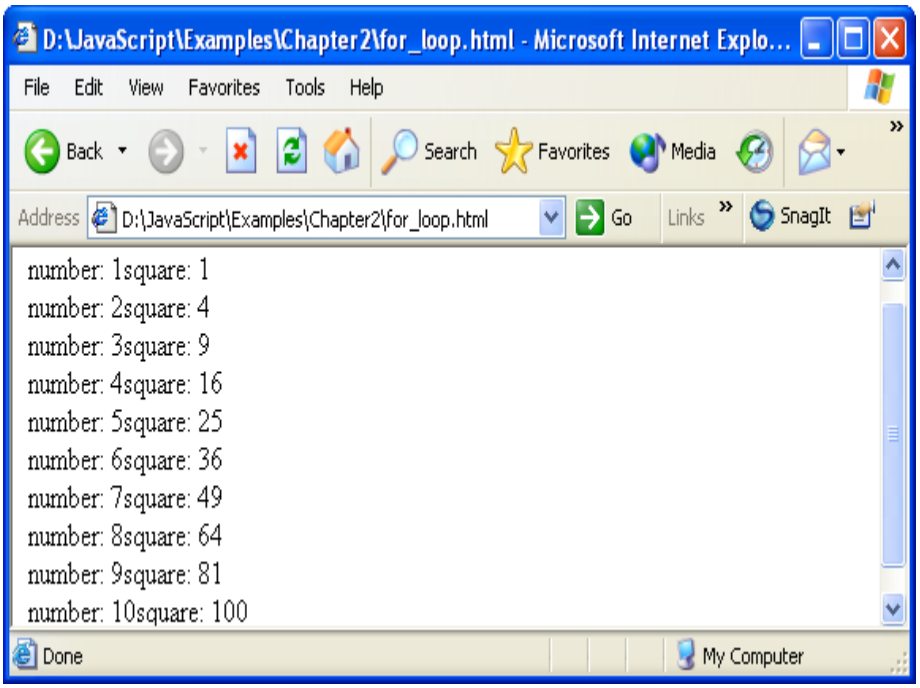
Looping Statements

The 'for' Statement:

```
<HTML>
<HEAD>
<SCRIPT LANGUAGE="JavaScript">
<!-- hide script
for (i=1; i<=10; i++)
{
  sq=i*i
  document.write("number: " + i + "square: " + sq + "<BR>")
}
// end script hiding -->
</SCRIPT>
</HEAD>
<BODY></BODY></HTML>
```

Example 2.2 For Construct (for_ex.html)
And it produces the output as:

Instructor Notes:



The 'while' Statement

The while statement continues to repeat the loop as long as the condition is true. The syntax for the while statement is as follows:

while (condition):

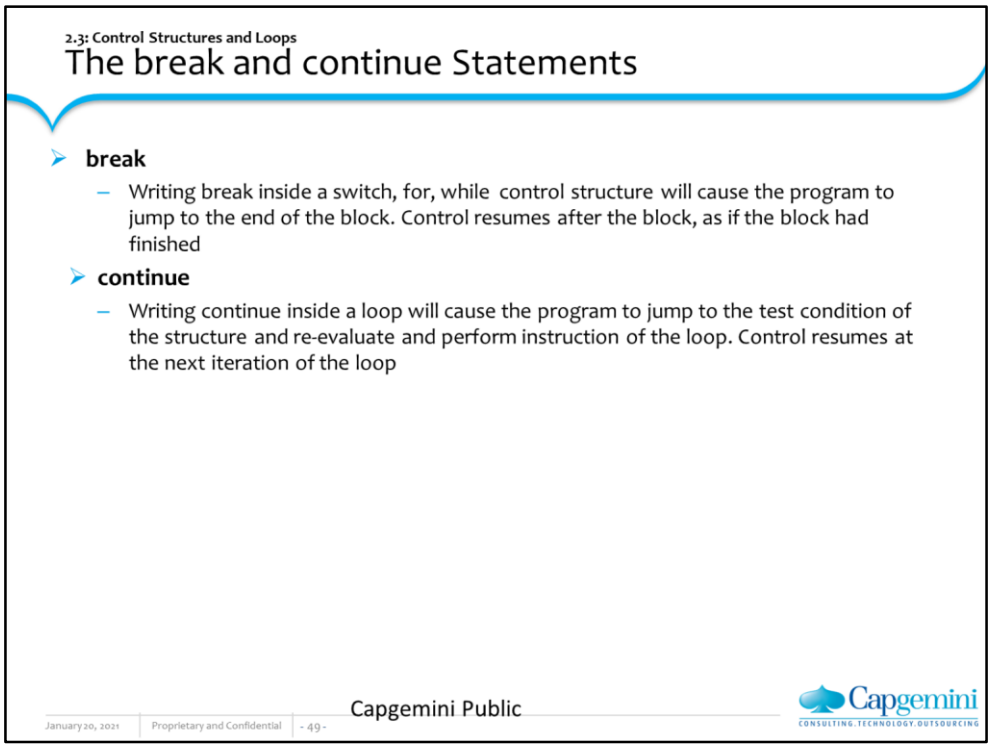
```
{
  statements
}
```

```
<HTML>
<HEAD>
<SCRIPT LANGUAGE="JavaScript">
<!-- hide script
i=1
while (i<=10)
{
  sq=I*i
  document.write("number:" + i + "square:" + sq + "<BR>")
  i++
}
// end script hiding -->
</SCRIPT>
</HEAD>
<BODY></BODY>
</HTML>
```

Example 2.3 While Construct

And it produces the output as in the previous screen shot.

Instructor Notes:



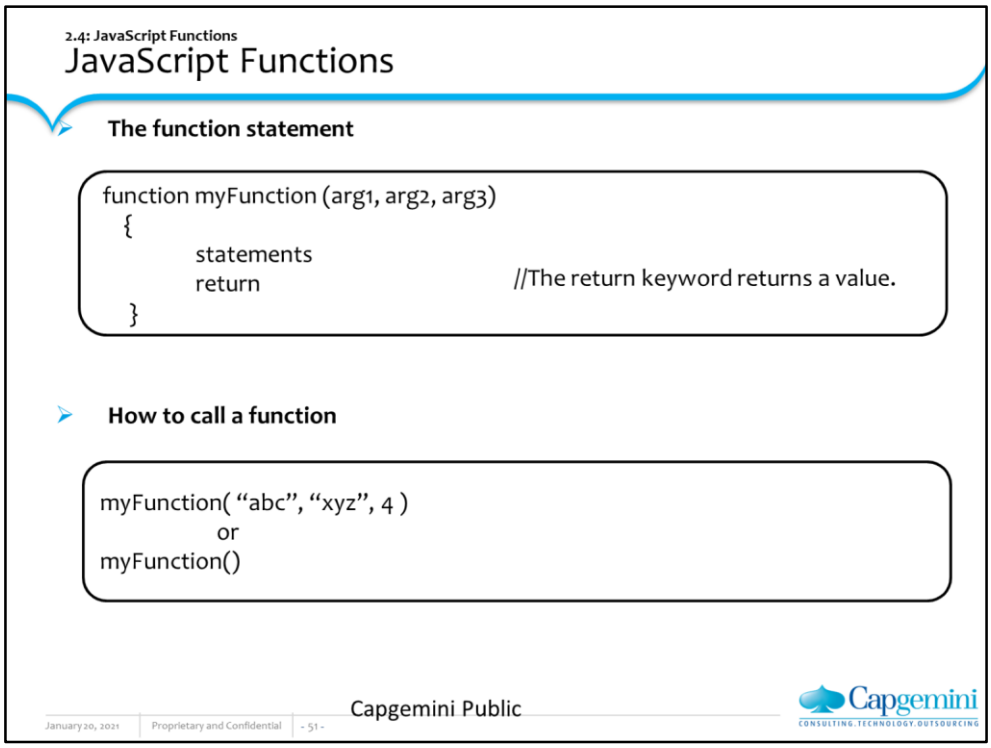
The break statement:
This statement is used to break out of the current 'for' or 'while' loop. Control resumes after the loop, as if it had finished.
The continue Statement:
This statement continues a 'for' or 'while' loop without executing the rest of the loop. Control resumes at the next iteration of the loop.

Instructor Notes:

Demo

- For_ex.html

Instructor Notes:



The function Statement:

A function contains some code that will be executed by an event or a call to that function. A function is a set of statements. You can reuse functions within the same script, or in other documents. You define functions at the beginning of a file (in the head section), and call them later in the document.

The syntax of a typical function is as follows:

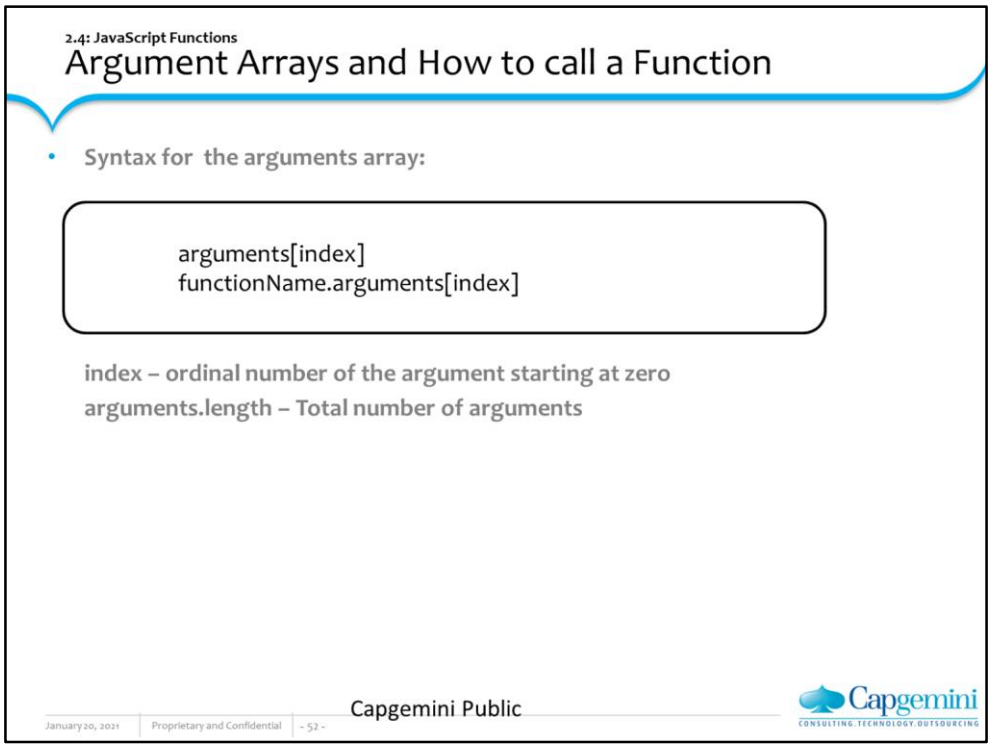
```
function myfunction(arg1, arg2, arg3)

{
  statements
}
```

How to Call a Function?

A function is not executed before it is called. You can call a function containing arguments:

```
myfunction("abc","xyz",4)
or without arguments:
myfunction()
```

Instructor Notes:



Using the arguments Array

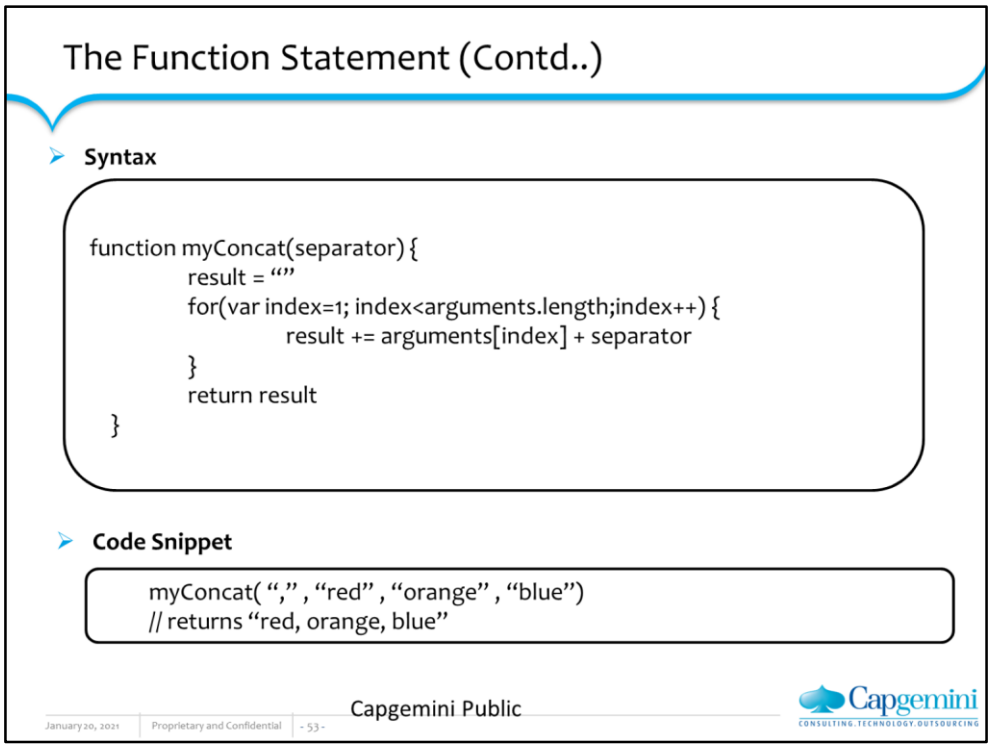
The arguments of a function are maintained in an array. Within a function, you can address the parameters passed to it as follows:

arguments[*i*]

functionName.arguments[i]

where i is the ordinal number of the argument, starting at zero. So, the first argument passed to a function would be arguments[0]. The total number of arguments is indicated by arguments.length. Using the arguments array, you can call a function with more arguments than it is formally declared to accept. This is often useful if you don't know in advance how many arguments will be passed to the function. You can use arguments.length to determine the number of arguments actually passed to the function, and then treat each argument using the arguments array.

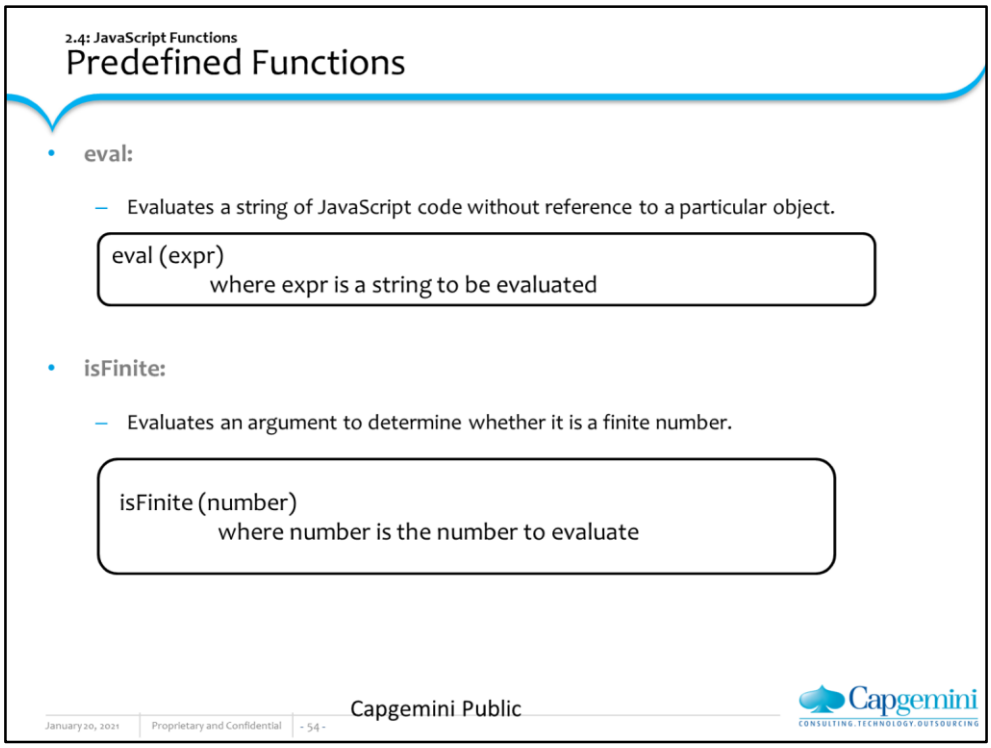
Instructor Notes:



For example, consider a function that concatenates several strings. The only formal argument for the function is a string that specifies the characters that separate the items to concatenate. The function is defined as shown on the slide.
You can pass any number of arguments to this function, and it creates a list using each argument as an item in the list.

```
// returns "red, orange, blue, "
myConcat(", ","red","orange","blue“)
// returns "elephant; giraffe; lion; cheetah;"
myConcat("; ","elephant","giraffe","lion", "cheetah“)
// returns "sage. basil. oregano. pepper. parsley. "
myConcat(". ","sage","basil","oregano", "pepper", "parsley")
```

Instructor Notes:



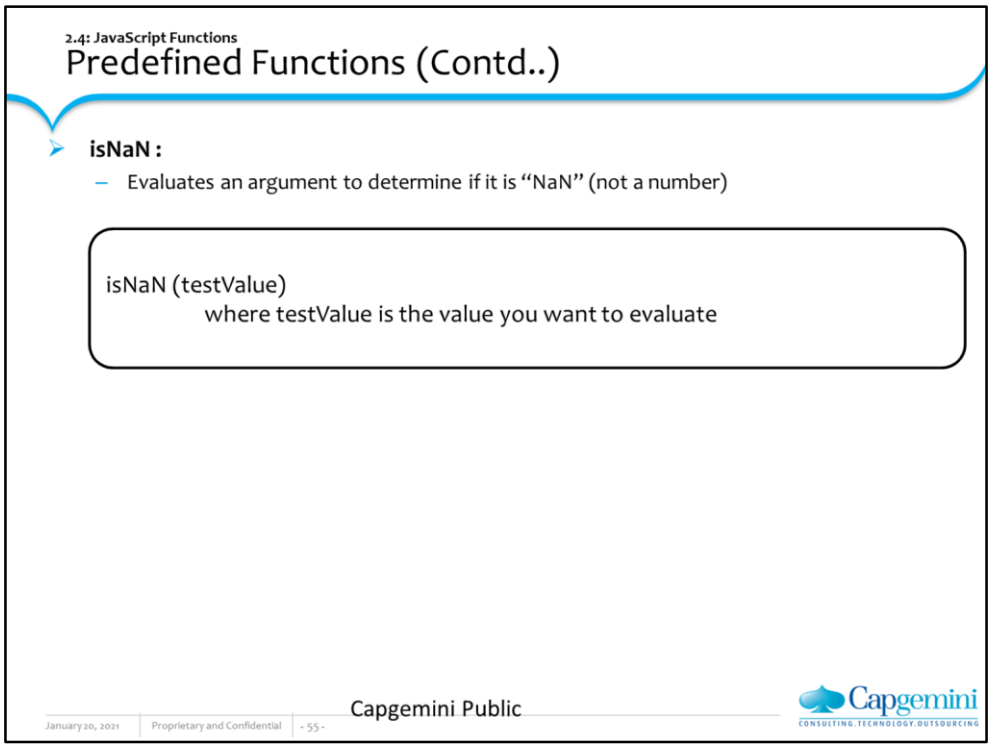
Predefined Functions:
JavaScript has several top-level predefined functions:
Eval, isFinite, isNaN, parseInt and parseFloat, Number and String

eval Function
The eval function evaluates a string of JavaScript code without reference to a particular object. The syntax of eval is:
eval(*expr*) where expr is a string to be evaluated.
If the string represents an expression, eval evaluates the expression. If the argument represents one or more JavaScript statements, eval performs the statements. Do not call eval to evaluate an arithmetic expression; JavaScript evaluates arithmetic expressions automatically.

isFinite Function
The isFinite function evaluates an argument to determine whether it is a finite number. The syntax of isFinite is:
isFinite(*number*) where number is the number to evaluate.
If the argument is NaN, positive infinity or negative infinity, this method returns false, otherwise it returns true. The following code checks client input to determine whether it is a finite number.

```
if(isFinite(ClientInput)== true)
{
  /* take specific steps */
}
```

Instructor Notes:



isNaN Functions:

The isNaN function evaluates an argument to determine if it is "NaN" (not a number). The syntax of isNaN is:

```
isNaN(testValue)
```

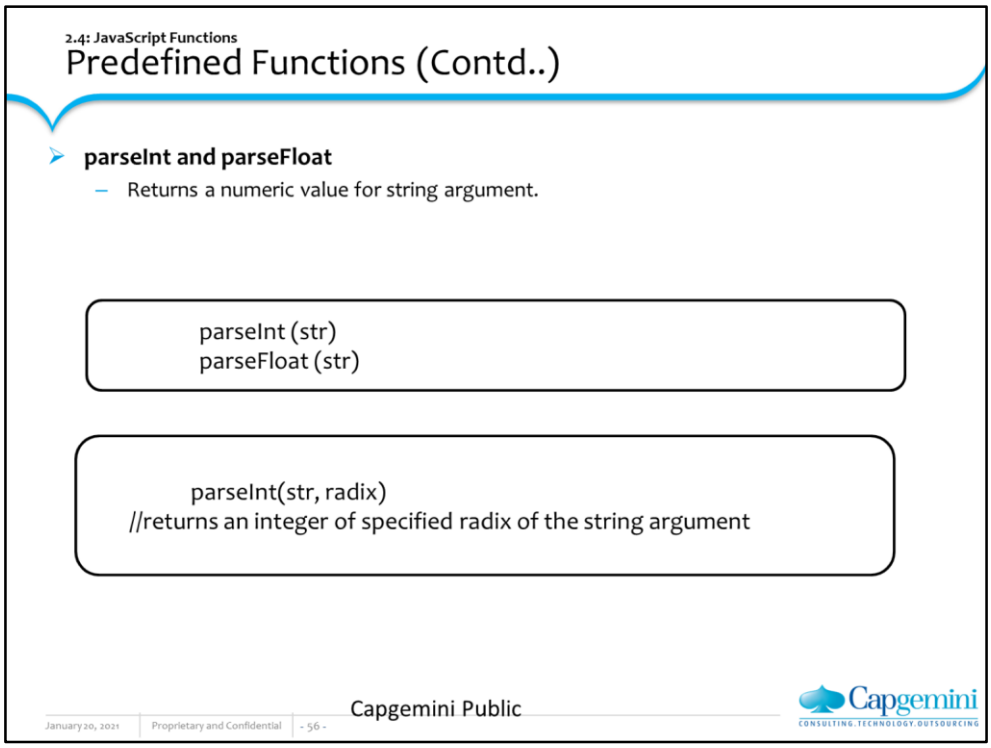
where testValue is the value you want to evaluate.

The parseFloat and parseInt functions return "NaN" when they evaluate a value that is not a number. isNaN returns true if passed "NaN," and false otherwise.

The following code evaluates floatValue to determine if it is a number and then calls a procedure accordingly:

```
floatValue=parseFloat(toFloat)
        if (isNaN(floatValue)){
            notFloat()
          } else {
            isFloat()
          }
```

Instructor Notes:



parseInt and parseFloat Functions:

The two "parse" functions, parseInt and parseFloat, return a numeric value when given a string as an argument.

The syntax of parseFloat is:

parseFloat(*str*)

where parseFloat parses its argument, the string str, and attempts to return a floating-point number. If it encounters a character other than a sign (+ or -), a numeral (0-9), a decimal point, or an exponent, then it returns the value up to that point and ignores that character and all succeeding characters. If the first character cannot be converted to a number, it returns "NaN" (not a number).

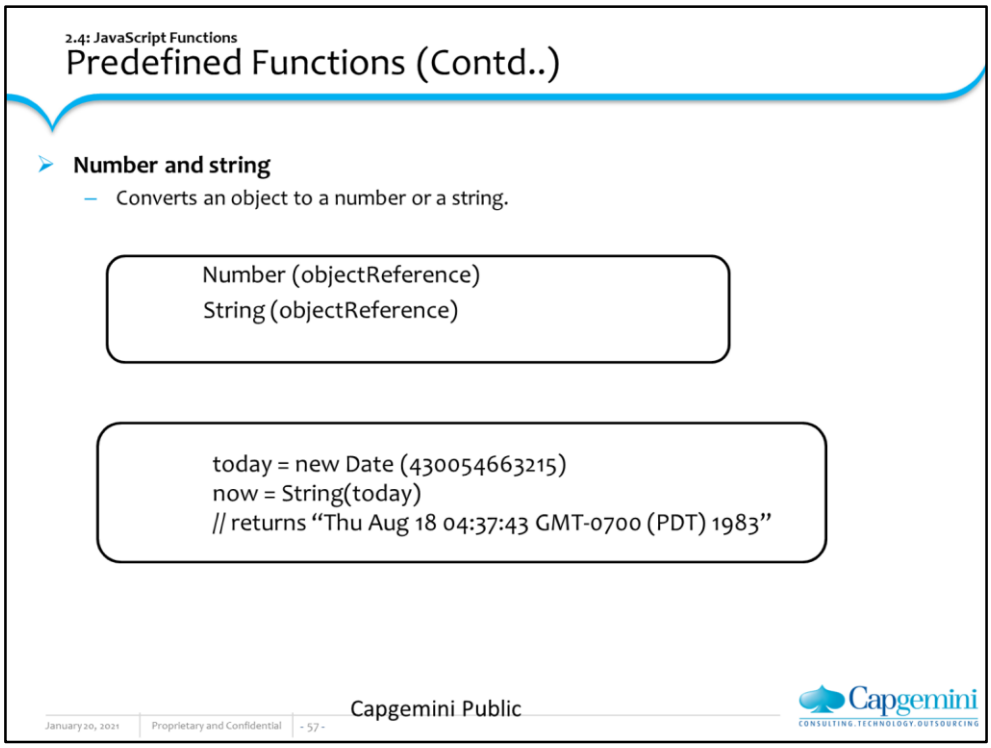
The syntax of parseInt is:

parseInt(str [, radix])

where parseInt parses its first argument, the string str, and attempts to return an integer of the specified radix (base), indicated by the second, optional argument, radix. For example, a radix of ten indicates to convert to a decimal number, eight octal, sixteen hexadecimal, and so on. For radixes above ten, the letters of the alphabet indicate numerals greater than nine. For example, for hexadecimal numbers (base 16), A through F are used.

If parseInt encounters a character that is not a numeral in the specified radix, it ignores it and all succeeding characters and returns the integer value parsed up to that point. If the first character cannot be converted to a number in the specified radix, it returns "NaN." The parseInt function truncates the string to integer values.

Instructor Notes:



Number and String Functions:

The Number and String functions let you convert an object to a number or a string. The syntax of these functions is:

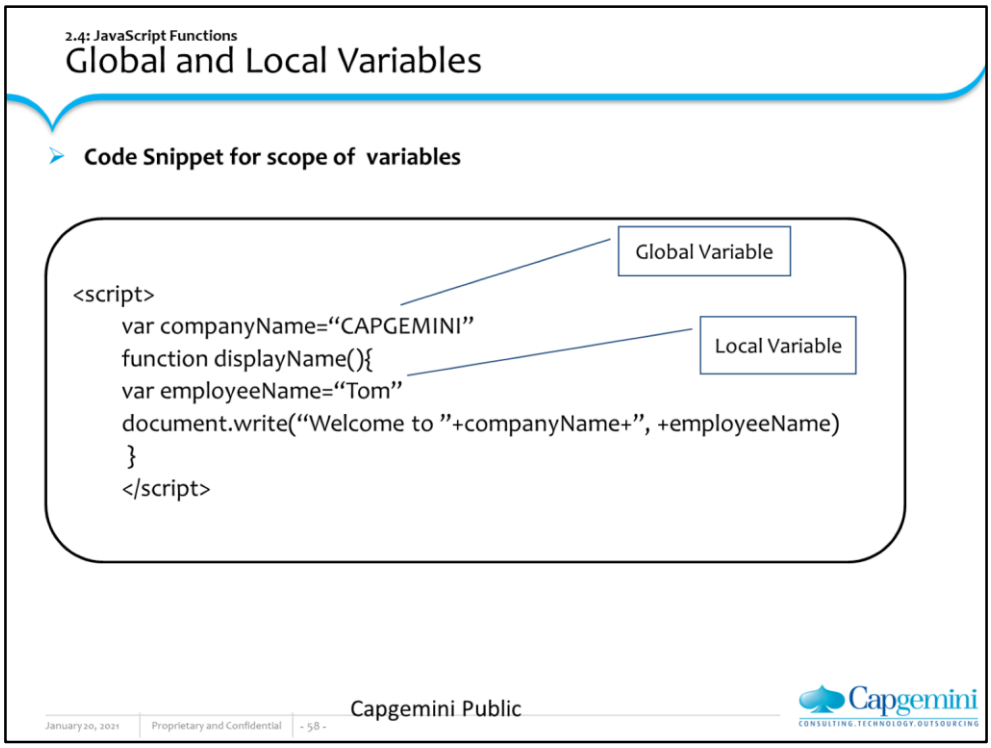
Number(*objRef*)

String(*objRef*)

where objRef is an object reference. The following example converts the Date object to a readable string.

```
D = new Date (430054663215)
// The following returns
// "Thu Aug 18 04:37:43 GMT-0700 (Pacific Daylight Time) 1983"
x = String(D)
```

Instructor Notes:



Scope of Variables:

JavaScript supports two variable scopes:

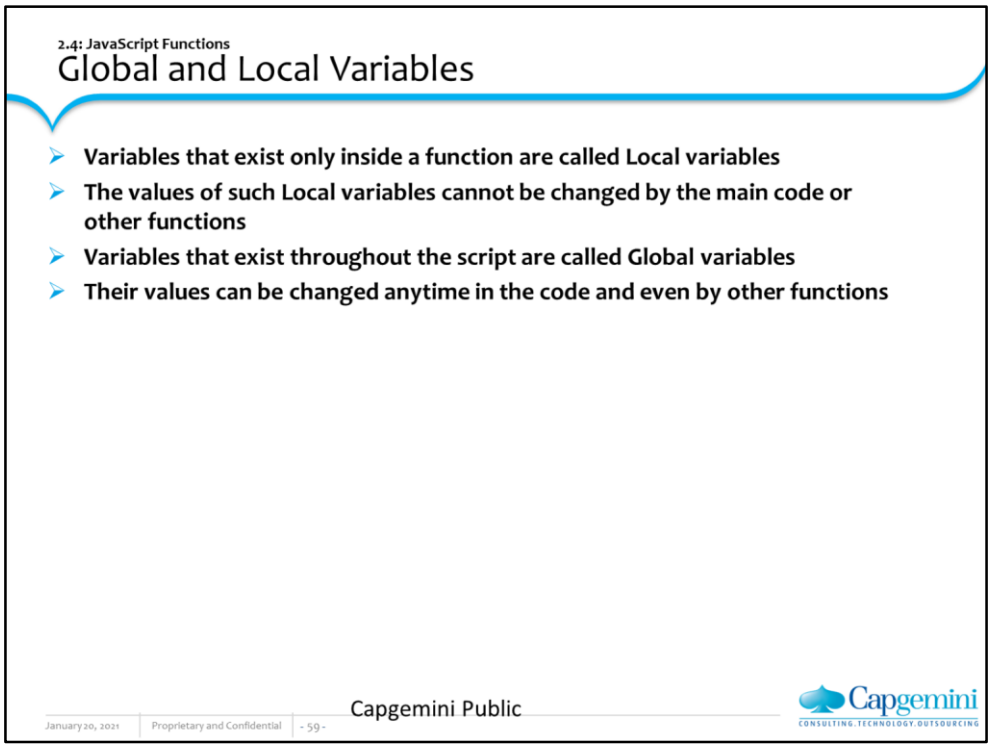
Global variables
Local variables

The local variable applies only within a function and limits the scope of the variable to that function. To declare a local variable, the variable name must be preceded by var, as shown following:

```
var MaxValue=0;
```

Any variable declaration that is not within a function, is treated as a global variable. The syntax to declare a global variable is the same as that for local variable.

Instructor Notes:



Using Global and Local Variables:

You can choose between local and Global variables by using the following guidelines:

If the value of a variable is meant to be used by any part of the program, both inside and outside functions, the variable should be declared outside any function. This has the effect of making it global and modifiable by any part of the program. The best place to declare global variables is in the <head> block of the HTML document.

If the variable is needed only within a particular function, the variable should be declared inside that function.

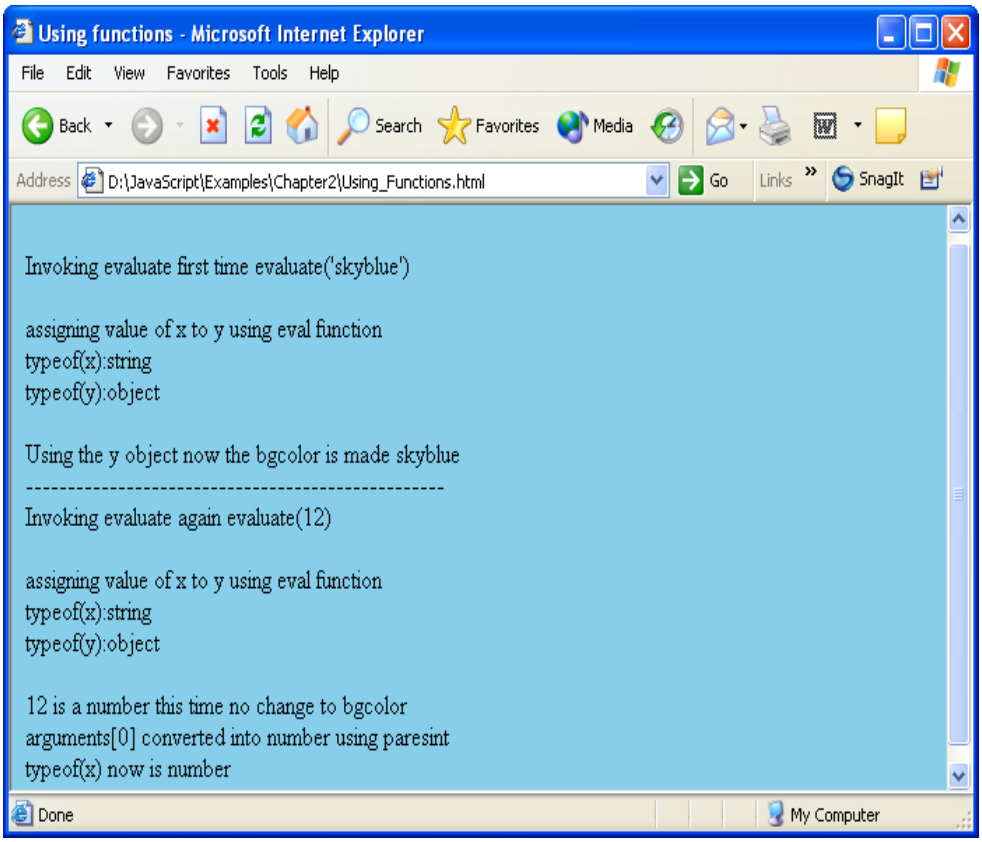
If you want the value of a variable to be modified only by the main script of a single function, but you need to use it in another function, pass the variable as an argument to that function. This has the effect of making a copy of the variable and assigning its value to the argument. As the function works and modifies its own copy of the variable, it will not effect the original. Argument variables are automatically declared as local to that function. Even if the argument has the same name as the variable being passed, making changes to it does not effect the variable that was passed. The only exception to this is objects. When an object is passed as an argument, it is passed by reference as opposed to being passed by value. Instead of making a copy of the object, the function uses the original object. Changes made to an object's properties within the function have an effect on the original object.

Instructor Notes:

```
<HTML>
<HEAD>
<META NAME="GENERATOR" Content="Microsoft Visual Studio 6.0">
<TITLE>Using functions</TITLE>
<script language="Javascript">
<!--
  function evaluate()
  {
     var x;
      var y;
    x = "document“
    document.write("<br>assigning value of x to y using eval function");
    y = eval(x);
    document.write("<br>typeof(x):");
    document.write(typeof(x)+"<br>");
    document.write("typeof(y):");
    document.write(typeof(y)+"<br>");
     if(isNaN(arguments[0]))
    {
        document.write("<br>Using the y object now the bgcolor is made
                                    "+arguments[0] );
         y.bgColor=arguments[0];
    }
    else
    {
      document.write("<br>"+arguments[0]+" is a number this time no
                                  change to bgcolor");
      x=parseInt(arguments[0]);
      document.write("<br> arguments[0] converted into number using
                                 parseInt ");
      document.write("<br>typeof(x) now is ");
      document.write(typeof(x)+"<br>");
    }
  }
-->
</script></HEAD><BODY><script>
document.write("<br>Invoking evaluate first time evaluate('skyblue')<br>");
evaluate("skyblue");
document.write("<BR>");
for(var i=0;i<50;i++)
document.write("-");
document.write("<br>Invoking evaluate again evaluate(12)<br>");
evaluate(12);
</script>
</BODY>
</HTML>
```

Instructor Notes:

Example 2.5 Demo of creating functions and predefined function (num_string_fun.html)
And it produces the output as:



Instructor Notes:

Demo

- If_else.html
- Switch_ex.html
- For_ex.html
- Break_con_ex.html
- Fun_ex.html
- Num_string_fun.html

Instructor Notes:

Lab

- Lab 2 :
- The JavaScript language

Instructor Notes:

Summary

- **Data Types & Variables**
  - Numbers, Strings, Boolean, and Null
- **Operators & Expressions**
- **Functions**
- **Predefined Functions**
  - eval, isFinite, isNAN, parseInt & parseFloat, Number & String
- **Global and Local variables**

Instructor Notes:

Answers

1. Option 0
2. True

Review Question

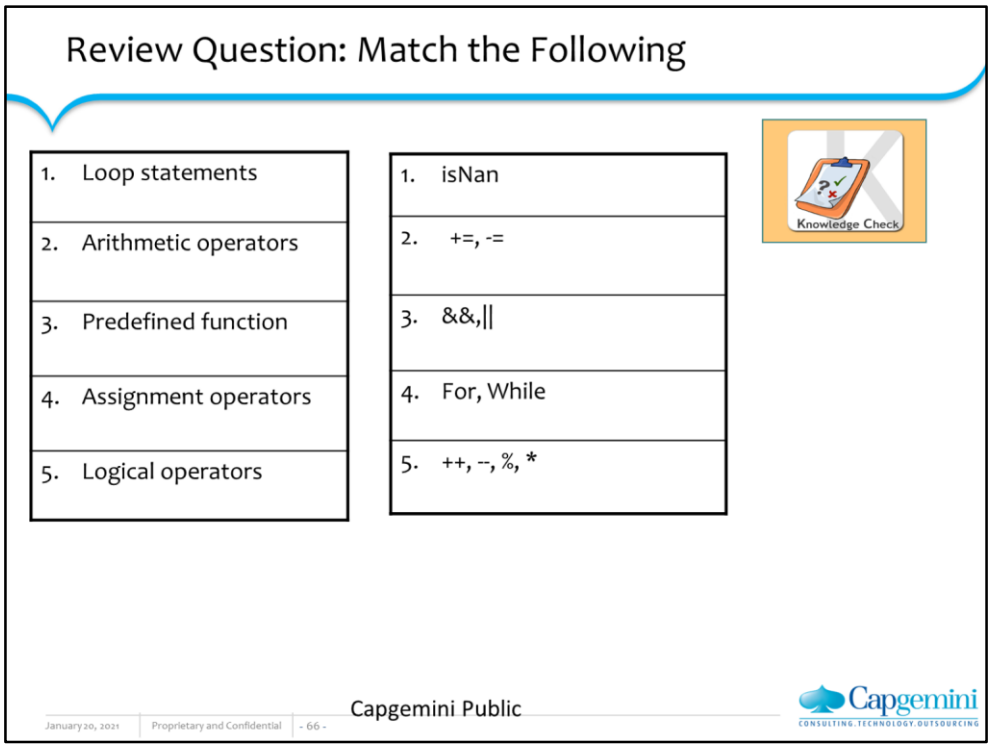
- **Question 1: Which of the following two variable scopes is supported by JavaScript:**
  - Global, Local
  - Functional, Non functional
  - Static, Dynamic

- **Question 2: The eval function evaluates a string of JavaScript code without reference to a particular object.**
  - True/False

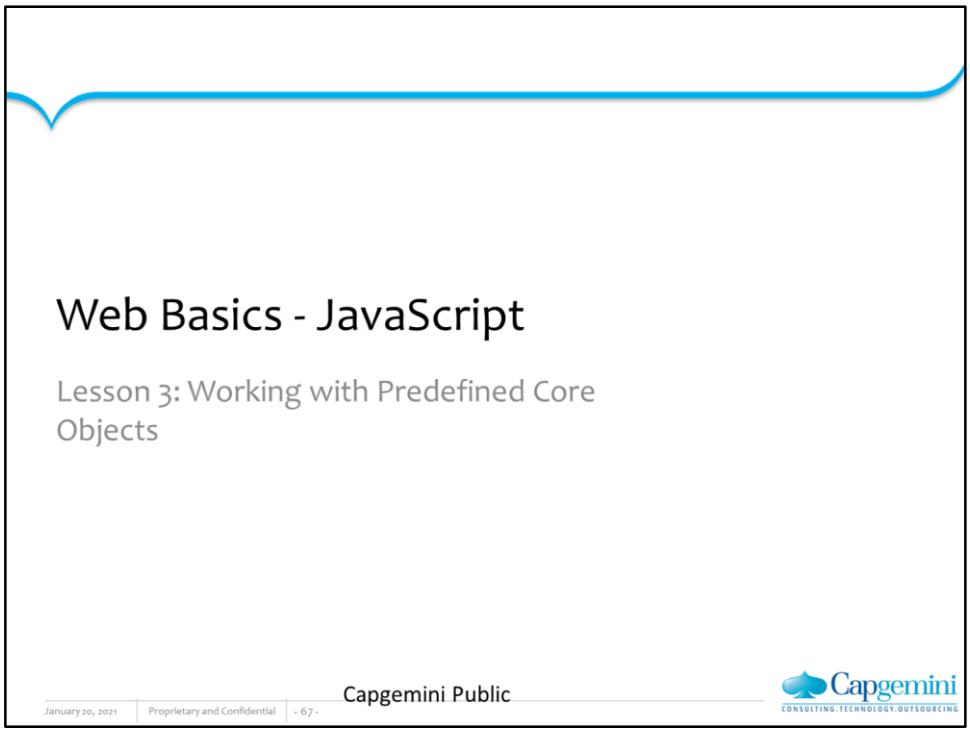
Instructor Notes:

Answers

1. 4
2. 5
3. 1
4. 2
5. 3



Instructor Notes:

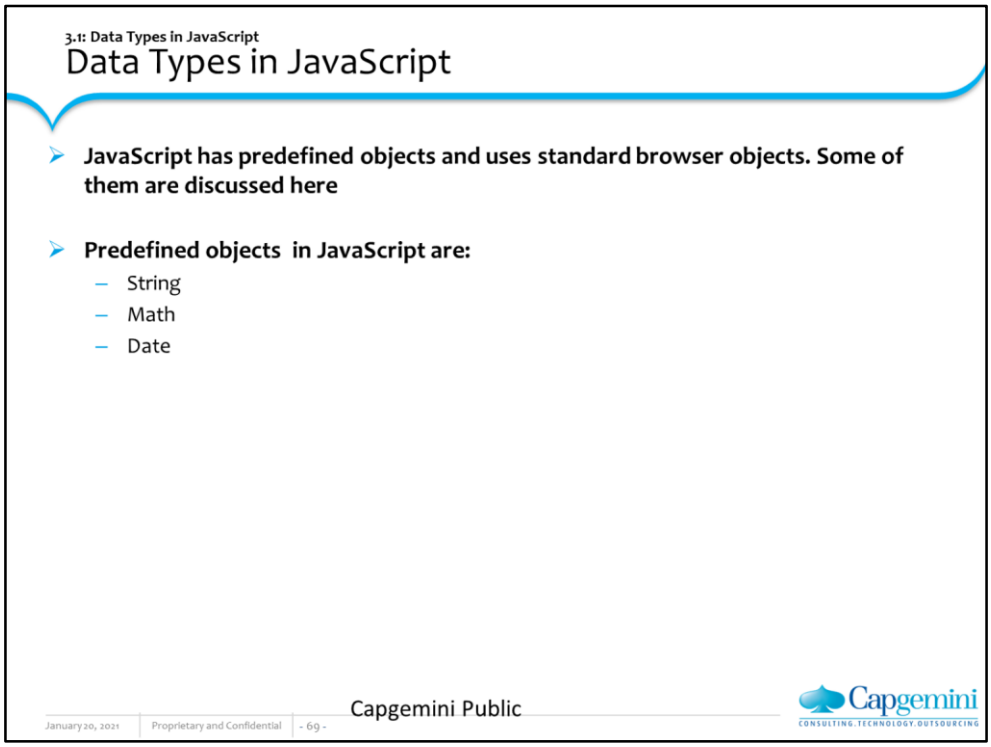


Instructor Notes:

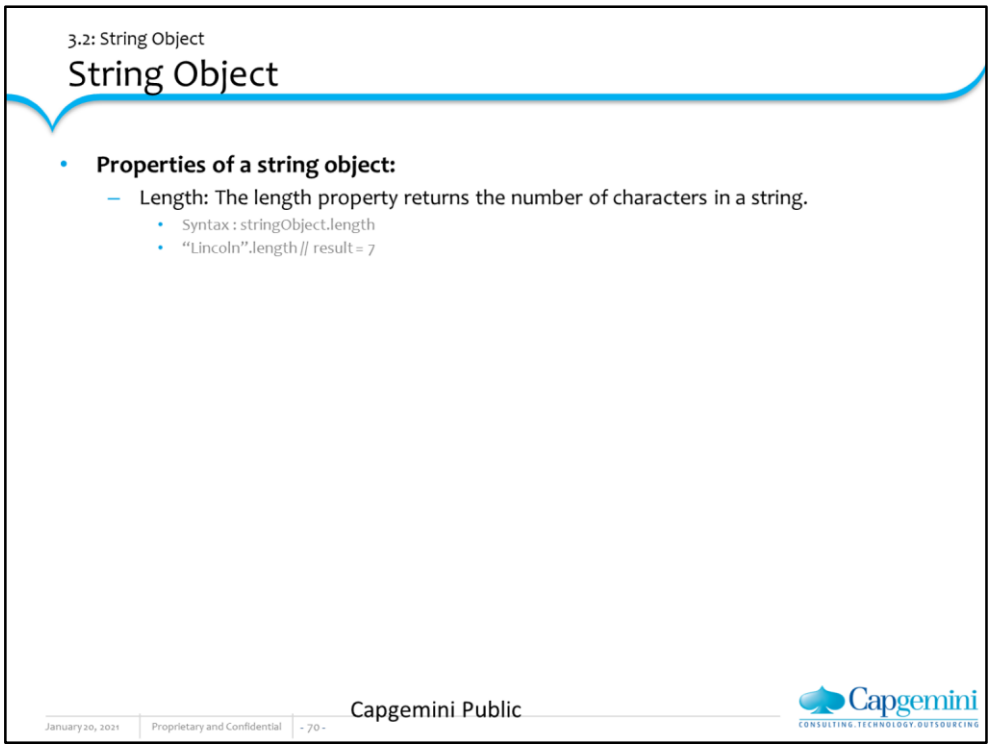
Lesson Objectives

- Data Types in JavaScript
- String Object
- Math Object
- Date Object

Instructor Notes:



Instructor Notes:



String Objects: Creating a String

A string consists of one or more standard text characters between matching quote marks. JavaScript is forgiving in one regard: You can use single or double quotes, as long as you match two single quotes or two double quotes around a string.

JavaScript draws a fine line between a string value and a string object. Both let you use the same methods on their contents, so by and large, you do not have to create a string object (with the new String() constructor) every time you want to assign a string value to a variable. A simple assignment operation (var myString = "fred") is all you need to create a string value that behaves on the surface very much like a full-fledged string object.

```
var myString = new String("characters")
var myString = "fred"
```

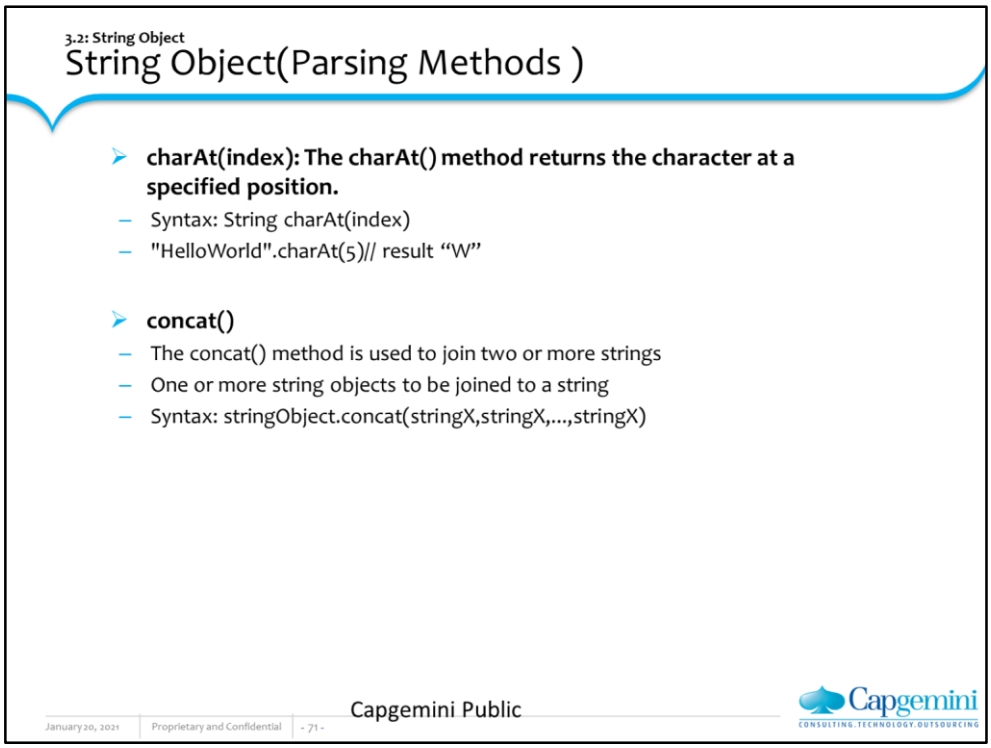
Properties of String Objects:

Length:

The most frequently used property of a string is length. To derive the length of a string, extract its property as you would extract the length property of any object. The length value represents an integer count of the number of characters within the string. Spaces and punctuation symbols count as characters. Any backslash special characters embedded in a string count as one character, including such characters as newline and tab.

```
"Four score".length // result = 10
"One\ntwo".length // result = 7
"".length // result = 0
```

Instructor Notes:



String Objects (Parsing Methods):

String.chartAt(index)

Use the string.charAt() method to extract a single character from a string when you know the position of that character. For this method, you specify an index value in the string as a parameter to the method. The index value of the first character of the string is 0. If your script needs to get a range of characters, use the string.substring() method. It is a common mistake to use string.substring() to extract a character from inside a string, when the string.charAt() method is more efficient.

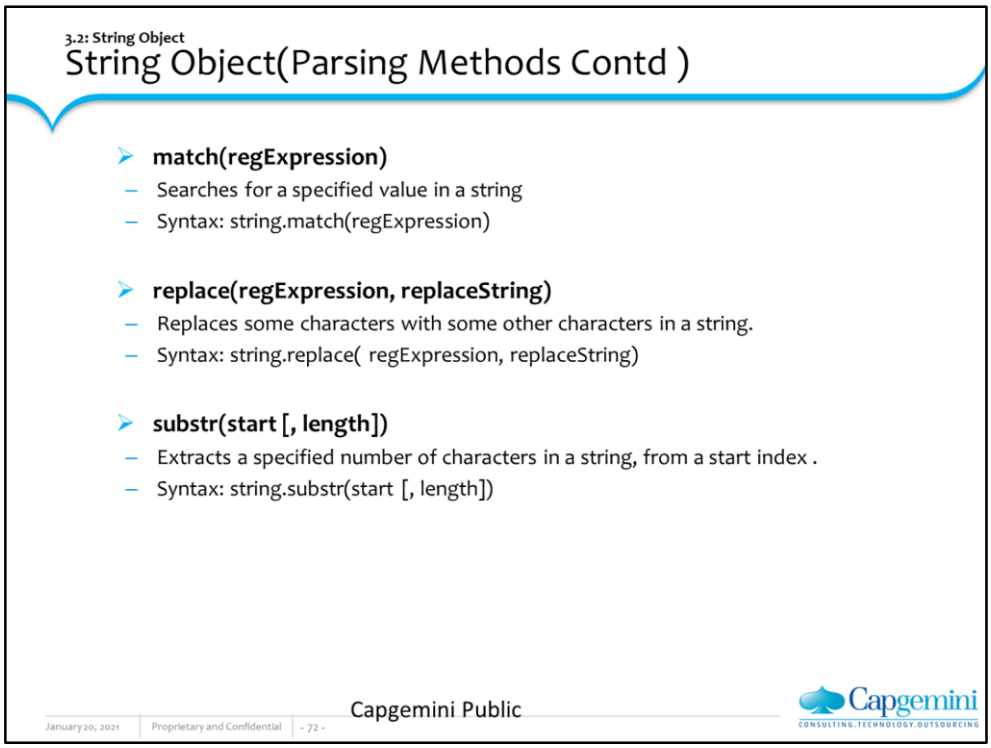
string.concat(string2)

Returns: Combined string.

"abc".concat("def") // result: "abcdef"

JavaScript's add-by-value operator (+=) provides a convenient way to concatenate strings. N4 and IE4, however, introduces a string object method that performs the same task. The base string to which more text is appended is the object or value to the left of the period. The string to be appended is the parameter of the method, as the example demonstrates. Like the add-by-value operator, the concat() method doesn't know about word endings. You are responsible for including the necessary space between words if the two strings require a space between them in the result.

Instructor Notes:



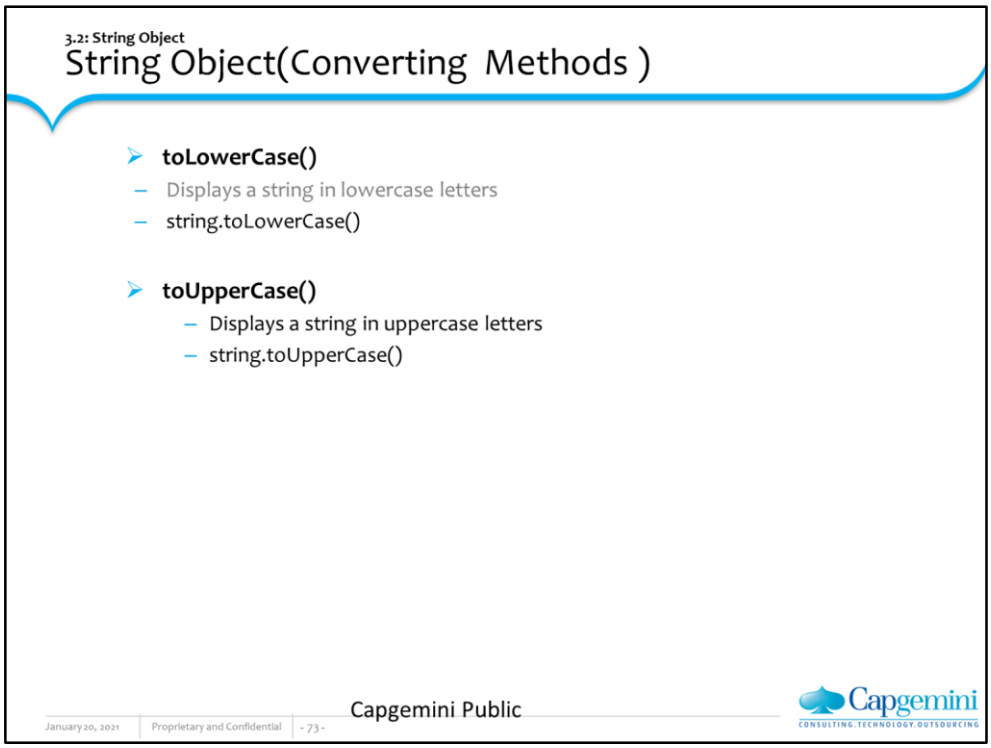
String Objects (Parsing Methods contd..):

string.match(regExpression)**:**Returns Array of matching strings.
The string.match() method relies on the RegExp (regular expression) object. The parameter must be a regular expression object. This method returns an array value when at least one match turns up; otherwise the returned value is null.
string.replace(regExpression, replaceString): Returns Changed string.
Regular expressions are commonly used to perform search-and-replace operations. JavaScript's string.replace() method provides a simple framework in which to perform this kind of operation on any string.

```
var str = "To be, or not to be: that is the question."
var regexp = /be/
str.relace(regexp, "exist")
```

string.substr(start [, length]) :Returns Characters of the string from the first character of the given string to the specified length.

Instructor Notes:



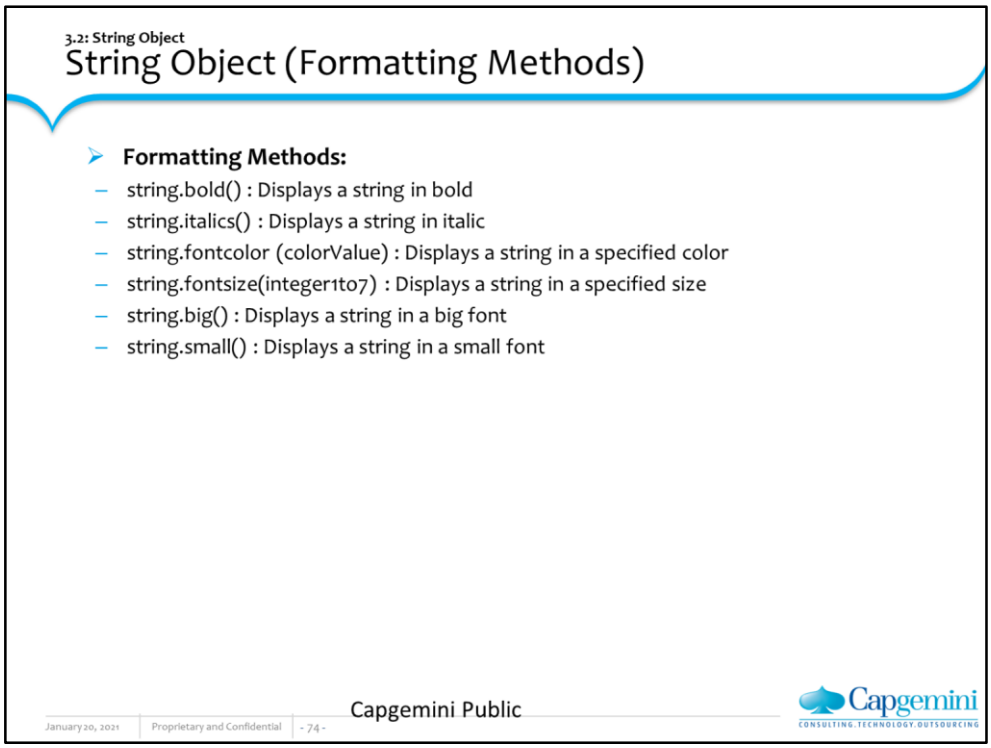
String Objects (Parsing Methods contd..):

string.toLowerCase() and string.toUpperCase()

Returns: The string in all lower- or uppercase, depending on which method you invoke.

A great deal of what takes place on the Internet (and in JavaScript) is case-sensitive. URLs on some servers, for instance, are case-sensitive for directory names and filenames. These two methods, the simplest of the string methods, convert any string to either all lowercase or all uppercase. Any mixed-case strings get converted to a uniform case. If you want to compare user input from a field against some coded string without worrying about matching case, you should convert both strings to the same case for the comparison.

Instructor Notes:



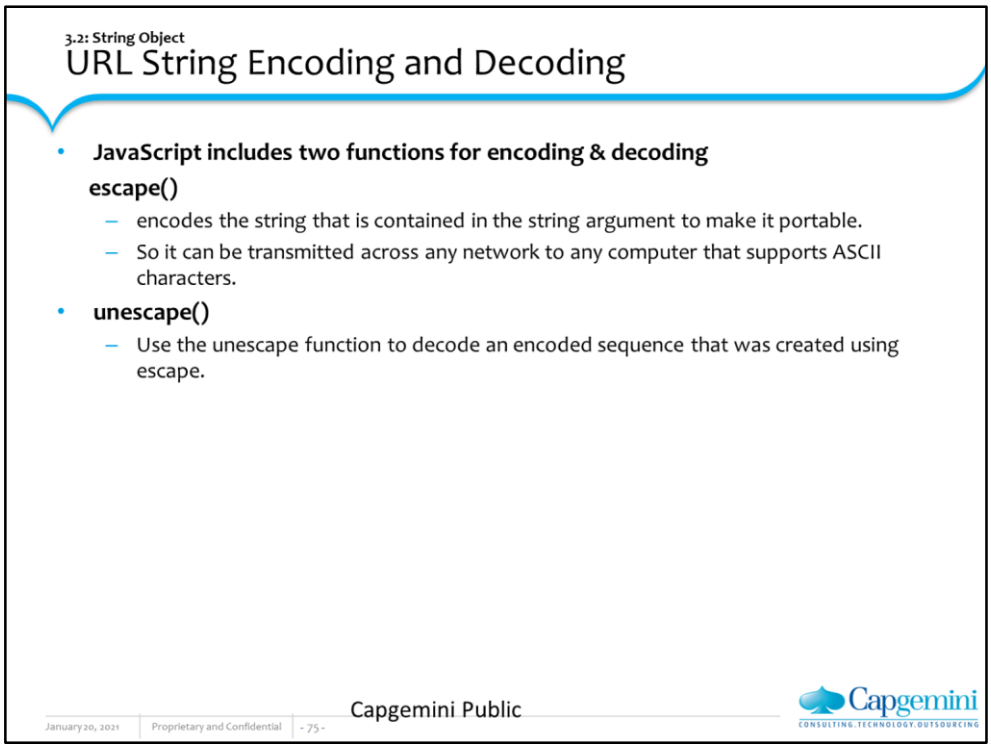
String Objects (Formatting Methods):

Now we come to the other group of string object methods, which ease the process of creating the numerous string display characteristics when you use JavaScript to assemble HTML code.

You can still use the standard HTML tags instead of by calling the string methods in your web pages. The choice is up to you. One advantage to the string methods is that they never forget the ending tag of a tag pair.

string.fontsize() and string.fontcolor() also affect the font characteristics of strings displayed in the HTML page. The parameters for these items are pretty straightforward —an integer between 1 and 7 corresponding to the seven browser font sizes and a color value (as either a hexadecimal triplet or color constant name) for the designated text.

Instructor Notes:



URL String Encoding and Decoding

escape("Howdy Pardner")// result = "Howdy%20Pardner"
unescape("Howdy%20Pardner")// result = "Howdy Pardner"

When browsers and servers communicate, some nonalphanumeric characters that we take for granted (such as a space) cannot make the journey in their native form. Only a narrower set of letters, numbers, and punctuation is allowed. To accommodate the rest, the characters must be encoded with a special symbol (%)and their hexadecimal ASCII values. For example, the space character is hex 20 (ASCII decimal 32). When encoded, it looks like %20. You may have seen this symbol in browser history lists or URLs.

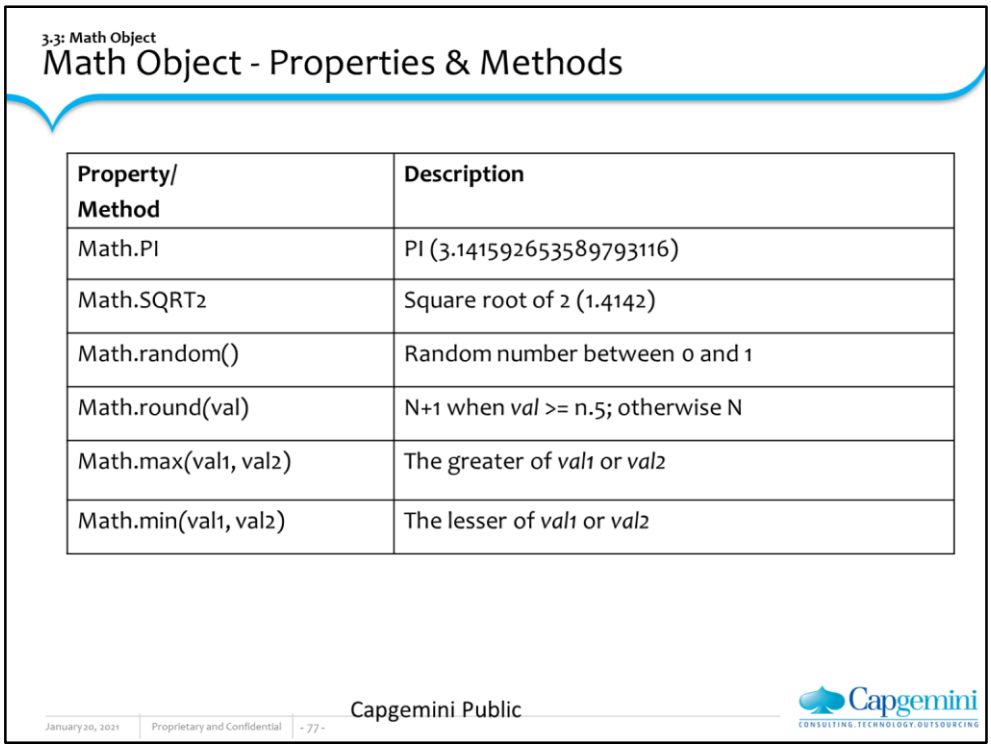
JavaScript includes two functions, escape() and unescape(), that offer instant conversion of whole strings. To convert a plain string to one with these escape codes, use the escape function, as in escape("Howdy Pardner") // result = "Howdy%20Pardner". The unescape() function converts the escape codes into human-readable form.

Instructor Notes:

Demo

- string_len.html
- string_method.html
- string_style.html

Instructor Notes:



Math Object – Properties & Methods:

The Math object allows you to perform mathematical tasks. All properties and methods can be called without creating the Math object.

You can use them in your regular arithmetic expressions since they return constant values. For example, to obtain the circumference of a circle whose diameter is in variable d, you use the statement shown below.

```
circumference = d * Math.PI
```

The Math.random() method returns a floating-point value between 0 and 1. If you are designing a script to act like a card game, you need random integers between 1 and 52; for dice, the range is 1 to 6 per die. To generate a random integer between zero and any top value, use the first formula shown where n is the top number.

To generate random numbers between a different range use the second formula where m is the lowest possible integer value of the range and n equals the top number of the range minus m. In other words n+m should add up to the highest number of the range you want. For the dice game, use the third formula for each throw of the die.

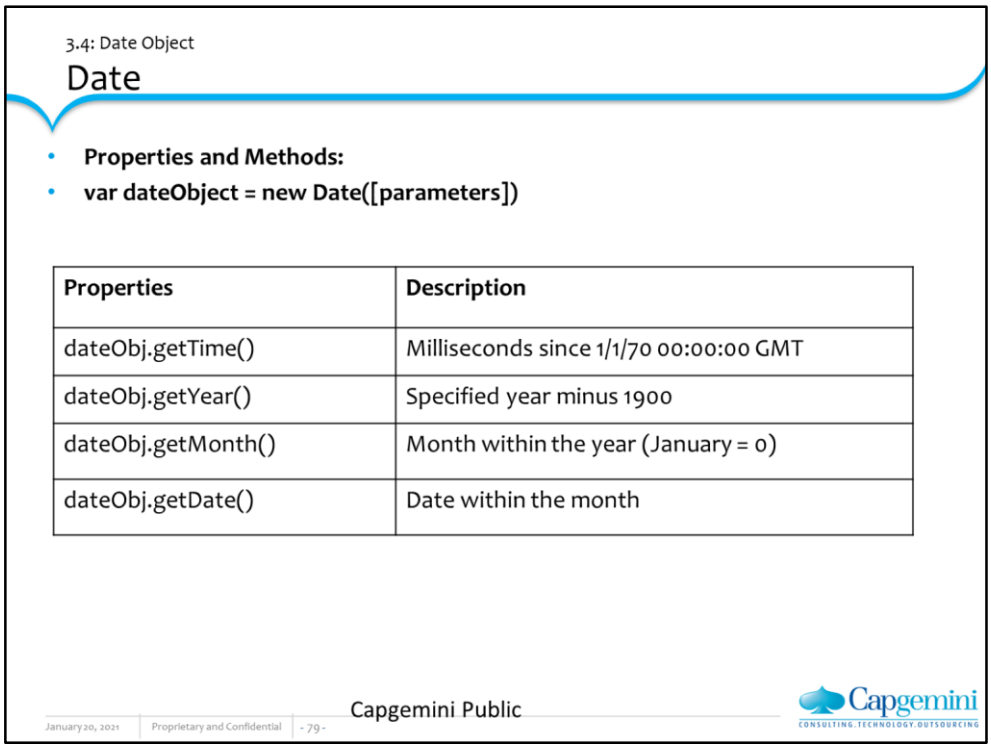
```
Math.round(Math.random() * n)
Math.round(Math.random() * n) + m
newDieValue = Math.round(Math.random() * 5) + 1
```

Instructor Notes:

Demo

- **Rand_fun.html**

Instructor Notes:

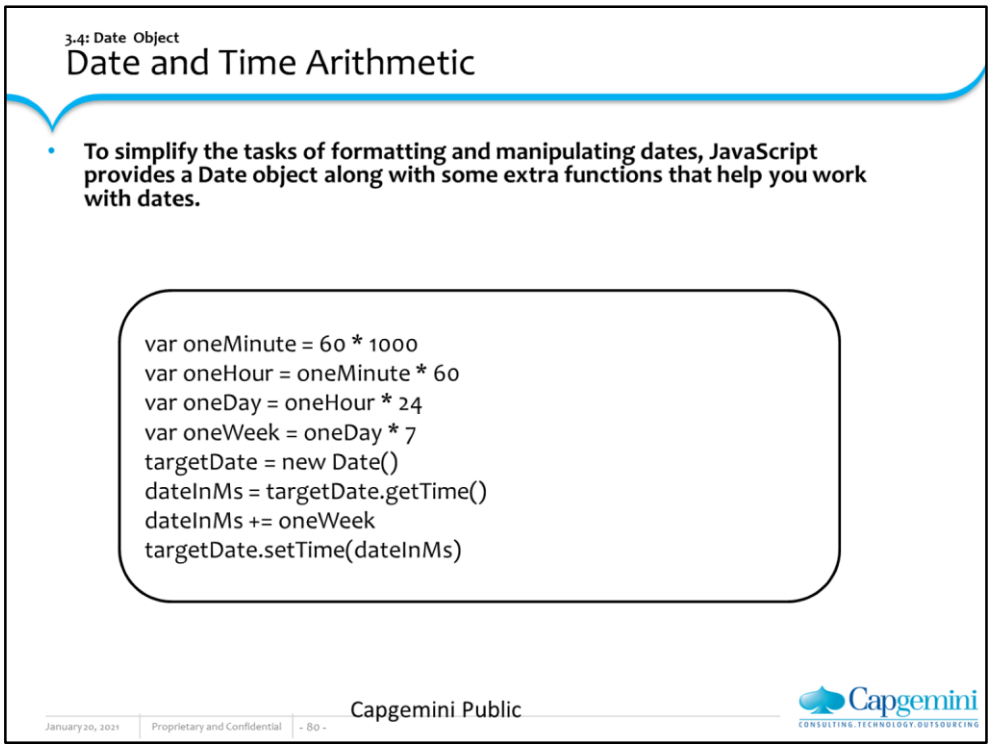


Date Object: Creating a Date object

```
var dateObject = new Date([parameters])
new Date("Month dd, yyyy hh:mm:ss")
new Date("Month dd, yyyy")
new Date(yy,mm,dd,hh,mm,ss)
new Date(yy,mm,dd)
new Date(milliseconds)
```

The Date object evaluates to an object rather than to some string or numeric value. If you leave the parameters empty, JavaScript takes that to mean you want today's date and the current time to be assigned to that new Date object. To create a Date object for a specific date or time, you have five ways to send values as a parameter to the new Date() constructor function.
The Date object has only a prototype property, which enables you to apply new properties and methods to every Date object created in the current page.

Instructor Notes:



Date and time arithmetic:

You may need to perform some math with dates for any number of reasons. Perhaps you need to calculate a date at some fixed number of days or weeks in the future or figure out the number of days between two dates. When calculations of these types are required, remember the *lingua franca* of JavaScript date values: the milliseconds.

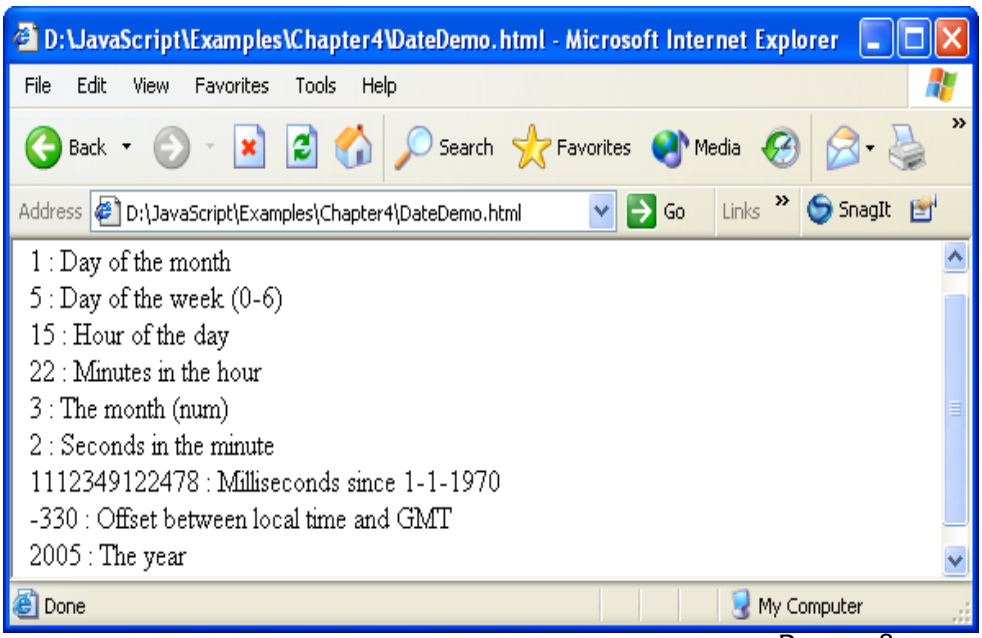
What you may need to do in your date-intensive scripts is establish some variable values representing the number of milliseconds for minutes, hours, days, or weeks, and then use those variables in your calculations. On the slide you can see an example that establishes some practical variable values, building on each other. With these values established in a script, you can use one to calculate the date one week from today. Following is the complete example.

Instructor Notes:

Date and time arithmetic (contd..):

```
<!DOCTYPE html>
<HTML>
<HEAD>
<TITLE></TITLE>
</HEAD>
<BODY>
<script>
dateinfo = new Date();
document.write(dateinfo.getDate() + " : Day of the month" + "<br>")

document.write(dateinfo.getDay()              +" : Day of the week  (0-6)"
+ "<br>" )
document.write(dateinfo.getHours()+" : Hour of the day " + "<br>")

document.write(dateinfo.getMinutes()+" : Minutes in the hour" + "<br>")
document.write(dateinfo.getMonth()+": The month              (num)" +
"<br>" )
document.write(dateinfo.getSeconds()+ " : Seconds in the minute " +
"<br>")
document.write(dateinfo.getTime()+" : Milliseconds since 1-1-1970" +
"<br>")
document.write(dateinfo.getTimezoneOffset() +" : Offset between local
time and GMT" + "<br>")
document.write(dateinfo.getYear() +" : The year" + "<br>")
</script>
</BODY>
</HTML>
```

And it produces the output as:



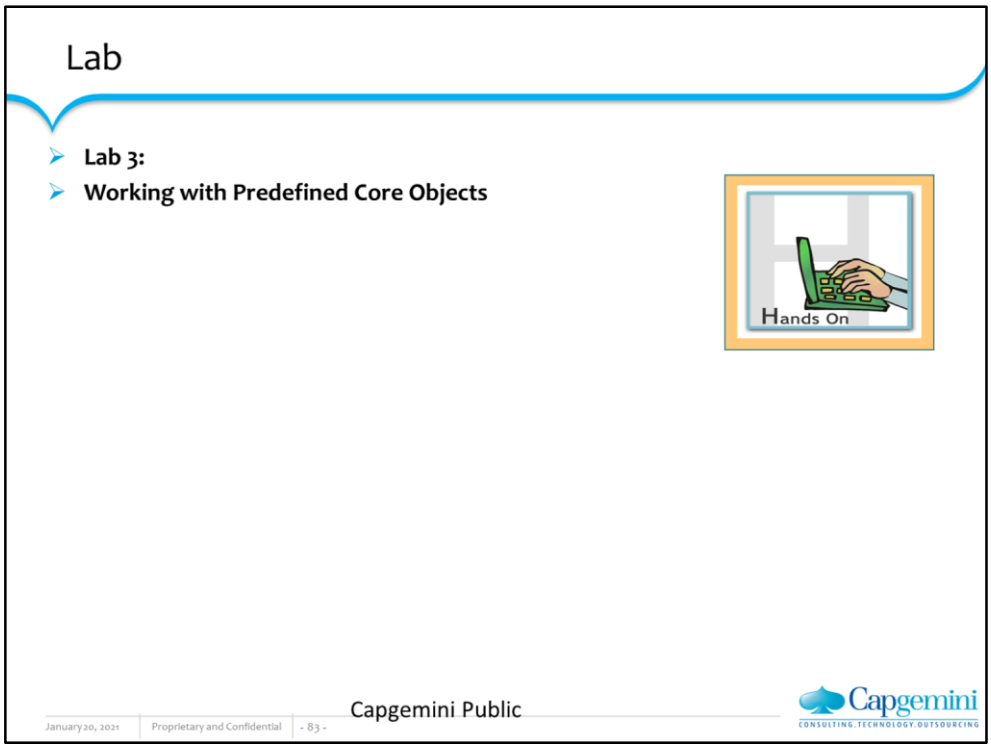
Instructor Notes:

Demo

- Date_info.html
- DateDifference.html
- Utc_ex.html

Add the notes here.

Instructor Notes:



Add the notes here.

Instructor Notes:

Summary

- Predefined objects of JavaScript like String, Math and Date
- How to use predefined objects
- How to manipulate their properties and invoke methods

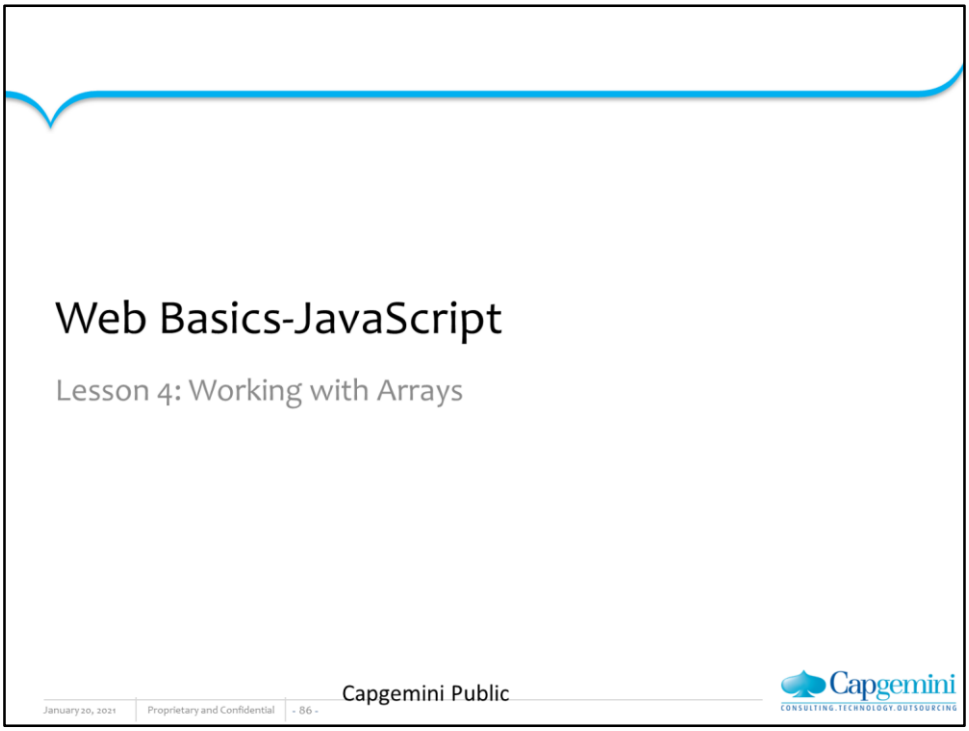
Instructor Notes:

Answers:
Question 1: option 1
Question 2: False
Question 3: escape

Review Question

- **Question 1: Which is the method to extract a single character from a string when you know the position of that character.**
  - Option 1: string.charAt()
  - Option 2: string.charAtIndex()
- **Question 2: getDate() returns the day within the month.**
  - True/False
- **Question 3: To convert a plain string to one with these escape codes, use the _____ function.**

Instructor Notes:

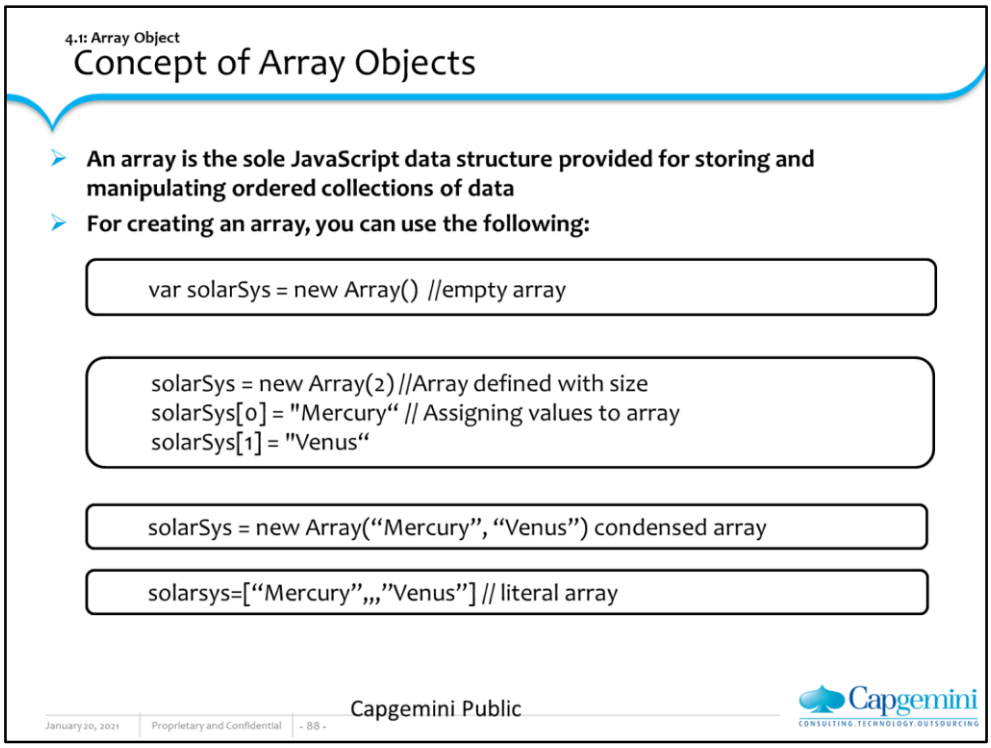


Instructor Notes:

Lesson Objectives

- **In this lesson, you will learn about:**
  - Array object
  - Properties and methods of Array objects

Instructor Notes:



Array Objects:

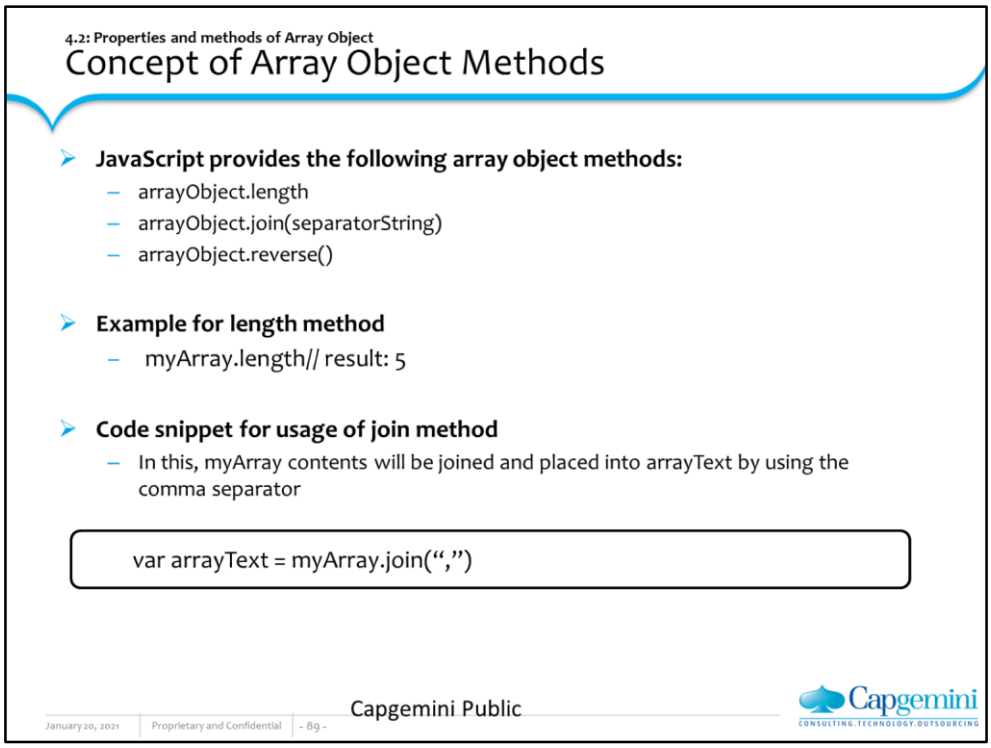
An **array** is a kind of variable that can hold more than one value at a time. However, unlike some other programming languages, JavaScript's arrays are very forgiving as to the kind of data you store in each cell or entry of the array. This allows, for example, an array of arrays, providing the equivalent of multidimensional arrays customized to the kind of data your application needs.

You can see a few examples listed on the slide for creating an array. We see example of creating an empty array. To limit the size of array you can specify the optional integer value as seen in the second example.

Another way of defining an array is called as condensed array which allows you to combine the array and array elements definitions into one step.

Literal arrays are define by assigning the value in square brackets. To create an array with initial undefined values you can simple enter a comma. For eg myarray=[“Pune”,, , “Mumbai]

Instructor Notes:



Array Object Methods:

After you have information stored in an array, JavaScript provides several methods to help you manage that data.

arrayObject.join(separatorString)

It returns string of entries from the array delimited by the separatorString value.

```
var arrayText = myArray.join(“,”)
```

arrayObject.reverse()

It returns array of entries in the op.

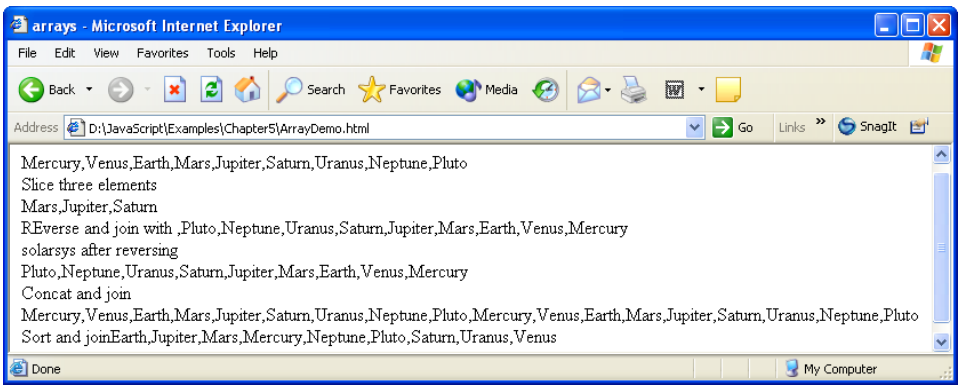
The element that was last in the array becomes the 0 index item in the array. Note that when you do this, you are restructuring the original array, and not copying it.

Instructor Notes:

Demo

- Arr_demo.html

Demo of Array object (Arr_Demo.html) produces the output as shown below:



Instructor Notes:

Lab

- Lab 4 :
- Working with Arrays

Instructor Notes:

Summary

- An array is a set of variables
- You can create an array object by using new operator
- Array Object properties are length
- Array Object Methods are concat, join, reverse, slice etc

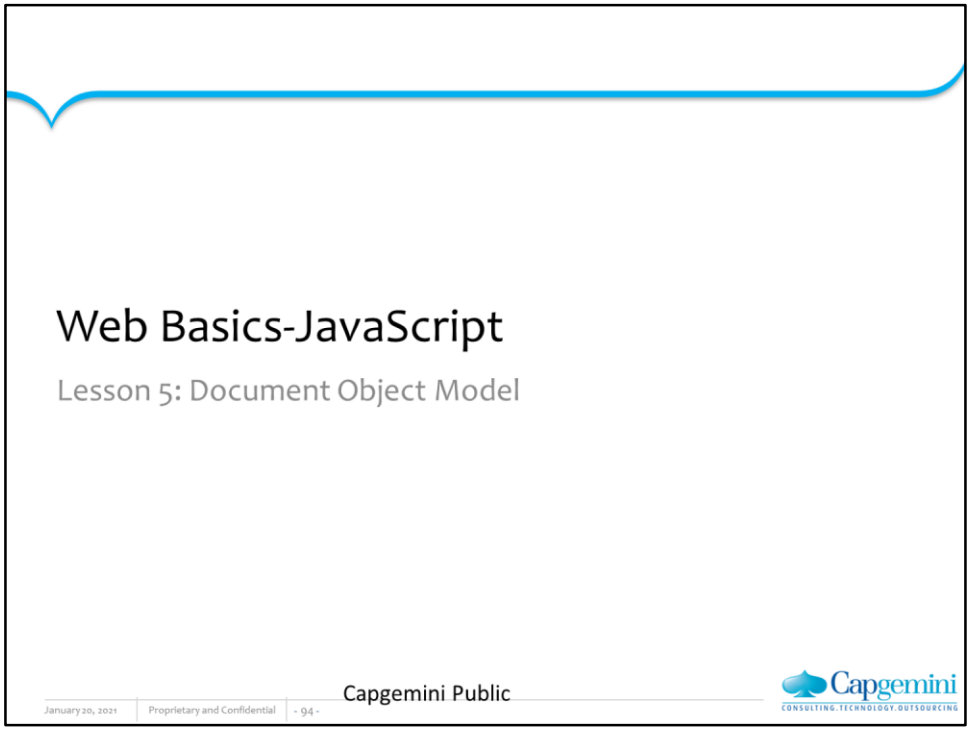
Instructor Notes:

Answers:
Question 1: option 2
Question 2: False

Review Question

- **Question 1: The ___ method allows you to join array contents and place it into a text**
  - Option 1: array.concat()
  - Option 2: array.join()
- **Question 2: An array object automatically has a size property.**
  - True/False

Instructor Notes:

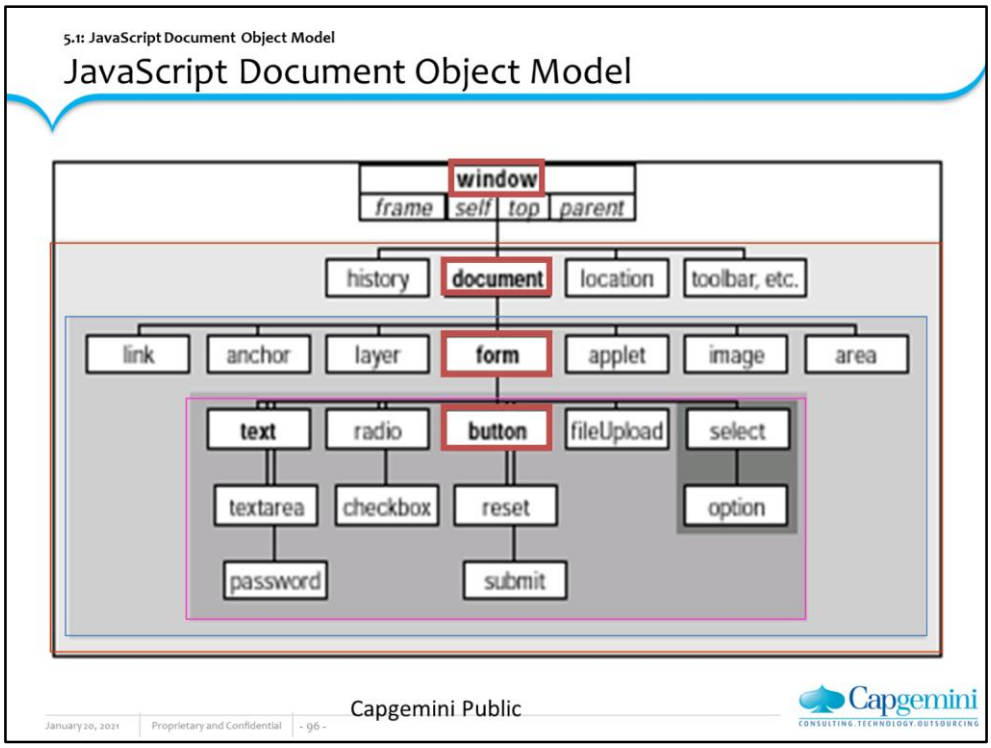


Instructor Notes:

Lesson Objectives

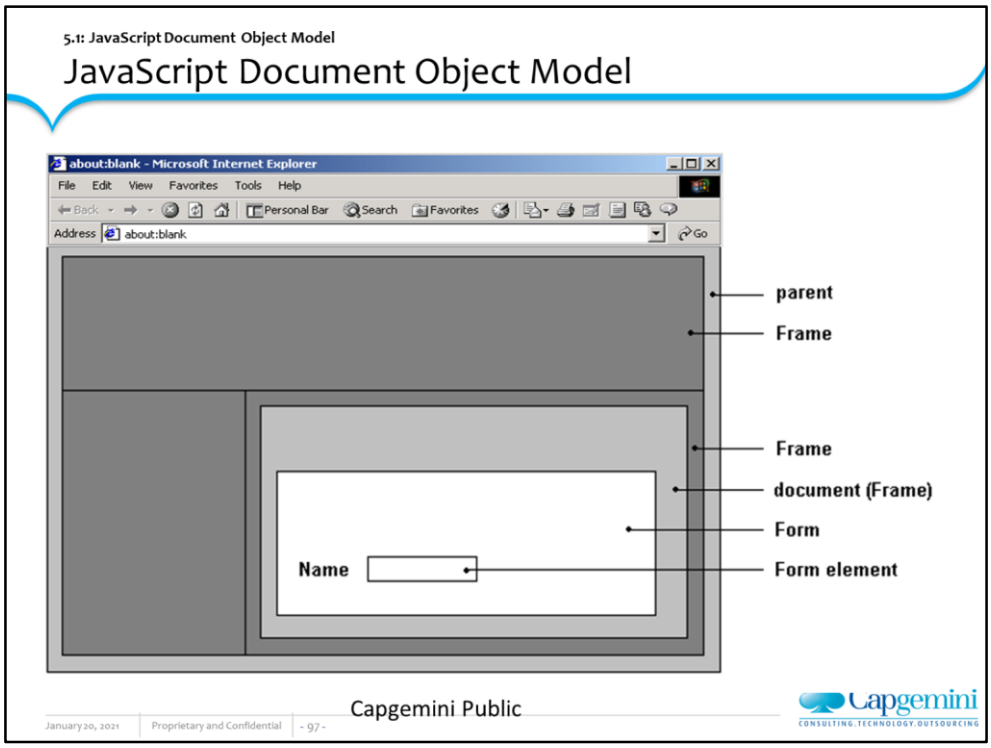
- **After completing this module you will be able to:**
  - Understand the JavaScript Object Model
  - Understand the *Window* object and Navigator Object
  - Working with Location and History Object

Instructor Notes:



The figure shows the complete JavaScript document object hierarchy as implemented in Netscape Navigator 4. Notice that the window object is the topmost object in the entire scheme. Everything you script in JavaScript is in the browser's window, be it the window itself or a form element. Of all the objects shown in the figure, you are likely to work most with the ones that appear in **boldface**. Objects whose names appear in *italics* are synonyms for the window object, and are used only in some circumstances. Pay attention to the shading of the concentric rectangles. Every object in the same shaded area is at the same level relative to the window object. When a link from an object extends to the next darker shaded rectangle, that object contains all the objects in darker areas. There exists at most one of these links between levels. A window object contains a document object; a document object contains a form object; a form object contains many different kinds of form elements. Study this figure to establish a mental model for the scriptable elements of a Web page. After you script these objects a few times, the object hierarchy will become second nature to you — even if you do not remember every detail (property, method, and event handler) of every object. At least you know where to look for information.

Instructor Notes:

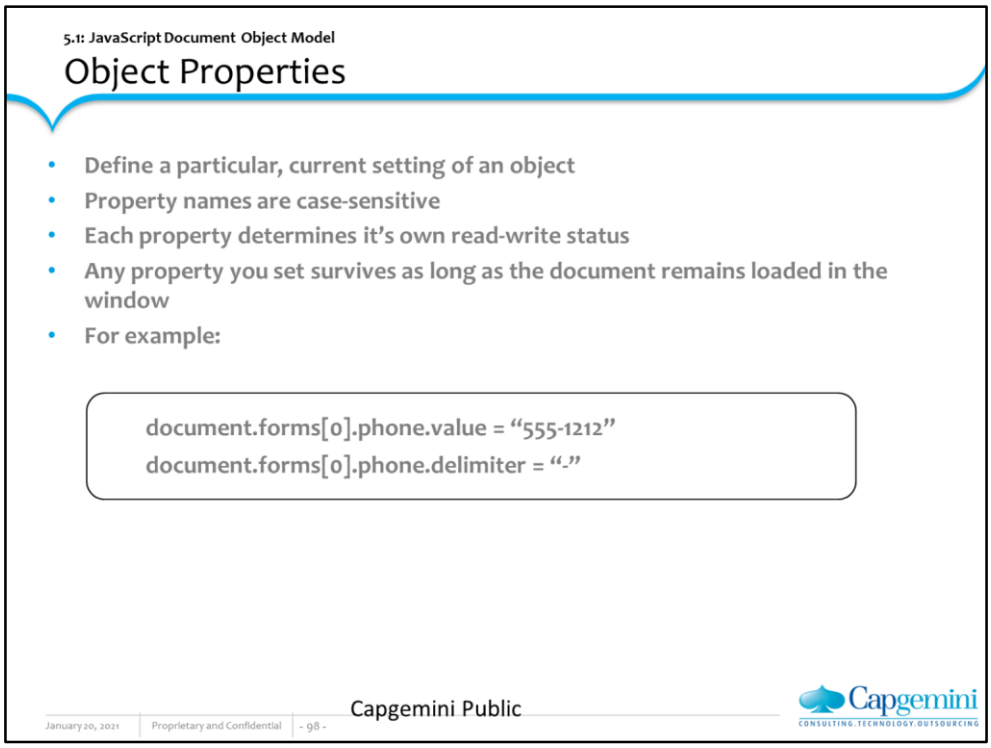


Creating JavaScript Objects

Most of the objects that a browser creates for you are established when an HTML document loads into the browser. The same kind of HTML code you used to create links, anchors, and input elements tell a JavaScript-enhanced browser to create those objects in memory. The objects are there whether or not your scripts call them into action.

The only visible differences to the HTML code for defining those objects are one or more optional attributes specifically dedicated to JavaScript. By and large, these attributes specify the event you want the user interface element to react to and what JavaScript should do when the user takes that action. If you rely on the document's HTML code to perform the object generation, you spend more time figuring out how to do things with those objects or have them do things for you. Bear in mind that objects are created in their load order, which is why you should put most, if not all, deferred function definitions in the document's Head. If you create a multi-frame environment, a script in one frame cannot communicate with another frame's objects until both frames load.

Instructor Notes:



Object Properties

A property generally defines a particular, current setting of an object. The setting may reflect a visible attribute, such as a document's background color. It may also contain information that is not so obvious, such as the form *action* and *method* when it is submitted.

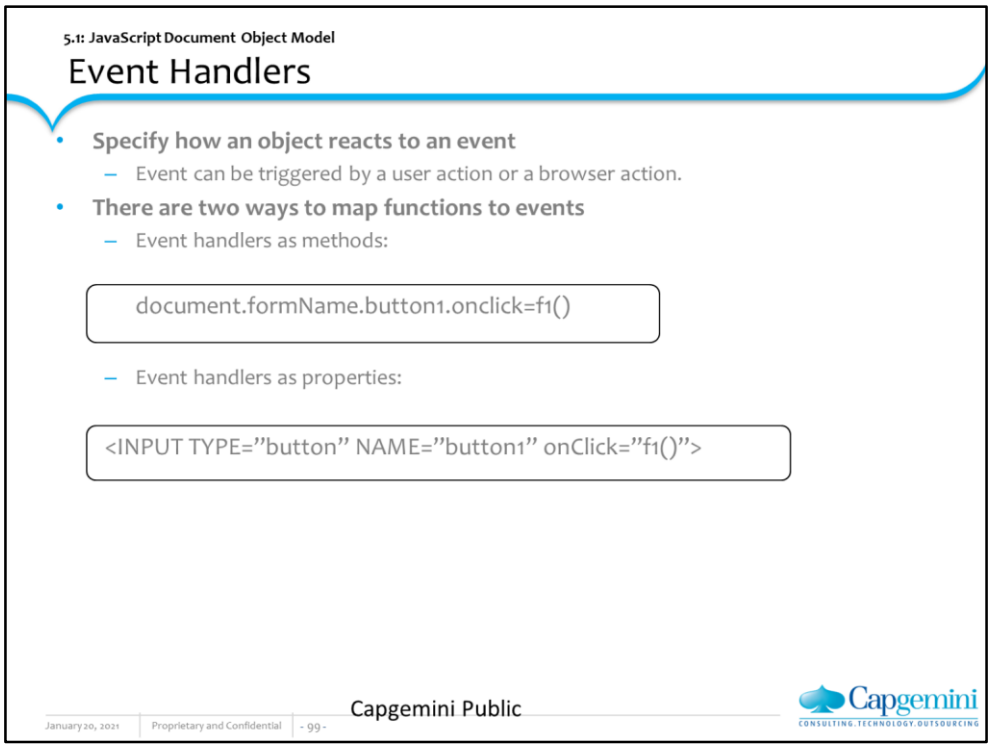
Document objects have most of their properties assigned by attribute settings of HTML tags that generate the objects. Thus, a property may be a string (for example, a name) or a number (for example, a size). A property can also be an array, such as an array of images contained by a document. If the HTML does not include all attributes, the browser usually provides default value for both attributes and corresponding JavaScript properties.

When used in script statements, property names are case-sensitive. Therefore, if you see a property name listed as *bgColor*, you must use it in a script statement with that exact case usage. But when you set an initial value of a property by way of an HTML attribute, the attribute name (like all of HTML) is not case-sensitive. Thus, **<BODY BGCOLOR="white">** and **<body bgcolor="white">** both set the same property value.

Object Methods

An object's method is a command that a script can give to that object. Some methods return values, but that is not a prerequisite for a method. Also, not every object has methods defined for it. In a majority of cases, invoking a method from a script causes some action to take place. It may be an obvious action, such as resizing a window, or something more subtle, such as processing a mouse click.

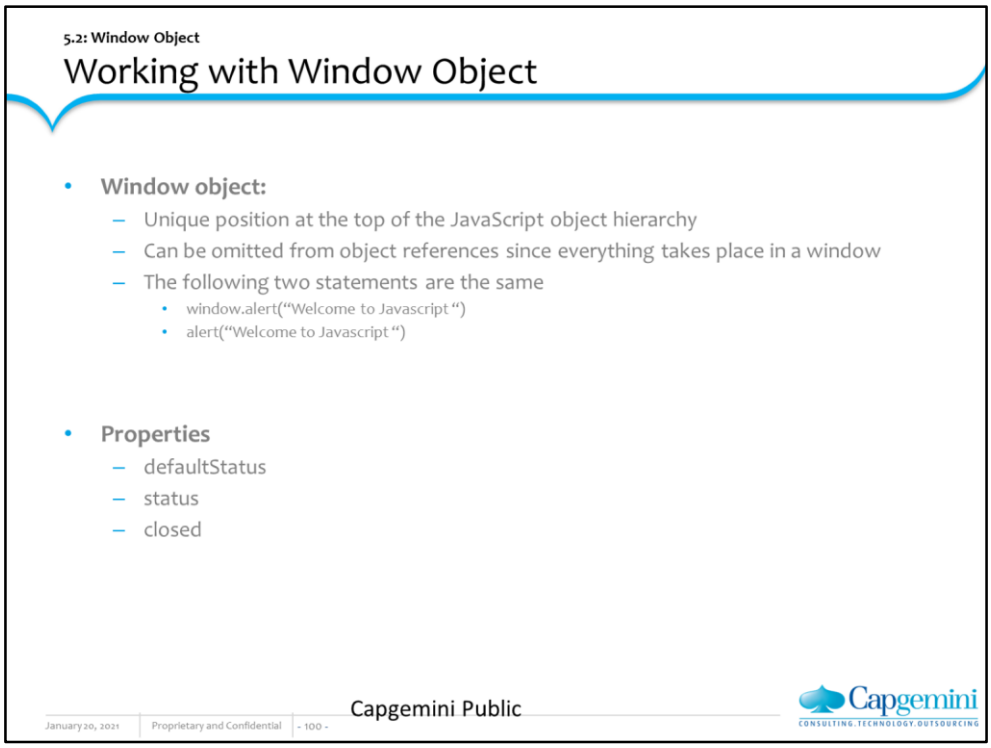
Instructor Notes:



Object Event Handlers

Event handlers specify how an object reacts to an event, whether the event is triggered by a user action (for example, a button click) or a browser action (for example, the completion of a document load). Event Handlers can be specified as methods or they can be specified using attributes in tags.

Instructor Notes:

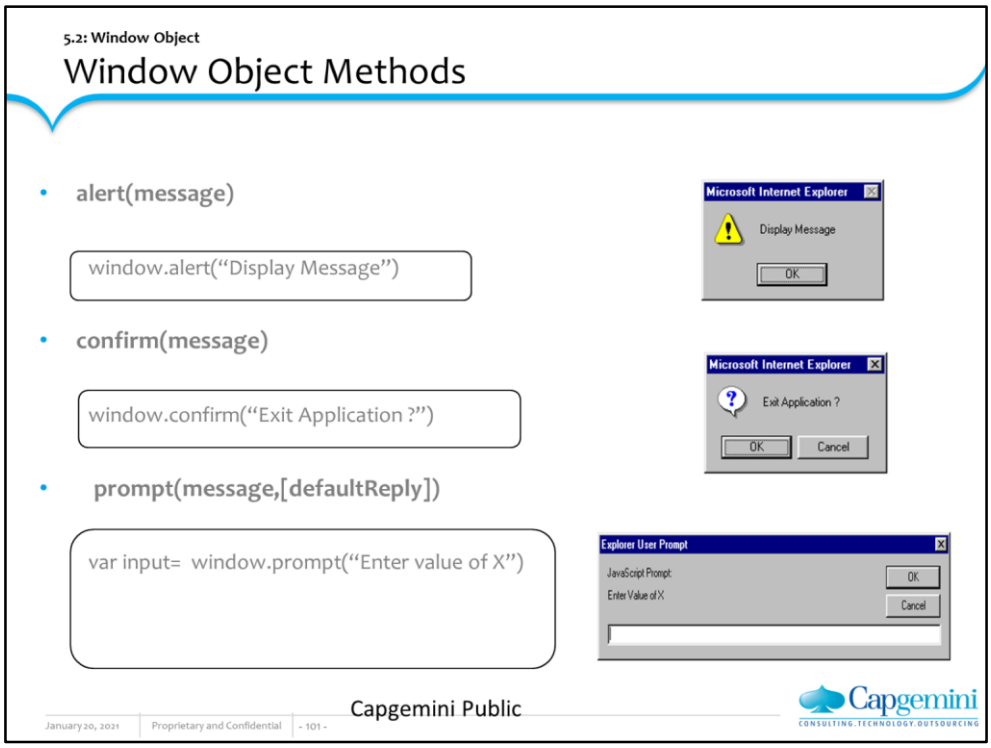


About this Object

The *window* object has the unique position of being at the top of the JavaScript object hierarchy. This exalted location gives it a number of properties and behaviors unlike any other object. Among the list of properties for the window object is one called *self*. This property is synonymous to the window object itself. When you start your browser, it usually opens a window. That window is a valid window object, even if it is blank. This object is also the level at which a script asks the browser to display any of the three styles of the dialog boxes (a plain alert dialog box, an OK-Cancel confirmation dialog box, or a prompt for user text entry).

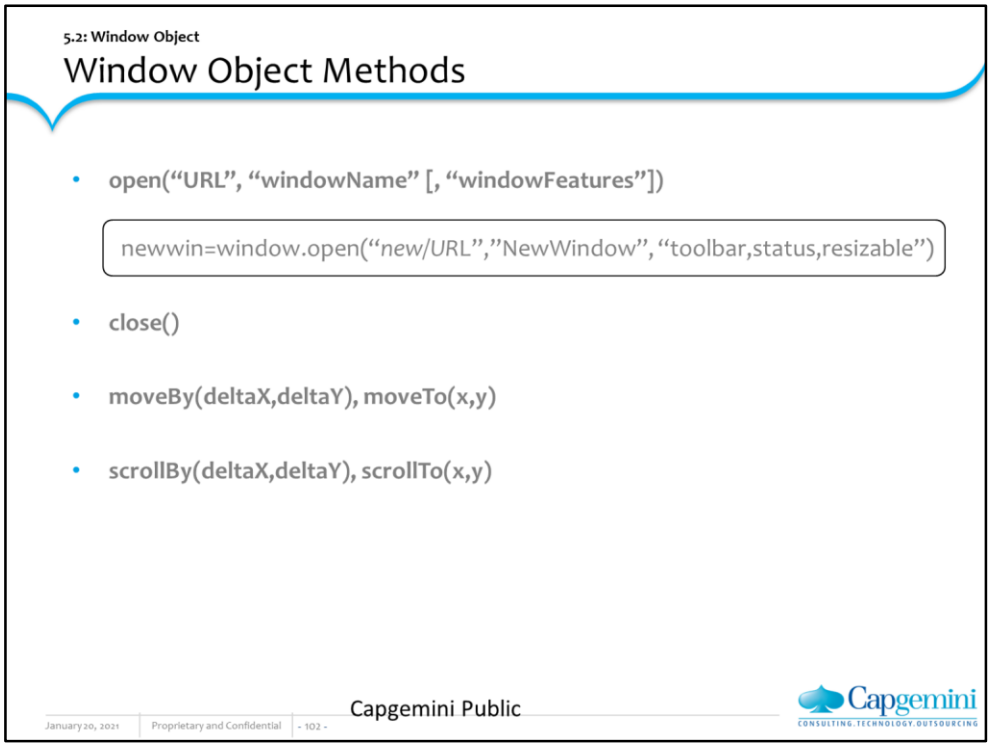
Property	Description
defaultStatus	window.defaultStatus property is normally an empty string, it sets or returns the default text which is in the statusbar of the window
Status	This property sets a text value to be displayed in the status bar
closed	Returns a boolean value which indicated if the window has been closed or no

Instructor Notes:



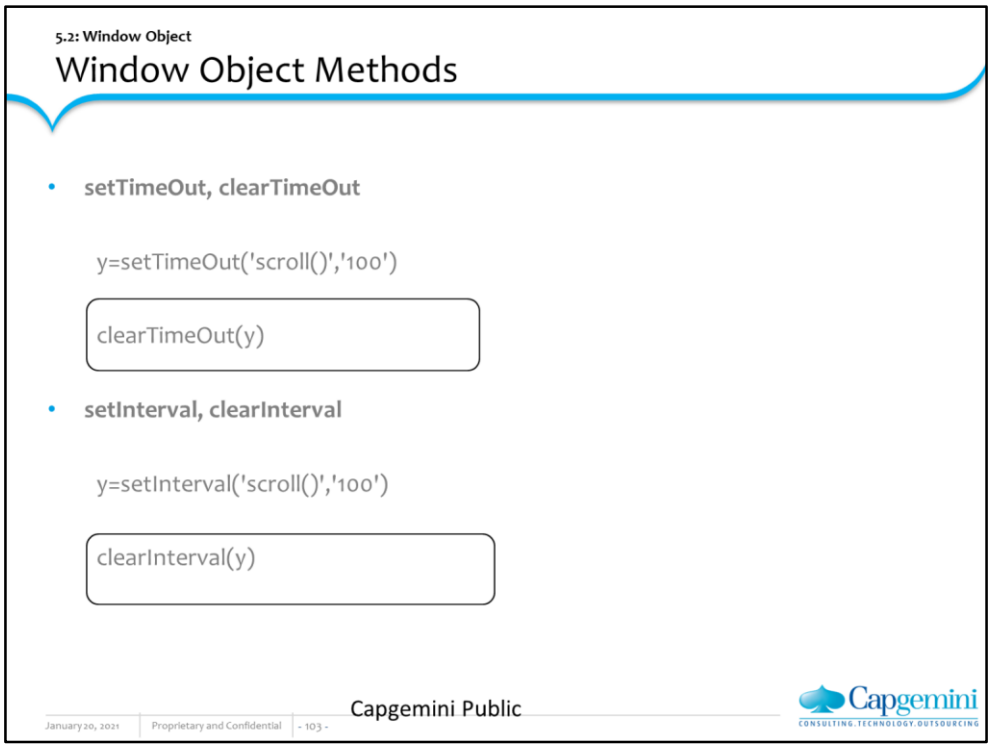
Method	Description
alert(message)	An alert dialog box is a modal window that presents a message to the user with a single OK button to dismiss the dialog box.
confirm(message)	A confirm dialog box presents a message in a modal dialog box along with OK and Cancel buttons. Such a dialog box can be used to ask a question of the user, usually prior to a script performing actions that will not be undoable.
prompt(message, defaultReply)	The third kind of dialog box that JavaScript can display includes a message from the script author, a field for user entry, and two buttons (OK and Cancel).

Instructor Notes:



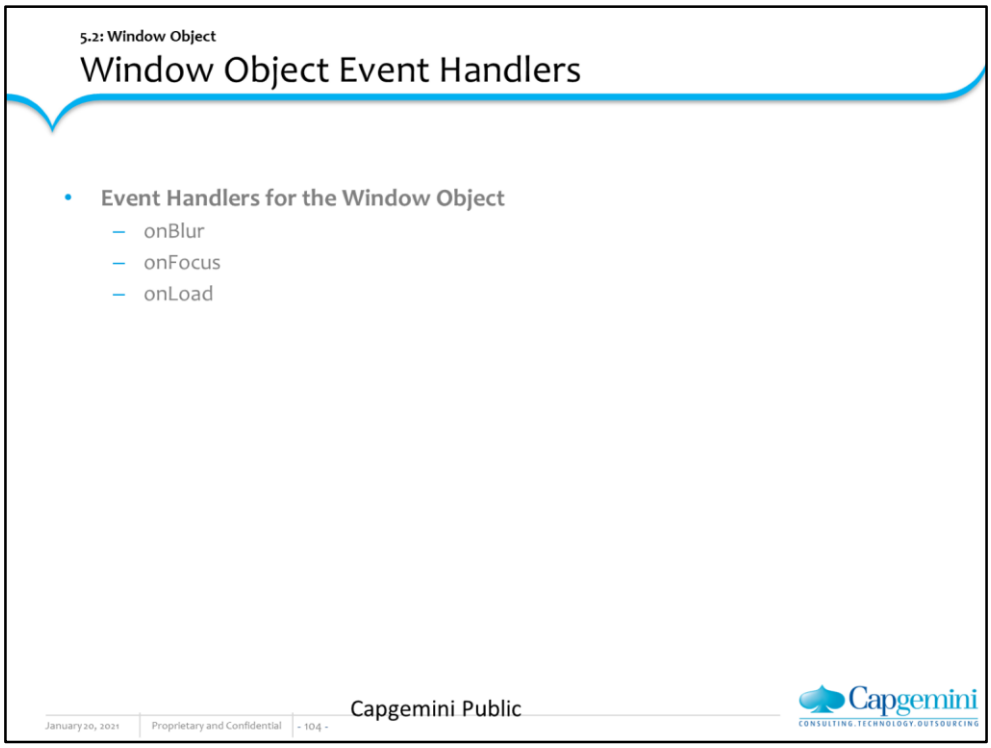
open("URL", "windowName" [, "windowFeatures"])	The *window.open()* method, provides a Web site designer with options for the way a new brwoser window should look on the user's computerscreen. The optional *windowFeatures* parameter is *one string*, that comprises a comma-separated list of assignment expressions. Boolean values for true can be either yes, 1, or just the feature name by itself; for false, use a value of no or 0. If you omit any Boolean attributes, they are rendered as false. Therefore, if you want to create a new window that shows only the toolbar and statusbar and is resizable, the method looks like this: window.open("*newURL*","NewWindow", "toolbar,status,resizable").
close()	The *window.close()* method closes the browser window referenced by the window object.
scrollBy(deltaX,deltaY) scrollTo(x,y)	scrollBy(..) method scrolls the content by the specified number of pixels which is relative scroll. scrollTo(..) is an absolute scroll to the specified coordinates
moveBy(deltax,delta Y) moveTo(x,y)	moveBy(..) moves the window relative to the current position. moveT(..) moves it to the specified coordinates

Instructor Notes:



setTimeout(" functionOrExpr", msecDelay [, funcarg1, ..., funcargn])	Javascript holds a statement or function from executing for the desired amount of time. The timeout value is in milliseconds
setInterval("functionOrExpr", msecDelay,language)	Use this method when your script needs to call a function or execute some expression repeatedly with a fixed time delay between calls to that function or expression. The timeinterval is in milliseconds. Optional Language i.e Javascript,vbscript
clearInterval (intervalIDnumber)	Use this method to turn off an interval loop action started with the *window.setInterval()* method. The parameter is the ID number returned by the *setInterval()* method.
clearTimeout (timeoutIDnumber)	Use the *clearTimeout()* method in concert with the *window.setTimeout()* method when you want your script to cancel a timer that is waiting to run its expression. The parameter for this method is the ID number that the *setTimeout()* method returns when the timer starts ticking.

Instructor Notes:



Event Handlers

Table 6.3 Window Object Event Handlers

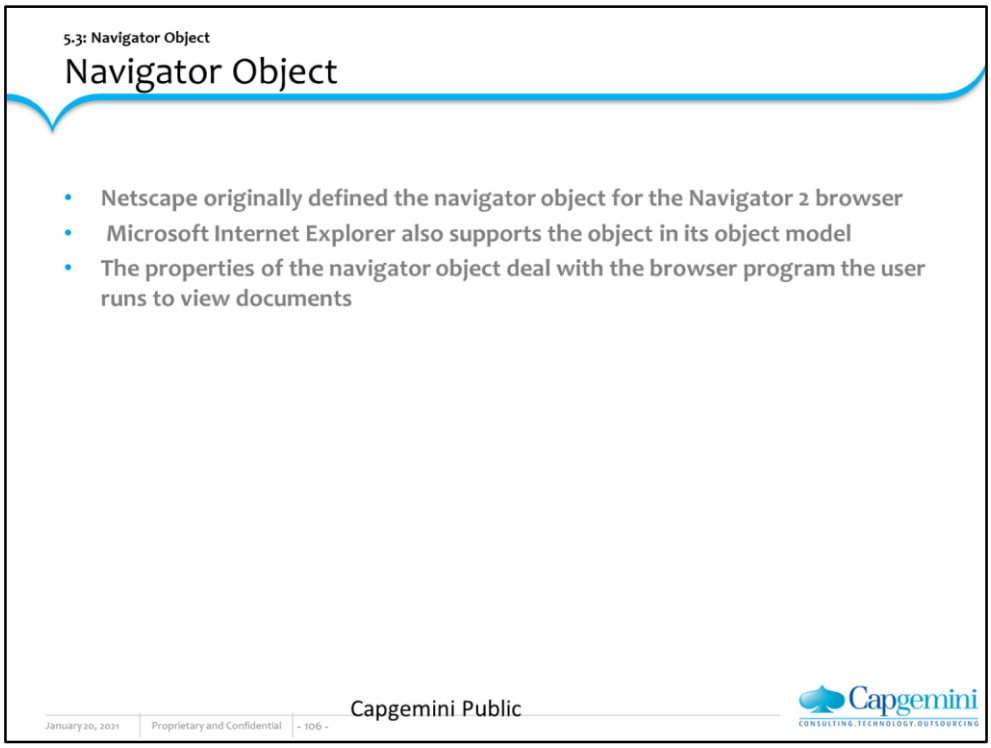
Event Handler	Description
onBlur onFocus	Fired when window or frame has been activated and deactivated respectively.
onLoad	The load event is sent to the current window at the end of the document loading process.

Instructor Notes:

Demo

- Window_object.html
- setTimeOut_method.html
- Window_ex.html
- setInterval_method.html

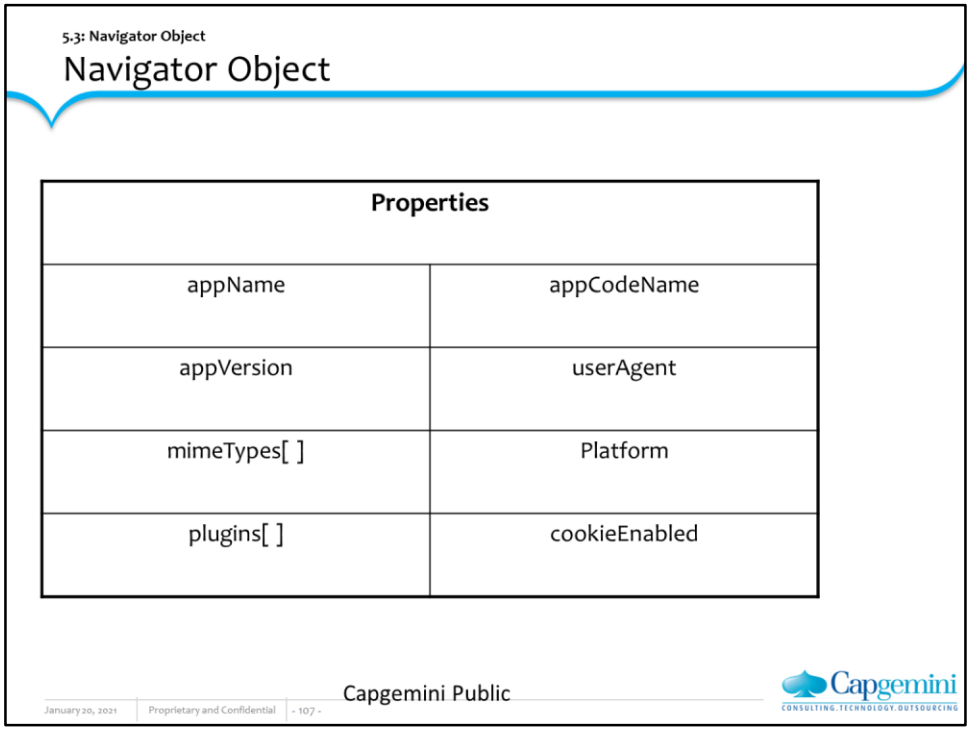
Instructor Notes:



Navigator Object

Netscape originally defined the navigator object for the Navigator 2 browser. Microsoft Internet Explorer also supports the object in its object model. Properties of the navigator object deal with the browser program the user runs to view documents. Properties include those for extracting the version of the browser and the platform of the client running the browser.

Instructor Notes:



Property	Description
appName appCodeName appVersion userAgent Platform cookieEnabled	The *appName* and *appCodeName* properties are simply the official name and the internal code name for the browser. *appVersion* returns version information of the browser and *userAgent* returns the user-agent header sent by the browser to the server. Platform returns for which platform the browser is compiled. cookieEnabled determines if cookies are enabled in the browser
plugins[]	Returns an array of plugins available on the client browser.
mimeTypes[]	Returns an array of MIME types supported by the browser

Instructor Notes:

Demo

- property_bro_ver_name.html
- plugin_object.html
- MIME_Type.html

Instructor Notes:

Lab

- Lab 5 :
- Working with Document Object Model (DOM)

Instructor Notes:

Summary

- Document Object Model is a interface that allows programs and scripts to dynamically access and update content, structure and style of documents
- Window object is the topmost object in the entire scheme. It has properties, methods and event handlers
- **The history object has an array of history items having details of the URL's visited from within that window**
- **The Location object contains information about the current URL**

Instructor Notes:

Answers

1. Option 2
2. True

Review Questions

- **Question 1: Closed property returns _____ if the window object is closed either by a script or by the user.**
  - Option 1: 1
  - Option 2: True
  - Option 3: 0
- **Question 2: An alert dialog box is a modal window that presents a message for users with a single OK button to dismiss it.**
  - True / False

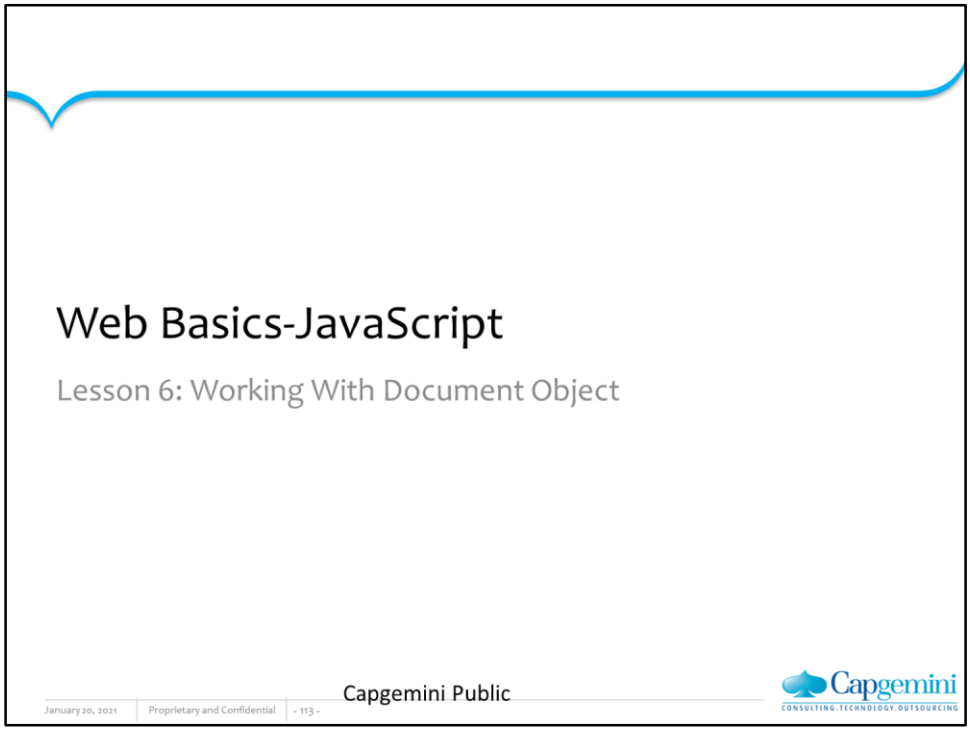
Instructor Notes:

Answers
4. Option 2
5. Option 1
6. host

Review Questions (Contd..)

- Question 4: The _____ and appCodeName properties are simply the official name and the internal code name for the browser application.
  - Option 1: Appname
  - Option 2: appName
  - Option 3: applname
- Question 5: The _____ property supplies a string of the entire URL of the specified window object.
  - Option 1: location.href
  - Option 2: hostname
  - Option 3: hash
- Question 6: The _____ property describes both the hostname and port of a URL.

Instructor Notes:

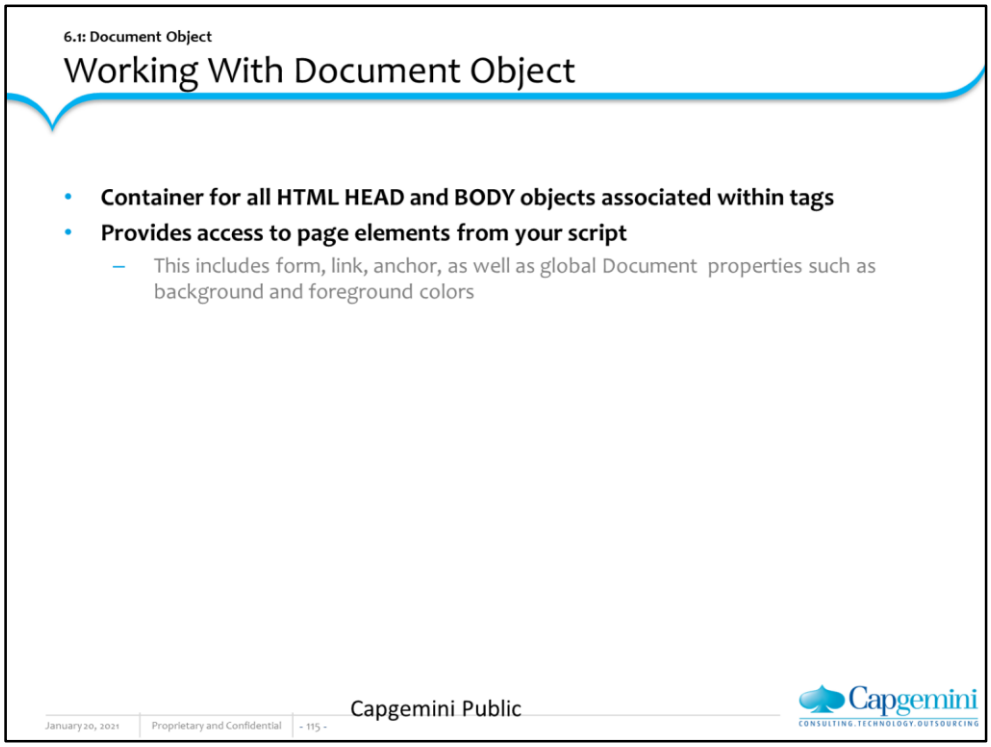


Instructor Notes:

Lesson Objectives

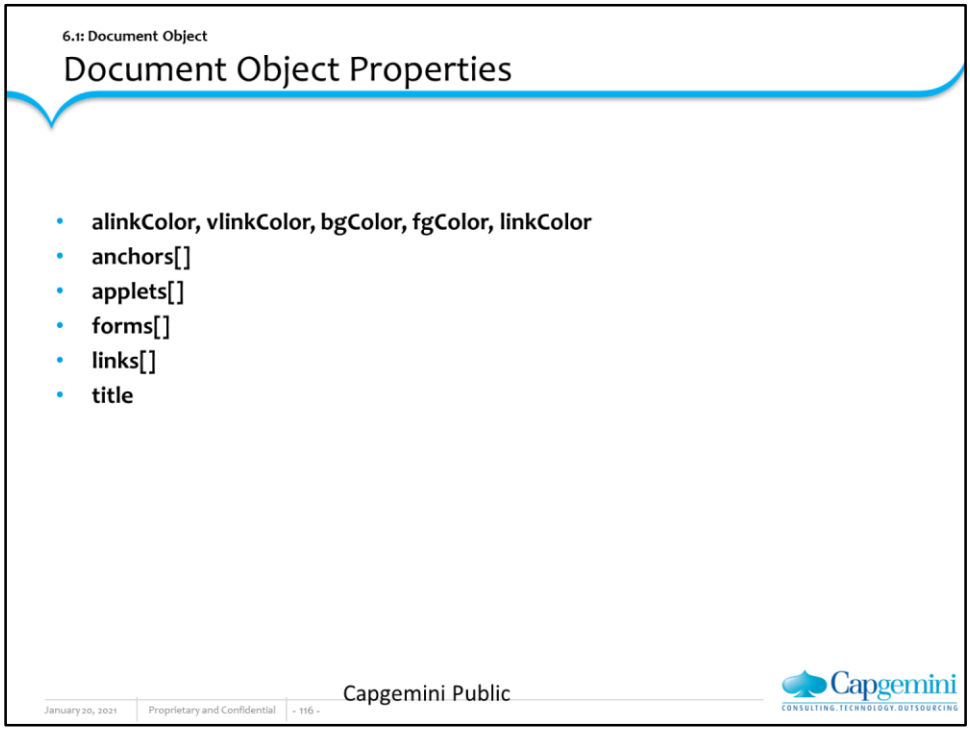
- **To understand the following topics:**
  - Document Object and its properties and methods
  - Cookies object

Instructor Notes:



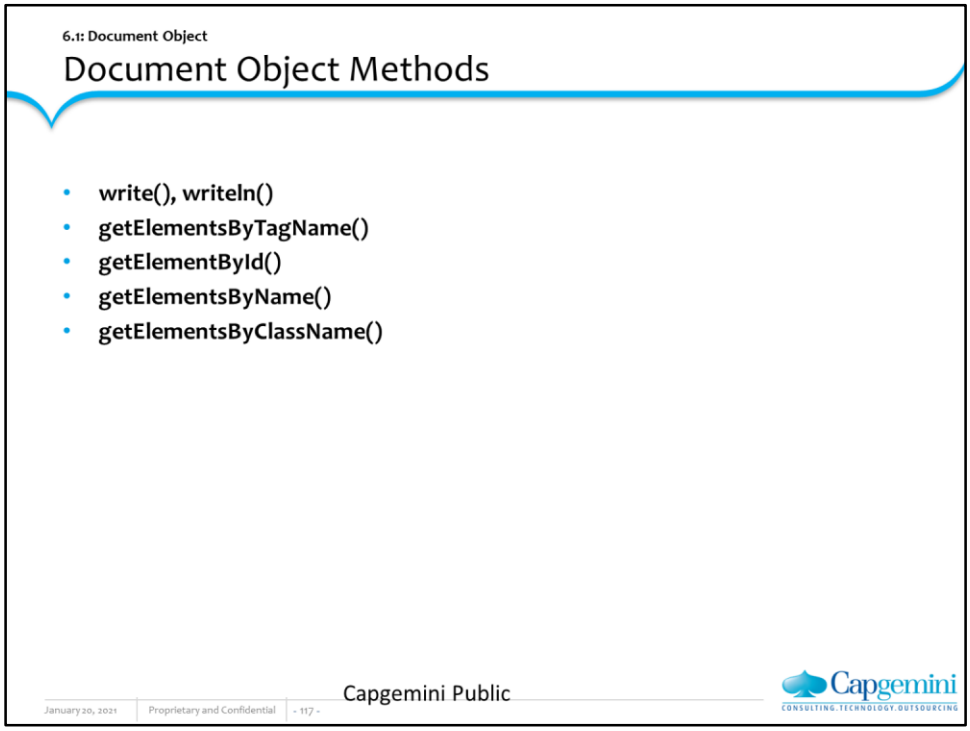
Document object is part of the Window object. It is used to access all elements in a page. It provides access to the elements in an HTML page from within the script. This includes the properties of every form, link and anchor (and, where applicable, any sub-elements), as well as global document properties such as background and foreground colors.

Instructor Notes:



Property	Description
alinkColor vlinkColor bgColor fgColor linkColor	Get and set the properties of document – activated link, visited link, background color, foreground color (text) and hyperlink color.
anchors[], forms[], links[]	These properties retrieve array of values respectively as present in the document object
title	Gets the title of the document which occurs between the TITLE tags.

Instructor Notes:



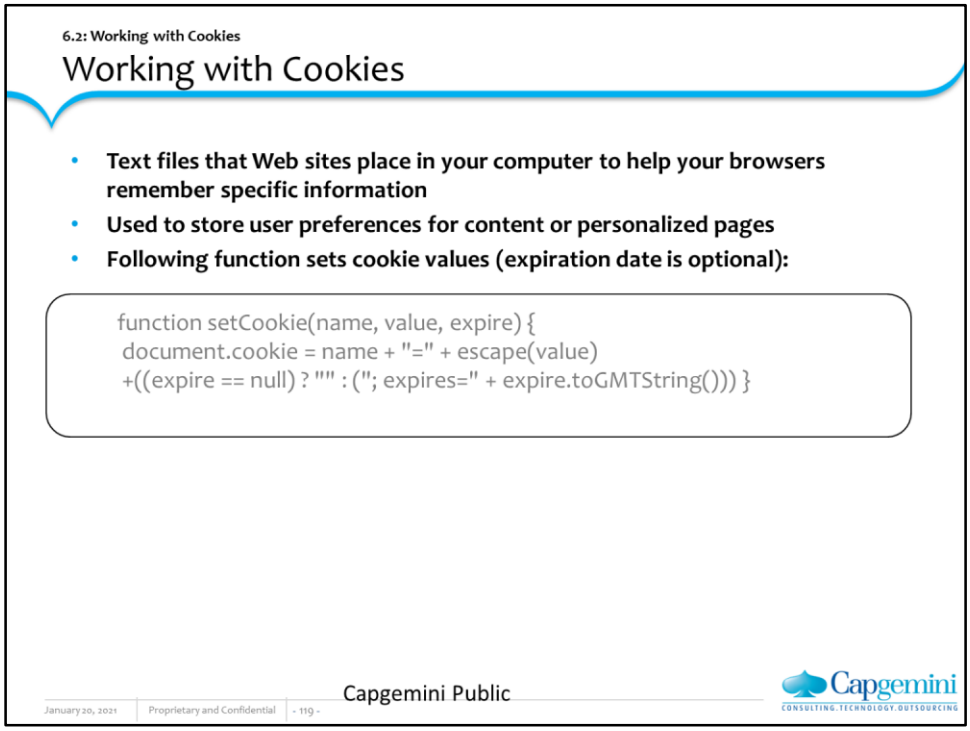
Property	Description
write("string1", …) writeln("string1", ..)	Both of these methods send text to a document for display in its window. The only difference between the two methods is that *document.writeln()* appends a carriage return to the end of the string it sends to the document (you must still write a to insert a line break).
getElementById("#para1")	This method locates the element whose id has been passed. The text within this element can then be accessed using properties innerHTML or innerText
getElementsByTagName("p")	This method locates all the elements which match the tagname passed. Each element of this type of tag can then be accessed in an array like manner
getElementsByName()	This method locates all the elements which match the name passed. Same name to many elements is usually given for radio buttons.
getElementsByClass()	This method locates all the elements which match the class name passed.

Instructor Notes:

Demo

- Link_Anchor_object.html
- Meta_information.html
- locate_element_by_id.html
- locate_elements_by_tagname.html
- locate_elements_by_name.html
- locate_element_by_class_name.html

Instructor Notes:



Using Cookies

Cookies are a mechanism for storing persistent data on the client in a file called *cookies.txt*. Because HyperText Transport Protocol (HTTP) is a stateless protocol, cookies provide a way to maintain information between client requests. This section discusses basic uses of cookies and illustrates with a simple example.

Each cookie is a small item of information with an optional expiration date and is added to the cookie file in the following format:

name=value;expires=expDate;

Name is the name of the datum being stored, and value is its value. If *name* and *value* contain any semicolon, comma, or blank (space) characters, you must use the *escape* and *unescape* functions to encode and decode them respectively.

expDate is the expiration date, in GMT date format:

Wdy, DD-Mon-YY HH:MM:SS GMT

Although it is slightly different from this format, the date string returned by the *Date* method *toGMTString* can be used to set cookie expiration dates.

The expiration date is an optional parameter indicating how long to maintain the cookie. If *expDate* is not specified, the cookie expires when the user exits the current browser session. Browser maintains and retrieves a cookie only if its expiration date has not yet passed.

Instructor Notes:

Limitations

Cookies have these limitations:

300 total cookies in the cookie file.

4 Kbytes per cookie, for the sum of both the cookie's name and value.

20 cookies per server or domain (completely specified hosts and domains are treated as separate entities and have a 20-cookie limitation for each, not combined).

Cookies can be associated with one or more directories. If your files are all in one directory, then you need not worry about this. If your files are in multiple directories, you may need to use an additional path parameter for each cookie.

Using Cookies with JavaScript

The *document.cookie* property is a string that contains all names and values of Navigator cookies. Use this property to work with cookies in JavaScript.

Instructor Notes:

Demo

- Ch9_ex1.cookie.html
- Demo_Cookies.html

Instructor Notes:

Lab

- **Lab 7:**
- **Working with Document object.**

Instructor Notes:

Summary

- JavaScript Document Object contains HTML elements contained in the <head> and <body> sections of a web page
- All the anchors are contained in anchor array.
- All the links are contained in link array
- Cookies are small text files stored on the site visitor's computer by their browser

Summary

In this chapter, you understood:

- DOM structure
- How to work with Document Object
- How to work with cookies

Instructor Notes:

Answers

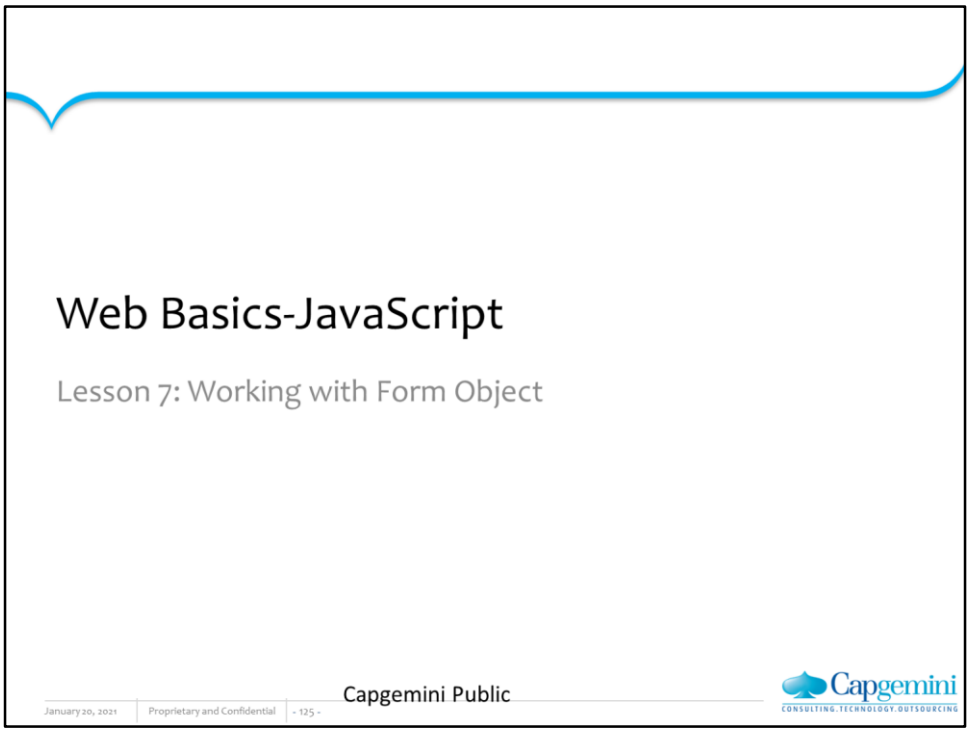
1. Option 1
2. anchors[]

Review Questions

- **Question 1: The _____ is the container for all HTML HEAD and BODY objects.**
  - Option 1: Document
  - Option 2: Object
  - Option 3: Container

- Question 2: _____ property in document object retrieves an indexed array of anchors in a document.

Instructor Notes:

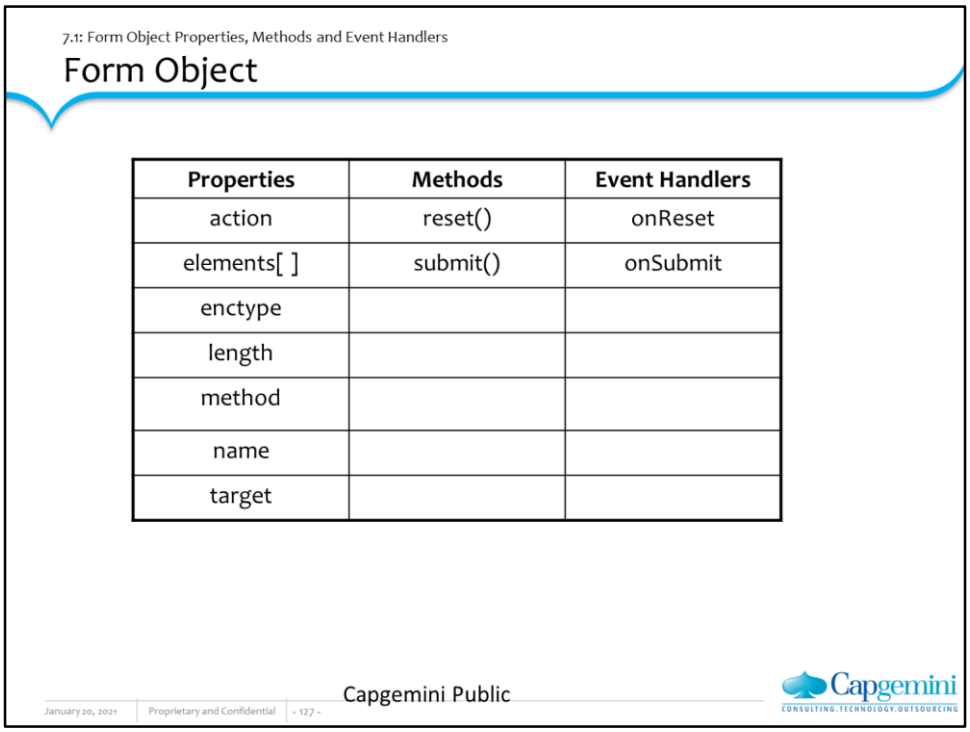


Instructor Notes:

Lesson Objectives

- **To understand the following topics:**
  - Form Object Properties, Methods & Event Handlers
  - Text-Related Objects
  - Button Objects
  - Check Box and Radio Objects
  - Select Objects

Instructor Notes:

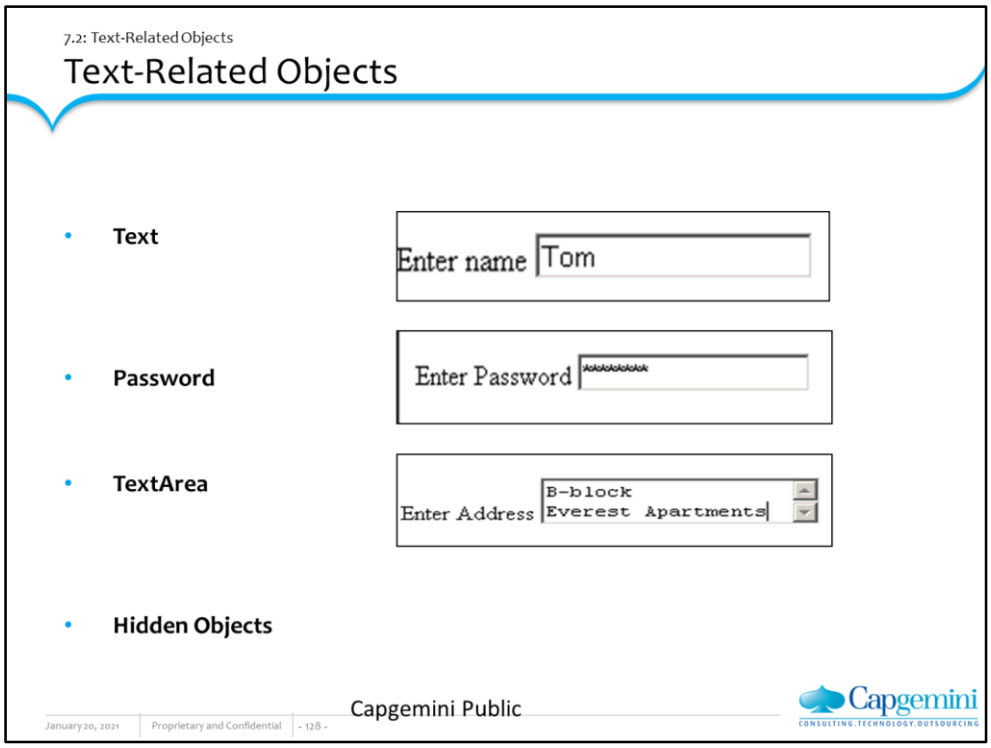


Working with Form Objects: Form Object Properties:

A form element provides the only way that users can enter textual information or make a selection from a predetermined set of choices, whether those choices appear in the form of an on/off checkbox, one of a set of mutually exclusive radio buttons, or a selection from a list.

Property/ Method/ Events	Description
action	This property is the same as the value you assign to the ACTION attribute of a <FORM> tag. The value is typically a URL on the server where queries or postings are sent for submission.
elements[]	Returns an array of elements. It includes all the user interface elements defined for a form: text fields, buttons, radio buttons, checkboxes, selection lists, and more.
encoding	You can define a form to alert a server that the data being submitted is in a MIME type. This property reflects the setting of the ENCTYPE attribute in the form definition. The default value is an empty string.
method	A form's method property is either the GET or POST values assigned to the METHOD attribute in a <FORM> tag.

Instructor Notes:



Text-Related Objects:

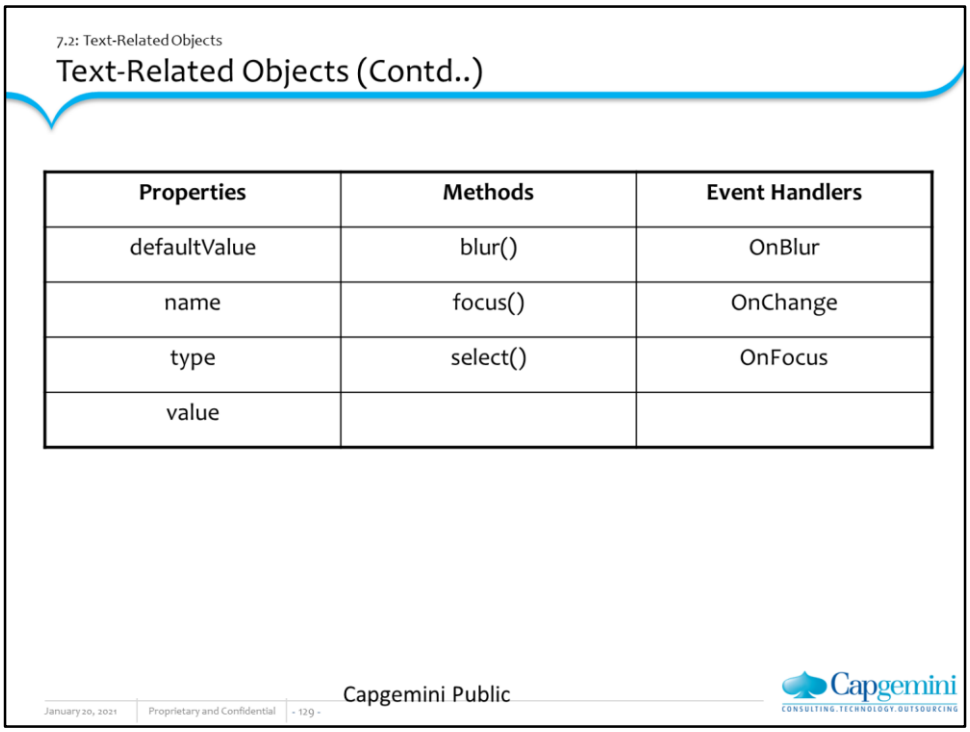
Text Objects : The text object is the primary medium for capturing user-entered text.

Password Object: A password-style field looks like a text object, but when the user types something into the field, only asterisks or bullets (depending on your operating system) appears in the field.

Textarea Object: A textarea object closely resembles a text object, except for attributes that define its physical appearance on the page.

Hidden object: A hidden object is a simple string holder within a form object whose contents are not visible to the user of your Web page. With no methods or event handlers, the hidden object's value to your scripting is as a delivery vehicle for strings that your scripts need for reference values or other hard-wired data.

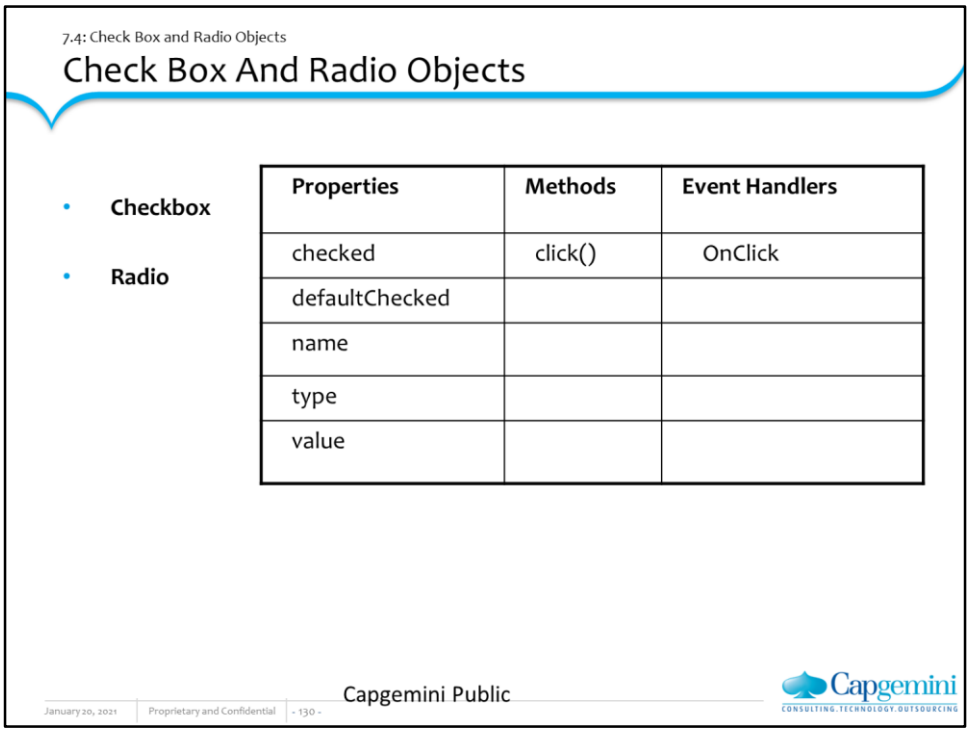
Instructor Notes:



The properties, methods and event handlers are same for text object, text area and Password. For hidden object the properties are same but no methods and event handlers are associated with this object.

Property/ Events/ Methods	Description
defaultValue	Specifies or returns a defaultValue for a text related objects.
name	This property can be used to reference the text object in the script.
type	Returns the type of text related object
value	A reference to an object's value property returns the string currently displayed in the field.

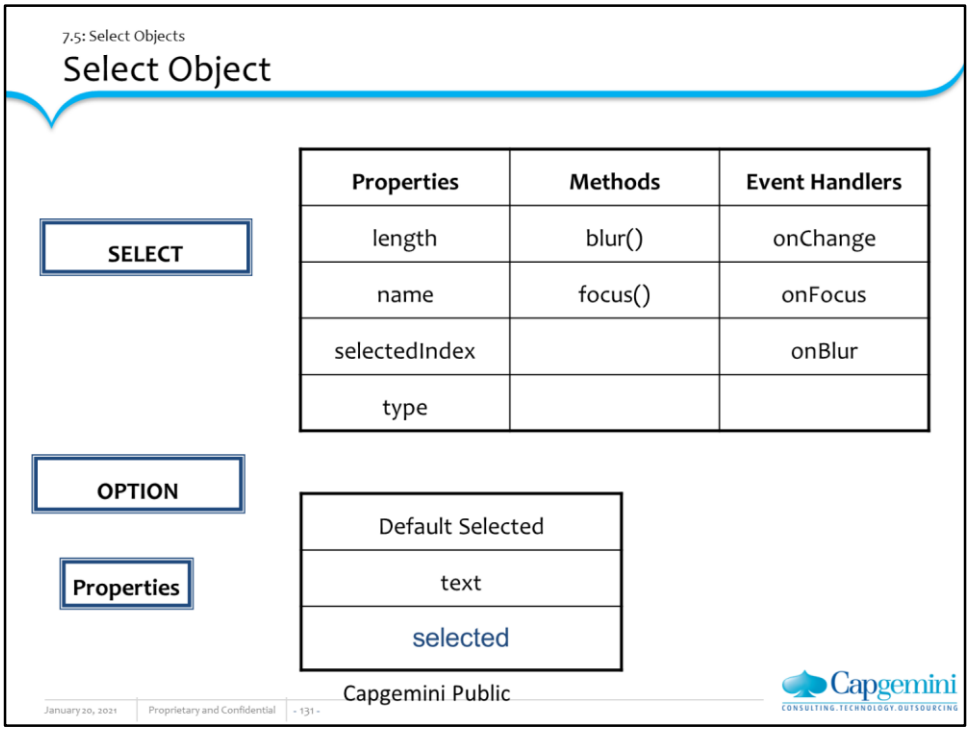
Instructor Notes:



Checkbox object:

Property/ Events/ Methods	Description
checked	The simplest property of a checkbox gets or lets you set whether or not a checkbox is checked. The value is true for a checked box and false for an unchecked box. Only one radio button in a group can be highlighted checked) at a time. That one button's checked property is set to true, whereas all others in the group are set to false.
defaultChecked	If you add the CHECKED attribute to the <INPUT> definition for a checkbox or radio button, the defaultChecked property for that object is true; otherwise, false.
name	The name property allows user to access name for the checkbox or radio button through script.
type	Use the type property to help you identify a checkbox object or a radio button object from an unknown group of form elements.

Instructor Notes:



Property/ Methods/ Events	Description
length	Returns the number of items available in the list. A select object with three choices in it has a length property of 3.
Name	A select object's name property is the string you assign to the object by way of its NAME attribute in the object's <SELECT> tag which can be
selectedIndex	When a user clicks on a choice in a selection list, the selectedIndex property changes to a number corresponding to that item in the list.
type	Use the type property to help you identify a select object from an unknown group of form elements.
blur() focus()	Your scripts can bring focus to a select object by invoking the object's focus() method. To remove focus from an object, invoke its blur() method. These methods work identically with their counterparts in the text object.
onChange	As a user clicks on a new choice in a select object, the object receives a change event that can be captured by the onChange event handler.

Instructor Notes:

Using 'this' keyword
The 'this' keyword can be used to reference the object which called the function. It can be used within a function scope or global scope and it receives a different value in each scope. Depending on which object has called the function the value of 'this' will differ. The 'this' keyword always points to the object that is calling a particular method.
Consider the example given below:

The 'this' keyword is used in the showColor() function of an object. In this context, this is equal to car, making this code functionality equivalent to the following code snippet

So the reason for using 'this' is you never know what kind of variable names you will use to instantiate an object. By using 'this' you are sure to invoke the correct function with the correct value. Also it allows you to use the same function any number of times. To understand this consider the following code:

```
var oCar = new Object;
oCar.color = "red";
oCar.showColor = function () {
    alert(this.color); //outputs "red"
};
```

In the above snippet both oCar1 and oCar2 refer to the same function. The function gives the output according to the object which called the function.

```
var oCar = new Object;
oCar.color = "red";
oCar.showColor = function () {
   alert(oCar.color); //outputs "red"
};
```

```
function showColor() {
      alert(this.color);
}
var oCar1 = new Object;
oCar1.color = "red";
oCar1.showColor = showColor;
var oCar2 = new Object;
oCar2.color = "blue";
oCar2.showColor = showColor;
oCar1.showColor(); //outputs "red"
oCar2.showColor(); //outputs "blue"
```

Instructor Notes:

Demo

- Form_Object.html
- Select_option.html
- Element_array.html
- Enctype.html
- Hidden_value.html

Instructor Notes:

Lab

- **Lab 8:**
- **Working with Form Object**

Instructor Notes:

Summary

- Form Object corresponds to an HTML input form constructed with the FORM tag
- Forms have their own properties, objects, methods & events
- A form can be submitted by calling the JavaScript submit method or clicking the form submit button
- JavaScript can do entry-level validation & do it very easily

Summary

This module provided an understanding of:
Form object and its components.
How to create form objects.
How to handle events.
How to validate data.
How to submit a form.

Instructor Notes:

Answers:

1. Option 1
2. True
3. click()

Review Questions

- **Question 1: A form's _____ property is either the GET or POST values assigned to the METHOD attribute in a <FORM> definition.**
  - **Option 1:** Method
  - **Option 2:** Class
  - **Option 3:** Object

- **Question 2: The intention of the click() method is to enact, via a script, the physical act of clicking a radio button.**
  - True / False

- **Question 3: A button's _____method should replicate, via scripting, the human action of clicking that button.**